# A self-contained reading of the paper statements and their Lean encodings

A typeset rendering of `PaperFormalization/Statements.lean`

| | |
|---|---|
| Source file | `PaperFormalization/Statements.lean` (4,923 lines) |
| SHA-256 | `1dfc9ee1b784cea156bac5490211349cfdaeee0734a42d7fbcf113934e7ebcd3` |
| Lean toolchain | `leanprover/lean4:v4.35.0` |
| Mathlib | `v4.35.0` (`lakefile.toml`) |
| Rendered | 31 October 2026 |

**About this rendering.** Every line of `Statements.lean` appears below, in file order. Documentation comments (`/-! ... -/`) are set as text. Lean code is reproduced character for character, with the file's line numbers in the margin; a ↪ marks where a line too long for the page was wrapped. The paper excerpts, which the file quotes as verbatim TeX, are typeset here with the macros of the `paper.tex` preamble; equation labels are shown as their keys, e.g. (eq:saw), and references to numbered statements by their numbers. Blue text links to definitions (D nnn), glossary entries (G nn) and statements.

paper excerpt | Lean statement (`theorem`) | Lean definition body (D-block) | other Lean code | **keyword** ∀ → ∧ ℕ Finset | linked definition | proof term

# Contents


**File header** . . . 3

**Coverage legend and proof status** . . . 3

**Reading the encodings** . . . 3

**Lean and Mathlib conventions used throughout** . . . 4

Theorem 1 `thm:onb` coverage: direct . . . 7

Theorem 2 `thm:box` coverage: direct . . . 11

Definition 3 `def:equicomp` coverage: direct-definition . . . 16

Proposition 4 `prop:UNmap` coverage: direct . . . 19

Theorem 5 `thm:onbexplicit` coverage: all-degrees-conclusion . . . 22

Lemma 6 `lem:poisson1d` coverage: direct . . . 27

Lemma 7 `lem:fouriermass` coverage: direct . . . 29

Definition 8 `def:shiftedbasis` coverage: direct-definition . . . 31

Proposition 9 `prop:shiftedONS` coverage: component-only . . . 33

Proposition 10 `prop:shiftedbound` coverage: direct . . . 34

Lemma 11 `lem:facenorm` coverage: component-only . . . 35

Lemma 12 `lem:absorption` coverage: direct . . . 38

Definition 13 `def:balanced` coverage: direct-definition . . . 39

Theorem 14 `thm:general` coverage: direct . . . 40

Proposition 15 `prop:multiwall` coverage: direct . . . 41

Lemma 16 `lem:sgn` coverage: direct . . . 43

Lemma 17 `lem:poisson` coverage: component . . . 46

Lemma 18 `lem:gaussphase` coverage: direct . . . 48

Lemma 19 `lem:weylgauss` coverage: direct . . . 50

Definition 20 `def:balanced-W` coverage: direct-definition . . . 52

Theorem 21 `thm:decomp` coverage: direct . . . 54

Definition 22 `def:piece` coverage: direct-structure . . . 56

Definition 23 `def:transition` coverage: direct-structure . . . 59

Lemma 24 `lem:compose` coverage: decomposition-component . . . 61

Lemma 25 `lem:split` coverage: decomposition-component . . . 66

Lemma 26 `lem:short-coordinate` coverage: direct-strengthened . . . 71

Proposition 27 `prop:pieces` coverage: direct . . . 73

Proposition 28 `prop:fold` coverage: direct . . . 74

**Library glossary (Mathlib and Lean core)** . . . 80

**Alphabetical definition index** . . . 88

# File header

Statements.lean:1

LEAN · FILE HEADER line 1

```
1 import PaperFormalization.Compatibility
```

This file sets each numbered statement of the paper, verbatim, beside its full Lean type, in paper order. Paper definitions and notation precede the statements that use them; unnumbered ones appear in "Paper context" sections with the source excerpt and a note on their use, followed by the Lean bodies. See also the accompanying certificate for Theorem 1.

Every definition that a displayed statement uses, directly or through other definitions, is shown in full: 95, all from `Core.lean`. `Paper.definition3` (the paper's Definition 3, below) and `Paper.definition3Weak` are shown too, though no statement uses either. Nothing else is shown: not the other 77 definitions of `Core.lean` (among them `AllDegreesProof.goodEquiv`, which only an alias uses), nor the 19 paper-numbered aliases of `Compatibility.lean` (one-line `abbrev`s, listed after the index). The paper-numbered statements and both Definition 3 predicates are declared in `Compatibility.lean`, which this file imports; each wrapper below has the same type as the corresponding theorem there and forwards to its proof in `Core.lean`, or, where `Compatibility.lean` carries the proof itself (`Paper.lemma16CotWeightBound`, the four hypothesis bridges of Theorem 5 and Proposition 10, and the restated `theorem5UniformBound` and `proposition10`), to that wrapper.

Definition 3: `Paper.definition3` is now the corrected predicate, with the paper's two disjoint unions stated as a set equality plus pairwise disjointness. The earlier, too weak encoding — whose unique-membership clauses force neither the pieces into P nor their images into Q (`affine` context) — is kept under the name `Paper.definition3Weak`, so that the counterexamples in that context name a real declaration; it is not the paper's definition. Both are declared in `Compatibility.lean`; this file adds no definition of its own. No theorem uses either predicate.

# Coverage legend and proof status

Statements.lean:36

Coverage tags (how endpoints are presented, not an axiom audit):

- `direct`: the main endpoint is shown; not every subsidiary clause need be.
- `direct-strengthened`: the paper's conclusion under weaker hypotheses or with extra output (Lemma 26).
- `all-degrees-conclusion` (Theorem 5): `theorem5` is part 3, the all-degrees basis; parts 1 and 2 are companion endpoints.
- `component`, `decomposition-component`: one named conclusion; companion endpoints are listed with it and may together cover the item.
- `component-only`: a partial presentation, not the full result.
- `direct-definition`, `direct-structure`: the source definition or structure, displayed for comparison, not certified equivalent to the paper.

Gaps: Proposition 9 and Lemma 11 are `component-only`.

The wrapper types are checked when the file is compiled; this file alone is no evidence of proof completeness. For that, see `AXIOMS.txt` in the project root: `#print axioms` on every exported wrapper, which must report only `propext`, `Classical.choice` and `Quot.sound`.

# Reading the encodings

Statements.lean:58

D-blocks are exact source excerpts, giving namespace, file, line and the statements that use them; nothing is shadowed or changed. Conventions G01–G14 (before Theorem 1) apply throughout; other library names are explained, with the Mathlib declarations that fix them, in the Library glossary near the end, and each block lists those it uses. The alphabetical definition index is at the end. Displaying an encoding does not prove it equivalent to the paper.

LEAN · NAMESPACE lines 69–71

```
69 namespace PaperReview
70
71 open scoped Matrix
```

## Lean and Mathlib conventions used throughout

Statements.lean:74

These conventions (G01–G14) apply to all Lean code below and are not listed per block. Text after `by` is a proof and never changes a value; `f (M := e)` supplies the argument named M. Entries cite the declaration that fixes the meaning (Mathlib v4.35.0, Lean v4.35.0, as pinned by `lean-toolchain`; the environment available here for checking the file was Lean 4.34.0-rc2, one patch release behind -- see README.md).

### G01 — `Fin n`

Statements.lean:83

Meaning: `Fin n` = {0, 1, …, n−1}: its elements are pairs ⟨v, h⟩ with v : ℕ and h : v < n (`t.isLt` is that proof); `Fin 0` is empty. `x : Fin k → R` is a k-tuple (x 0, …, x (k−1)) with coordinatewise +, −, scaling and `0`, so `w ≠ 0` means some coordinate is nonzero. Equality, ≤ and < compare values.

Pinned by: structure `Fin` (fields `val : Nat`, `isLt : val < n`); `Fin.eq_of_val_eq`: i.val = j.val → i = j; `Fin.le_def`: a ≤ b ↔ a.1 ≤ b.1; `Finset.card_fin`: #(univ : Finset (Fin n)) = n.

Conventions: 0-based: paper coordinate j (1 ≤ j ≤ k) is Lean coordinate j−1. Arithmetic in `Fin n` itself (+, −, *, numerals) is mod n (`Fin.val_ofNat`: (Fin.ofNat n a).val = a % n); the displayed definitions do index arithmetic on `.val` in ℕ instead (foldCoord, foldMap, foldInv). `Fin 0 → R` has one element (the empty tuple); sums over `Fin 0` are 0, products 1.

### G02 — Pairs: `α × β`, `(a, b)`, `p.1`, `p.2`

Statements.lean:100

Meaning: (a, b) : α × β is an ordered pair, `p.1 = p.fst`, `p.2 = p.snd`; (a, b, c) = (a, (b, c)). The notation `.1`, `.2` is generic: on a subtype ↥s it means n.val and n.property; on a Sigma type (m : Fin M) × Fin (N m), the index m and the element of Fin (N m).

Pinned by: structure `Prod` (fields `fst`, `snd`); `Prod.le_def`: x ≤ y ↔ x.1 ≤ y.1 ∧ x.2 ≤ y.2 (componentwise order; ×ₗ is the lexicographic one).

Conventions: For n in a subtype ↥s, `n.1` is the element and `n.2` a membership proof, not a second coordinate; in Lemma 24, `.1`, `.2` of `(sigmaPieceIndexEquiv N).symm t` are Sigma projections.

### G03 — `Finset α` and `Finset.univ`

Statements.lean:114

Meaning: A `Finset α` is a finite subset of α (a duplicate-free multiset, determined by its members). `Finset.univ` is the finset of all elements of a finite type (for `Fin n`: {0, …, n−1}). A finset coerces to a `Set α` with the same members.

Pinned by: `Finset.ext`: (∀ a, a ∈ s₁ ↔ a ∈ s₂) → s₁ = s₂; `Finset.mem_univ`: x ∈ (univ : Finset α); `Finset.mem_coe`: a ∈ (s : Set α) ↔ a ∈ s.

Conventions: No order and no multiplicity; #s counts distinct elements. `classical` in definition bodies supplies the decidability that finset operations need and does not change values.

### G04 — Big operators `∑ x ∈ s, f x`, `∑ i, f i`, `∏ x ∈ s, f x`, `∏ i, f i`

Statements.lean:128

Meaning: `∑ x ∈ s, f x` is `Finset.sum s f`, the sum of f over the finset s; `∑ i, f i` (or `∑ i : ι, f i`) is the sum over `Finset.univ` of the finite type ι (for `Fin k`: f 0 + ⋯ + f (k−1)); `∑ n : ↥s, f n` equals ∑ x ∈ s, f x. ∏ is the same with products.

Pinned by: notation docstring in Mathlib/Algebra/BigOperators/Group/Finset/Defs.lean; `Finset.sum_empty`: ∑ x ∈ ∅, f x = 0; `Finset.prod_empty`: ∏ x ∈ ∅, f x = 1; `Finset.sum_coe_sort`: ∑ i : s, f i = ∑ i ∈ s, f i; library note «operator precedence of big operators»: the body is parsed at precedence 67.

Conventions: Empty sum 0, empty product 1 (e.g. over Fin 0). Precedence 67 lies between + (65) and * (70): the body extends over *, / and ^ but stops at +, − and comparisons, so `∑ s, a s * x s + b` is (∑ s, a s * x s) + b and `∑ s, ‖z s‖ ^ 2 = 1` is (∑ s, ‖z s‖ ^ 2) = 1. The sum is computed in the codomain of f (a sum of ℕ-values is in ℕ).

### G05 — Casts `(n : ℝ)`, `((q : ℕ) : ℝ)`, `↑x` Statements.lean:147

Meaning: `(e : T)` elaborates e at type T, inserting the canonical coercion `↑e` if needed: `Nat.cast` from ℕ, `Int.cast` from ℤ, `Complex.ofReal` from ℝ to ℂ. Into ℤ, ℚ, ℝ, ℂ these are injective (semi)ring homomorphisms, order-preserving into ℤ, ℚ, ℝ, and they compose consistently (ℕ → ℤ → ℝ is ℕ → ℝ), so equalities and inequalities transfer exactly.

Pinned by: `Nat.cast_le`: (m : α) ≤ n ↔ m ≤ n; `Nat.cast_inj`: (m : R) = n ↔ m = n; `Int.cast_le`, `Int.cast_inj` (same for ℤ); `Complex.ofReal_inj`: (z : ℂ) = w ↔ z = w; `Int.cast_natCast`: ((n : ℤ) : R) = n; `Nat.cast_sub` (m ≤ n): ((n - m : ℕ) : R) = n - m; `Nat.cast_div` (n | m, (n : K) ≠ 0): (↑(m / n) : K) = m / n.

Conventions: Casts commute with +, *, ^, max, |·|, but not with ℕ subtraction or ℕ/ℤ division, so placement matters: `((N / k : ℕ) : ℤ)` floor-divides first, `((k - s.val : ℕ) : ℝ)` subtracts in ℕ first, `((m : ℤ) - 1)` casts first. An ascription of an arithmetic expression casts the leaves: `(a - b : ℝ)` with a b : ℕ is (a : ℝ) – (b : ℝ) (no truncation), while a parenthesised `(N / (k - s.val) : ℕ)` is one leaf computed in ℕ (src/Init/Notation.lean). A function expecting ℝ inserts the cast silently, e.g. `Real.sqrt (Nat.multinomial …)`.

### G06 — `|x|` (abs) and `‖z‖` (norm) Statements.lean:168

Meaning: |a| = max a (–a) on ℤ and ℝ. ‖·‖ is the norm of the argument's type: |x| on ℝ; the modulus $\sqrt{re^2 + im^2}$ on ℂ; on a finite product such as Fin K → ℂ the sup norm max_s ‖z s‖, not the Euclidean norm.

Pinned by: `abs_eq_max_neg`: |a| = max a (-a); `Real.norm_eq_abs`: ‖r‖ = |r|; `Complex.norm_eq_sqrt_sq_add_sq`: ‖z‖ = √(z.re ^ 2 + z.im ^ 2); `Pi.norm_def`: ‖f‖ = ↑(Finset.univ.sup fun b => ‖f b‖₊).

Conventions: For z : Fin K → ℂ, `‖z‖ < 1` would describe the polydisc, not the ball; the Euclidean norm has to be written out as √(∑ s, ‖z s‖ ^ 2).

### G07 — `a - b` on ℕ (truncated subtraction) Statements.lean:181

Meaning: For a b : ℕ, `a - b` is max(a – b, 0); on ℤ, ℚ, ℝ, ℂ it is ordinary subtraction. The type at which `-` is elaborated decides, not the values: `(N - b + k).choose k` and `(k - s.val : ℕ)` truncate, `((m : ℤ) - 1)` does not.

Pinned by: `Nat.sub_eq_zero_iff_le`: n - m = 0 ↔ n ≤ m; `Nat.sub_add_cancel`: m ≤ n → n - m + m = n; `Nat.cast_sub`: m ≤ n → ((n - m : ℕ) : R) = n - m.

Conventions: Left-associative: `N-i+k-1` is ((N – i) + k) – 1, so truncation can happen at an intermediate step (N – i = 0 if i > N). Cast placement: G05 (`((a - b : ℝ)` with a b : ℕ does not truncate; `((a - b : ℕ) : ℝ)` does).

### G08 — `/`, `%`, `⁻¹`, `∣` (division, remainder, inverse, divides) Statements.lean:194

Meaning: On ℕ, a / b = ⌊a/b⌋ and a % b is the remainder. On ℤ, `/` and `%` are Euclidean (`Int.ediv`, `Int.emod`): for b ≠ 0, a = b·(a / b) + a % b with 0 ≤ a % b < |b|, so a / b = ⌊a/b⌋ for b > 0 and ⌈a/b⌉ for b < 0. On ℚ, ℝ, ℂ, x / y = x * $y^{-1}$. `a ∣ b` means ∃ c, b = a * c.

Pinned by: `Int.ediv_emod_unique` (0 < b): a / b = q ∧ a % b = r ↔ r + b * q = a ∧ 0 ≤ r ∧ r < b; `Nat.div_add_mod`: n * (m / n) + m % n = m; `Nat.mod_lt`: 0 < y → x % y < y; `div_eq_mul_inv`: a / b = a * $b^{-1}$; `Int.dvd_def`: (a ∣ b) = ∃ c, b = a * c.

Conventions: Junk values: n / 0 = 0 and n % 0 = n on ℕ, ℤ (`Nat.div_zero`, `Nat.mod_zero`, `Int.ediv_zero`, `Int.emod_zero`); x / 0 = 0 and $0^{-1}$ = 0 in ℚ, ℝ, ℂ (`div_zero`, `inv_zero`). On ℤ, (–7) / 2 = –4 and (–7) % 2 = 1. `((N / k : ℕ) : ℤ)` and `(N / (k - s.val) : ℕ)` divide in ℕ (`Int.natCast_ediv`: ((m / n : ℕ) : ℤ) = (m : ℤ) / (n : ℤ)); `(a : ℝ) / (b : ℝ)` is real division. `0 ∣ n` only for n = 0.

### G09 — `x ^ n` with `n : ℕ` Statements.lean:213

Meaning: For n : ℕ and x in a monoid (ℕ, ℤ, ℝ, ℂ, square matrices, …), x ^ 0 = 1 and x ^ (n+1) = x ^ n * x. A numeral exponent such as 2 is a natural number by default. A real exponent (`lam ^ ((n : ℝ) / 2)`) is the different operation `Real.rpow` (G56).

Pinned by: `pow_zero`: a ^ 0 = 1; `pow_succ`: a ^ (n + 1) = a ^ n * a; default exponent type ℕ from `NPow.toPow` (`default_instance high`).

Conventions: 0 ^ 0 = 1, so a monomial factor z_s^0 is 1 even at z_s = 0. Application binds tighter than ^, and ^ tighter than * and /: `Real.sqrt (q : ℝ) ^ k` is (√q)^k.

### G10 — `Matrix m n R`, `Matrix.of`, and matrix arithmetic

Statements.lean:227

Meaning: `Matrix m n α` is `m → n → α`; `A i j` is the (i, j) entry, so `Matrix (Fin k) (Fin k) ℤ` is a k×k integer matrix with 0-based indices. `Matrix.of f` has entries f i j. `+`, `-`, `0`, `•` are entrywise, `A * B` is the product, `1` the identity (`-1` its negative), `A⁻¹` the inverse.

Pinned by: definition `Matrix m n α := m → n → α`; `Matrix.of_apply`: of f i j = f i j; `Matrix.mul_apply`: (M * N) i k = ∑ j, M i j * N j k; `Matrix.one_apply`: (1 : Matrix n n α) i j = if i = j then 1 else 0.

Conventions: The inverse is total: `Matrix.inv_def`: $A^{-1} = A.\det^{-1_r} \bullet A.\text{adjugate}$ ($^{-1_r}$ is `Ring.inverse`), so $A^{-1} = 0$ if A.det is not a unit (`Matrix.nonsing_inv_apply_not_isUnit`); over ℚ, a unit means ≠ 0. $*_v$ and $^\intercal$ are scoped notations enabled by `open scoped Matrix`.

### G14 — `max` and `min`

Statements.lean:243

Meaning: In a linear order (ℕ, ℤ, ℝ), max a b = if a ≤ b then b else a and min a b = if a ≤ b then a else b.

Pinned by: `max_def`, `min_def` (these equations); `Nat.cast_max`: (↑(max m n : ℕ) : α) = max (m : α) n; `Int.cast_max`: (↑(max a b) : R) = max (a : R) (b : R).

Terms used only inside definition bodies, one line each:

- G11 — `classical`: decidability instances by choice; values are unchanged.
- G12 — `abbrev`: a reducible definition; type classes see through the name.
- G13 — `noncomputable`: no executable code is generated; the meaning is unaffected (`Classical.choice`).

# Theorem 1 thm:onb

Statements.lean:260

coverage: direct

PAPER CONTEXT conventions

Statements.lean:264

## Notation and Lean reading conventions

TeX macros in the excerpts: \cp = $\mathbb{C}$, \RR = $\mathbb{R}$, \q/\Q = $\mathbb{Q}$, \z/\Z = $\mathbb{Z}$, \zp = nonnegative integers, \rp = nonnegative reals, \PolySpace and \H = H, \Sphere = S, \p = $\varphi$; \cP, \cS, \B, \C, \D are the P, S, B, C, D defined below; \ov = complex conjugation, \ft = Fourier transform, \dd = differential, \bo = boldface, \mymod{r} = modulo r; \supp, \vol, \len, \sect, \lcmk are defined below; shortened Greek commands (al, la, ga, …) are $\alpha$, $\lambda$, $\gamma$, ….

Notation: e(t) = exp(2πit), v·x = $\Sigma_j v_j x_j$, $|v|_1 = \Sigma_j |v_j|$, and $|A|_\infty = \max_{ij} |A_{ij}|$ for a matrix (not the maximum absolute row sum). For nonnegative X, Y, $X \ll_p Y$ means $X \le C(p)Y$ with C depending only on the subscripted parameters, and $X \asymp_p Y$ means both inequalities; so the position of `∃ C` relative to the other quantifiers matters. A disjoint union asserts both exhaustion and pairwise disjointness.

In Lean, `Fin k` indexes coordinates from 0: paper coordinate j is Lean coordinate j−1.

Lean definitions for this comparison: `expTwoPiI`. Statements depending on these definitions: Theorem 5, Lemma 7, Proposition 9, Proposition 10, Theorem 14, Proposition 15, Lemma 16, Lemma 17, Lemma 18, Lemma 19.

D001 **expTwoPiI**

Statements.lean:289

`Core.lean:617` · Used by: Lemma 16, Lemma 17, Lemma 18; Theorem 5, Lemma 7, Proposition 9, Proposition 10, Theorem 14, Proposition 15, Lemma 19 (indirectly).

```
296 noncomputable def expTwoPiI (x : ℝ) : ℂ := Complex.exp (2 * Real.pi * Complex.I * x)
```

Library terms: G58 `Real.exp`.

PAPER CONTEXT polynomials

Statements.lean:303

## The ball, sphere, Hardy space, and homogeneous polynomials

Lean represents H_N (also written H_{K,N}) by monomial coefficients: `PolySpaceH K N` holds coefficient vectors indexed by the multi-indices of total degree N (`degIdx`), with pointwise operations, and `polyEval` maps c to $z \mapsto \Sigma_j c_j z^j$. Neither the Hardy space nor a subspace of $L^2(S^{2K-1})$ is constructed; `polyInner` (`pairing` context) is a coefficient formula.

> ORIGINAL PAPER DEFINITION/CONTEXT article.txt:204-214 · typeset from the verbatim TeX at lines 313–323
>
> Let $\mathbb{B}_K := \{z \in \mathbb{C}^K : |z_1|^2 + \cdots + |z_K|^2 < 1\}$ be the complex unit ball and $\mathbb{S}^{2K-1} = \partial\mathbb{B}_K$ be the complex unit sphere. Let $H^2(\mathbb{B}_K)$ denote the Hardy space of holomorphic functions $f : \mathbb{B}_K \to \mathbb{C}$ that have radial boundary values in $L^2$, that is, $f(r\cdot)$ converges in $L^2(\mathbb{S}^{2K-1})$ as $r \to 1^-$. In this way, the Hardy space $H^2(\mathbb{B}_K)$ inherits the Hilbert space structure of $L^2(\mathbb{S}^{2K-1})$ (see [R]). Write $z = (z_1, \ldots, z_K)$ and let
>
> $$\mathrm{H}_N := \mathrm{span}_{\mathbb{C}} \left\{ z^j : j \in \mathbb{Z}^K_{\ge 0},\ j_1 + \cdots + j_K = N \right\} \quad (z^j = z_1^{j_1} \cdots z_K^{j_K})$$
>
> denote the space of homogeneous polynomials of degree $N$ (note that the dimension $K$ is implied from the context). A routine calculation shows that $z^j$ and $z^l$ with $j \ne l$ are orthogonal in $H^2(\mathbb{B}_K)$. In particular, we have the following orthogonal decomposition
>
> $$H^2(\mathbb{B}_K) = \overline{\bigoplus_{N \ge 0} \mathrm{H}_N}.$$

Lean definitions for this comparison: `degIdx`, `PolySpaceH`, `polyEval`, `UniformBoundUnit.cSphere`. Statements depending on these definitions: Theorem 1, Theorem 5, Proposition 9, Proposition 10, Lemma 11.

### D002 degIdx

Statements.lean:331

Core.lean:621 · Used by: Theorem 1, Theorem 5, Proposition 9, Proposition 10, Lemma 11 (indirectly).

```
337 def degIdx (K N : ℕ) : Finset (Fin K → ℕ) := Finset.piAntidiag Finset.univ N
```

Library terms: G34 `Finset.piAntidiag`.

### D003 PolySpaceH

Statements.lean:344

Core.lean:623 · Used by: Theorem 1, Theorem 5, Lemma 11; Proposition 9, Proposition 10 (indirectly).

```
350 abbrev PolySpaceH (K N : ℕ) : Type := ↥(degIdx K N) → ℂ
```

Library terms: G35 `↥s`.

### D004 polyEval

Statements.lean:357

Core.lean:625 · Used by: Theorem 1, Theorem 5, Proposition 10, Lemma 11.

```
362 noncomputable def polyEval (K N : ℕ) (c : PolySpaceH K N) (z : Fin K → ℂ) : ℂ :=
363   ∑ n : ↥(degIdx K N), c n * ∏ s, z s ^ ((n : Fin K → ℕ) s)
```

Library terms: G15 `x.val`; G35 `↥s`.

### D005 UniformBoundUnit.cSphere

Statements.lean:370

Core.lean:3185 · Used by: Theorem 5, Lemma 11.

```
375 def cSphere (K : ℕ) : Set (Fin K → ℂ) := {z | ∑ s, ‖z s‖ ^ 2 = 1}
```

PAPER CONTEXT `pairing`

Statements.lean:380

## Normalized surface measure and the polynomial inner product

With σ normalized so that σ(S^(2K−1)) = 1, the paper's monomial norms (excerpt below) give the pairing in coefficients,

$$\langle c,d\rangle = \text{choose}(N+K-1,K-1)^{-1}\ \Sigma_j\ \text{multinomial}(N;j)^{-1}\ c_j\ \overline{d_j},$$

which is the formula of `polyInner` (linear in c, conjugate-linear in d). Its weights are the classical integrals: for K ≥ 1 and |j| = N,

$$\begin{aligned}&\text{choose}(N+K-1,K-1)^{-1}\cdot\text{multinomial}(N;j)^{-1}\\&\quad=[(K-1)!\ N!/(N+K-1)!]\cdot[j_1!\cdots j_K!/N!]\\&\quad=(K-1)!\ j_1!\cdots j_K!/(N+K-1)!=\int|z^j|^2\,d\sigma\end{aligned}$$

(Rudin, Function Theory in the Unit Ball of $\mathbb{C}^n$, Prop. 1.4.9; this is the paper's reference [R]). With the orthogonality of distinct monomials, this shows by hand that `polyInner K N c d` is the normalized $L^2$(S^(2K−1)) pairing of `polyEval K N c` and `polyEval K N d`. This is a by-hand identification, not a Lean theorem: Lean defines the pairing by the coefficient formula, not by an integral, and the coefficient model is not a checked identification with the Hardy space.

`Module.Basis` plus `polyInner ... = if i = j then 1 else 0` encode the orthonormal basis of Theorem 1; its L∞ bound is on values on the open ball, not on coefficients (sphere endpoints use `cSphere` or its equation).

ORIGINAL PAPER DEFINITION/CONTEXT article.txt:468-482 · typeset from the verbatim TeX at lines 408–422

Recall that the $H^2(\mathbb{B}_K)$ space is equipped with the following inner product

$$\langle f, g\rangle := \int_{\mathbb{S}^{2K-1}} f(z)\overline{g(z)}\,\mathrm{d}\sigma(z),$$

where $\sigma$ is the induced Lebesgue surface measure on $\mathbb{S}^{2K-1}$ normalized so that $\sigma(\mathbb{S}^{2K-1}) = 1$. A routine computation (see [R]) shows that for any $j, l \in \mathbb{Z}^K_{\geq 0}$ with $N = j_1 + \cdots + j_K = l_1 + \cdots + l_K$ we have

$$\langle z^j, z^l\rangle = \binom{N+K-1}{K-1}^{-1}\binom{N}{j}^{-1}\boldsymbol{\delta}_{j,l}, \quad \text{where} \quad \binom{N}{j} := \frac{N!}{j_1!\cdots j_K!}. \tag{def:innerprod}$$

ORIGINAL PAPER DEFINITION/CONTEXT article.txt:526-529 · typeset from the verbatim TeX at lines 427–430

For $z \in \mathbb{S}^{2K-1}$ we put $z_s = \sqrt{r_s}e(\theta_s)$, for $s = 1, \ldots, K$, where $e(z) = e^{2\pi i z}$, $\theta_s \in \mathbb{R}$ and $r_s \geq 0$ so that $\sum_{s=1}^K r_s = 1$. In order to estimate uniformly

$$\|\varphi\|_{L^\infty(\mathbb{B}_K)} = \|\varphi\|_{L^\infty(\mathbb{S}^{2K-1})} = \sup_{\substack{r_1+\cdots+r_K=1\\ r_s\geq 0,\ \theta_s\in\mathbb{R}}} |\varphi(\sqrt{r_1}e(\theta_1), \ldots, \sqrt{r_K}e(\theta_K))|$$

Lean definitions for this comparison: `polyInner`. Statements depending on these definitions: Theorem 1, Theorem 5, Proposition 9, Lemma 11.

### D006 `polyInner`

Statements.lean:438

`Core.lean:628` · Used by: Theorem 1, Theorem 5, Proposition 9, Lemma 11.

```
443 noncomputable def polyInner (K N : ℕ) (c d : PolySpaceH K N) : ℂ :=
444   ((Nat.choose (N + K - 1) (K - 1) : ℂ))⁻¹ *
445     ∑ n : ↥(degIdx K N),
446       ((Nat.multinomial Finset.univ (n : Fin K → ℕ) : ℂ))⁻¹ * c n * (starRingEnd ℂ) (d n)
```

Library terms: G15 `x.val`; G35 `↥s`; G52 `Nat.choose`; G53 `Nat.multinomial`; G61 `starRingEnd ℂ`.

PAPER STATEMENT typeset from the verbatim TeX at lines 456–463

**Theorem 1.** *Let $K \geq 2$. Then for every $N \geq 0$ the space $\mathrm{H}_N$ admits an orthonormal basis $\{\varphi_{1,N}, \varphi_{2,N}, \ldots, \varphi_{\dim \mathrm{H}_N, N}\}$ such that*

$$\sup_{\substack{N\geq 0\\ 1\leq i\leq \dim \mathrm{H}_N}} \|\varphi_{i,N}\|_{L^\infty(\mathbb{B}_K)} < \infty.$$

Coverage: `direct`.

Definitions used: `degIdx` (D002), `PolySpaceH` (D003), `polyEval` (D004), `polyInner` (D006). Library terms: G67 `Module.Basis ι R M`; G68 `Module.finrank`.

LEAN STATEMENT `theorem1` lines 474–482

```
474 theorem theorem1 :
475   ∀ (K : ℕ) (hK : 2 ≤ K),
476 ∃ C : ℝ, 0 < C ∧
477       ∀ N : ℕ,
478         ∃ φ : Module.Basis (Fin (Module.finrank ℂ (PolySpaceH K N))) ℂ (PolySpaceH K N),
479           (∀ i j, polyInner K N (φ i) (φ j) = if i = j then 1 else 0) ∧
480           ∀ i, ∀ z : Fin K → ℂ, ∑ s, ‖z s‖ ^ 2 < 1 →
481             ‖polyEval K N (φ i) z‖ ≤ C :=
482   @_root_.thm_onb
```

# Theorem 2 `thm:box`

Statements.lean:485

coverage: direct

PAPER CONTEXT `geometry` Statements.lean:489

## Integer polytopes, boxes, volume, and sections

`polyPZ` is the paper's P(L) ⊆ ℤ^k (a ℤ-polytope need not be bounded); `polyP` tests the same inequalities on ℕ-vectors. Further names: J ↦ `boxZ`, #$J_j$ ↦ `boxSide` (= max($hi_j$–$lo_j$+1,0)), vol ↦ `boxVol`, supp ↦ `vecSupport`, len ↦ `boxLen`, sect ↦ `boxSect`, P(L)∩J ↦ `polyBoxZ`; the `UniformBoundUnit.*Z` variants are separate source definitions. `quadExpSum k α c θ S` sums over an arbitrary finite S ⊆ ℤ^k; with S = `polyBoxZ k L lo hi` it is the paper's sum over P(L)∩J.

Theorem 2's Lean type writes the box, constraint filter, exponential sum, vol(I) and sect($w_0$;I) inline; unfolded, these agree with `boxZ`, `polyBoxZ`, `quadExpSum`, `boxVol` and `boxSect`, so root `ThmBox` unfolds to Theorem 2's type (`basis` context). `UniformBoundUnit.ThmBox` uses the `*Z` variants.

ORIGINAL PAPER DEFINITION/CONTEXT article.txt:358-375 · typeset from the verbatim TeX at lines 507–524

Let $k \geq 1$. For a finite set $L \subset \mathbb{Z}^k \times \mathbb{Z}$, whose elements we call *constraints*, put

$$\mathcal{P}(L) := \{n \in \mathbb{Z}^k : v \cdot n + b \geq 0 \text{ for every } (v, b) \in L\}.$$

A $\mathbb{Z}$-*polytope* is a set of the form $\mathcal{P}(L)$.

A *box* $J \subset \mathbb{Z}^k$ is a product of finite intervals, i.e., $J = J_1 \times \cdots \times J_k$ with $J_i = [a_i, b_i] \cap \mathbb{Z}$. For a box $J$ and $w = (w_1, \ldots, w_k) \in \mathbb{Z}^k$, put

$$\operatorname{len}(w; J) := \max\Big(1, \max_{j : w_j \neq 0} \#J_j\Big).$$

Thus $\operatorname{len}(w; J)$ is the longest side seen by $w$. Define

$$\operatorname{vol}(J) := \prod_{i=1}^{k} \#J_i \quad \text{and} \quad \operatorname{sect}(w; J) := \frac{\operatorname{vol}(J)}{\operatorname{len}(w; J)}.$$

ORIGINAL PAPER DEFINITION/CONTEXT article.txt:381-385 · typeset from the verbatim TeX at lines 529–535

Finally, for any symbol $\square$ whatsoever, whenever we write $\square \in F^k$ with $F$ a set we will denote by $\square_j$ the $j$th component of $\square$ so that $\square = (\square_1, \ldots, \square_k)$. Thus if $s \in \mathbb{Q}^k$ then we automatically infer that $s_2$ refers to the second component of $s$, i.e., $s = (s_1, s_2, \ldots, s_k)$. For any vector $v \in F^k$ we will also denote by $\operatorname{supp} v$ the set of indices $j$ for which $v_j \neq 0$. For a real vector $v$ we write $|v|_1 = \sum_j |v_j|$ and $|v|_\infty = \max_j |v_j|$, and for a real matrix $A$ we write $|A|_\infty$ for the largest absolute value of an entry of $A$.

The three norms of the last sentence are `∑ j, |v j|`, `Finset.univ.sup fun j => (v j).natAbs` and `matInftyNorm` (D023) in the encodings below; `|v|_1 ≤ R` and `|A|_∞ ≤ R` are the complexity bounds of Definition 3, Definition 20 and Lemma 26, and `|v|_1 ≤ V` is the constraint bound of Theorem 2.

Lean definitions for this comparison: `polyPZ`, `polyP`, `vecSupport`, `boxZ`, `boxSide`, `boxVol`, `boxLen`, `boxSect`, `polyBoxZ`, `quadExpSum`, `UniformBoundUnit.latticePtsZ`, `UniformBoundUnit.boxVolZ`, `UniformBoundUnit.boxLenZ`, `UniformBoundUnit.boxSectZ`. Statements depending on these definitions: Theorem 5, Proposition 10, Theorem 14, Theorem 21, Lemma 24, Lemma 25, Lemma 26, Proposition 27, Proposition 28.

### D007 polyPZ

Statements.lean:548

Core.lean:9569 · Used by: Theorem 21, Lemma 25, Lemma 26, Proposition 28; Lemma 24, Proposition 27 (indirectly).

```
554 def polyPZ (d : ℕ) (W : Finset ((Fin d → ℤ) × ℤ)) : Set (Fin d → ℤ) :=
555   {x | ∀ p ∈ W, 0 ≤ (∑ s, p.1 s * x s) + p.2}
```

### D008 polyP

Statements.lean:560

Core.lean:639 · Used by: Theorem 5, Proposition 10 (indirectly).

```
565 def polyP (k : ℕ) (L : Finset ((Fin k → ℤ) × ℤ)) : Set (Fin k → ℕ) :=
566   {x | ∀ p ∈ L, 0 ≤ (∑ s, p.1 s * (x s : ℤ)) + p.2}
```

### D009 vecSupport

Statements.lean:571

Core.lean:9566 · Used by: Lemma 26; Lemma 25, Proposition 27 (indirectly).

```
576 def vecSupport {d : ℕ} (w : Fin d → ℤ) : Finset (Fin d) :=
577   Finset.univ.filter fun j => w j ≠ 0
```

Library terms: G26 Finset.filter.

### D010 boxZ

Statements.lean:584

Core.lean:1976 · Used by: Theorem 21; Proposition 10, Theorem 14, Lemma 24, Lemma 25, Lemma 26, Proposition 27 (indirectly).

```
590 def boxZ (k : ℕ) (lo hi : Fin k → ℤ) : Finset (Fin k → ℤ) :=
591   Fintype.piFinset fun s => Finset.Icc (lo s) (hi s)
```

Library terms: G25 Finset.Icc; G33 Fintype.piFinset.

### D011 boxSide

Statements.lean:598

Core.lean:1979 · Used by: Proposition 10, Theorem 14, Theorem 21, Lemma 25, Lemma 26, Proposition 27 (indirectly).

```
604 def boxSide (k : ℕ) (lo hi : Fin k → ℤ) (s : Fin k) : ℕ :=
605   (Finset.Icc (lo s) (hi s)).card
```

Library terms: G25 Finset.Icc; G28 Finset.card.

### D012 boxVol

Statements.lean:612

Core.lean:1982 · Used by: Theorem 14, Theorem 21, Proposition 27; Proposition 10 (indirectly).

```
618 def boxVol (k : ℕ) (lo hi : Fin k → ℤ) : ℕ :=
619   ∏ s, boxSide k lo hi s
```

### D013 boxLen

Statements.lean:624

Core.lean:1985 · Used by: Theorem 21; Proposition 10 (indirectly).

```
629 def boxLen (k : ℕ) (w : Fin k → ℤ) (lo hi : Fin k → ℤ) : ℕ :=
630   max 1 ((Finset.univ.filter fun s : Fin k => w s ≠ 0).sup (boxSide k lo hi))
```

Library terms: G26 `Finset.filter`; G29 `Finset.sup`.

### D014 boxSect

Statements.lean:637

Core.lean:1988 · Used by: Proposition 10 (indirectly).

```
642 noncomputable def boxSect (k : ℕ) (w : Fin k → ℤ) (lo hi : Fin k → ℤ) : ℝ :=
643   (boxVol k lo hi : ℝ) / (boxLen k w lo hi : ℝ)
```

### D015 polyBoxZ

Statements.lean:648

Core.lean:1991 · Used by: Theorem 14; Proposition 10 (indirectly).

```
653 def polyBoxZ (k : ℕ) (L : Finset ((Fin k → ℤ) × ℤ)) (lo hi : Fin k → ℤ) :
654     Finset (Fin k → ℤ) :=
655   (boxZ k lo hi).filter fun x => ∀ p ∈ L, 0 ≤ (∑ s, p.1 s * x s) + p.2
```

Library terms: G26 `Finset.filter`.

### D016 quadExpSum

Statements.lean:662

Core.lean:1995 · Used by: Theorem 14; Proposition 10 (indirectly).

```
667 noncomputable def quadExpSum (k : ℕ) (α : ℝ) (c : Fin k → ℕ) (θ : Fin k → ℝ)
668     (S : Finset (Fin k → ℤ)) : ℂ :=
669   ∑ n ∈ S, expTwoPiI (α * ∑ j, (c j : ℝ) * (n j : ℝ) ^ 2 + ∑ j, θ j * (n j : ℝ))
```

### D017 UniformBoundUnit.latticePtsZ

Statements.lean:674

Core.lean:3191 · Used by: Theorem 5 (indirectly).

```
679 def latticePtsZ (k : ℕ) (L : Finset ((Fin k → ℤ) × ℤ)) (lo hi : Fin k → ℤ) :
680     Finset (Fin k → ℤ) :=
681   (Fintype.piFinset fun j => Finset.Icc (lo j) (hi j)).filter
682     fun n => ∀ p ∈ L, 0 ≤ (∑ s, p.1 s * n s) + p.2
```

Library terms: G25 `Finset.Icc`; G26 `Finset.filter`; G33 `Fintype.piFinset`.

### D018 `UniformBoundUnit.boxVolZ`

Statements.lean:689

`Core.lean:3196` · Used by: Theorem 5 (indirectly).

```
694 def boxVolZ (k : ℕ) (lo hi : Fin k → ℤ) : ℕ :=
695   ∏ j, (Finset.Icc (lo j) (hi j)).card
```

Library terms: G25 `Finset.Icc`; G28 `Finset.card`.

### D019 `UniformBoundUnit.boxLenZ`

Statements.lean:702

`Core.lean:3199` · Used by: Theorem 5 (indirectly).

```
707 def boxLenZ (k : ℕ) (w : Fin k → ℤ) (lo hi : Fin k → ℤ) : ℕ :=
708   max 1 ((Finset.univ.filter fun j => w j ≠ 0).sup fun j =>
709     (Finset.Icc (lo j) (hi j)).card)
```

Library terms: G25 `Finset.Icc`; G26 `Finset.filter`; G28 `Finset.card`; G29 `Finset.sup`.

### D020 `UniformBoundUnit.boxSectZ`

Statements.lean:717

`Core.lean:3203` · Used by: Theorem 5 (indirectly).

```
722 noncomputable def boxSectZ (k : ℕ) (w : Fin k → ℤ) (lo hi : Fin k → ℤ) : ℝ :=
723   (boxVolZ k lo hi : ℝ) / (boxLenZ k w lo hi : ℝ)
```

PAPER CONTEXT `fractions`

Statements.lean:728

## Continued fractions and bounded partial quotients

The numbered statements bound only $a_i$ for $i \geq 1$, not $a_0$; the general introductory wording is quoted unchanged below. Lean bounds every entry of `(GenContFract.of α).partDens`, which omits $a_0$ (G76); convergents use `.nums` and `.dens`. `UniformBoundUnit.BddPartQuot` packages irrationality and this bound; other theorem types spell both conditions out.

> ORIGINAL PAPER DEFINITION/CONTEXT article.txt:377-379 · typeset from the verbatim TeX at lines 738–740
>
> Given an irrational number $\alpha$ write $\alpha := [a_0; a_1, a_2, \ldots]$ for the continued fraction expansion of $\alpha$. We say that $\alpha$ has bounded partial quotients when there exists a $C > 0$ such that $a_i \leq C$ for all $a_i$.

Lean definitions for this comparison: `UniformBoundUnit.BddPartQuot`. Statements depending on these definitions: Theorem 5.

### D021 `UniformBoundUnit.BddPartQuot`

Statements.lean:748

`Core.lean:3187` · Used by: Theorem 5.

```
753 def BddPartQuot (a α : ℝ) : Prop :=
754   Irrational α ∧
755     ∀ (i : ℕ) (x : ℝ), (GenContFract.of α).partDens.get? i = some x → x ≤ a
```

Library terms: G55 `Irrational`; G76 `GenContFract.of`.

**PAPER STATEMENT** typeset from the verbatim TeX at lines 764–784

**Theorem 2.** *Let $k \geq 1$, $r \geq 0$ and $V, h, a \geq 1$. Let $\alpha = [a_0; a_1, a_2, \ldots]$ be an irrational number with $a_i \leq a$ for all $i \geq 1$. Let $c_1, \ldots, c_k$ be positive integers with $c_j \leq h$. Let $I = I_1 \times \cdots \times I_k$ be a box of nonempty intervals. Let $L$ be a set of at most $r$ pairs $(v, b) \in \mathbb{Z}^k \times \mathbb{Z}$ with $|v|_1 \leq V$. Then*

$$\sup_{\theta \in \mathbb{R}^k} \Big| \sum_{n \in \mathcal{P}(L) \cap I} e\Big(\alpha \sum_{j=1}^{k} c_j n_j^2 + \theta \cdot n\Big)\Big| \ll_{k,r,V,h,a} \operatorname{vol}(I)^{1/2}.$$

*If both $(w_0, b_0)$ and $(-w_0, -b_0)$ belong to $L$, with $w_0 \neq 0$, then also*

$$\sup_{\theta \in \mathbb{R}^k} \Big| \sum_{n \in \mathcal{P}(L) \cap I} e\Big(\alpha \sum_{j=1}^{k} c_j n_j^2 + \theta \cdot n\Big)\Big| \ll_{k,r,V,h,a} \operatorname{sect}(w_0; I)^{1/2}.$$

Coverage: `direct`.

Library terms: G25 `Finset.Icc`; G26 `Finset.filter`; G28 `Finset.card`; G29 `Finset.sup`; G33 `Fintype.piFinset`; G55 `Irrational`; G57 `Real.sqrt`; G58 `Real.exp`; G76 `GenContFract.of`.

**LEAN STATEMENT** `theorem2` lines 795–818

```
795 theorem theorem2 :
796   ∀ (k r V h : ℕ) (a : ℝ), 1 ≤ k → 1 ≤ V → 1 ≤ h → 1 ≤ a →
797       ∃ C : ℝ, 0 < C ∧ ∀ α : ℝ, Irrational α →
798         (∀ (i : ℕ) (x : ℝ), (GenContFract.of α).partDens.get? i = some x → x ≤ a) →
799         ∀ c : Fin k → ℕ, (∀ j, 1 ≤ c j) → (∀ j, c j ≤ h) →
800         ∀ lo hi : Fin k → ℤ, (∀ s, lo s ≤ hi s) →
801         ∀ L : Finset ((Fin k → ℤ) × ℤ), L.card ≤ r →
802           (∀ p ∈ L, ∑ s, |p.1 s| ≤ (V : ℤ)) →
803         ∀ θ : Fin k → ℝ,
804           ‖∑ n ∈ (Fintype.piFinset fun s => Finset.Icc (lo s) (hi s)).filter
805                 (fun x => ∀ p ∈ L, 0 ≤ (∑ s, p.1 s * x s) + p.2),
806               Complex.exp (2 * (Real.pi : ℂ) * Complex.I *
807                 ((α * ∑ j, (c j : ℝ) * (n j : ℝ) ^ 2 + ∑ j, θ j * (n j : ℝ) : ℝ) : ℂ))‖
808             ≤ C * Real.sqrt ((∏ s, (Finset.Icc (lo s) (hi s)).card : ℕ) : ℝ) ∧
809           ∀ (w₀ : Fin k → ℤ) (b₀ : ℤ), w₀ ≠ 0 → (w₀, b₀) ∈ L → (-w₀, -b₀) ∈ L →
810             ‖∑ n ∈ (Fintype.piFinset fun s => Finset.Icc (lo s) (hi s)).filter
811                   (fun x => ∀ p ∈ L, 0 ≤ (∑ s, p.1 s * x s) + p.2),
812                 Complex.exp (2 * (Real.pi : ℂ) * Complex.I *
813                   ((α * ∑ j, (c j : ℝ) * (n j : ℝ) ^ 2 + ∑ j, θ j * (n j : ℝ) : ℝ) : ℂ))‖
814               ≤ C * Real.sqrt
815                   (((∏ s, (Finset.Icc (lo s) (hi s)).card : ℕ) : ℝ) /
816                     ((max 1 ((Finset.univ.filter fun s : Fin k => w₀ s ≠ 0).sup
817                       fun s => (Finset.Icc (lo s) (hi s)).card) : ℕ) : ℝ)) :=
818   @_root_.thm_box
```

# Definition 3 `def:equicomp`

Statements.lean:821

coverage: direct-definition

PAPER STATEMENT typeset from the verbatim TeX at lines 827–836

**Definition 3.** *Let $Aff_k(\mathbb{Z}) = \mathrm{GL}_k(\mathbb{Z}) \ltimes \mathbb{Z}^k$ be the group of affine transformations of the form $x \in \mathbb{R}^k \mapsto Ax + b \in \mathbb{R}^k$, where $A \in \mathrm{GL}_k(\mathbb{Z})$ and $b \in \mathbb{Z}^k$. Let $P, Q \subset \mathbb{Z}^k$ be $\mathbb{Z}$-polytopes. We say that $P$ and $Q$ are* $\mathrm{Aff}_k(\mathbb{Z})$-equidecomposable with complexity bounded by $R \geq 1$ *if there is a disjoint collection of $\mathbb{Z}$-polytopes $\mathcal{P}(L_1), \ldots, \mathcal{P}(L_r)$ and affine transformations $A_1 x + b_1, \ldots, A_r x + b_r \in Aff_k(\mathbb{Z})$ such that*

$$P = \bigsqcup_{i=1}^{r} \mathcal{P}(L_i) \quad \text{and} \quad Q = \bigsqcup_{i=1}^{r} (A_i \mathcal{P}(L_i) + b_i),$$

*where*

$$r \leq R\,,\ \forall i : \#L_i \leq R\,,\ \forall i : |A_i|_\infty \leq R \ \text{ and } \ \forall i \forall (v, b) \in L_i : |v|_1 \leq R.$$

Coverage: `direct-definition`, referring to `Paper.definition3` (D025), which states the paper's two disjoint unions as a set equality together with pairwise disjointness. The superseded weak encoding is kept as `Paper.definition3Weak` (D025w), displayed for comparison (see the `affine` context).

Conformance: no displayed statement uses either predicate. Proposition 4 writes the simplex–box equidecomposition out clause by clause; by its note, S_N, B_N ⊆ ℤ^k (lcm_k | N) satisfy `Paper.definition3 k R`. `IsUNmap` (D029), the ℕ-valued variant of Proposition 4's clauses that Theorem 5 and Proposition 10 use as a hypothesis (`simplex` context), is exported by `Compatibility.lean` as `Paper.Definition3.simplexBoxAdmissibleMap`. It is not a specialization of either Definition 3 predicate — its source-piece clause is the unique-membership form of D025w, it decomposes the target not at all (only a pointwise affine formula for U on each piece), and its complexity bounds are entrywise — so the abbreviation, formerly named `simplexBoxSpecialization`, has been renamed not to claim otherwise.

PAPER CONTEXT `affine`

Statements.lean:860

## Reading the affine-equidecomposition predicate

Known defect of `Paper.definition3Weak` (D025w), which is why it is not the paper's definition: its clause `x ∈ P ↔ ∃! i, x ∈ piece i` lets a point outside P lie in two pieces, and likewise for Q. Examples in $\mathbb{Z}^1$ with R = 2: pieces {0,…,3} and {1,…,3} with maps x↦x and x↦−x give P = {0} and Q = {−3,…,3}; for n ≥ 1, two copies of {0,…,n−1} with x↦x and x↦x+n give P = ∅ and Q = {0,…,2n−1}; pieces {0,…,n−1} and {n,…,2n−1} with x↦x and x↦x−n give P = {0,…,2n−1} and Q = ∅. None is an equidecomposition. `Paper.definition3` (D025) has the set equality and pairwise disjointness instead, and admits none of them.

`transitionSize` = `matInftyNorm` is the largest |entry| (the paper's |A|∞) and implies no invertibility; Definition 3 requires det = ±1, whereas Definition 23 requires only det ≠ 0.

Lean definitions for this comparison: `Paper.definition3`, `Paper.definition3Weak`, `matInftyNorm`, `transitionSize`, `pieceAffineImage`. Statements depending on these definitions: Theorem 21, Lemma 24, Lemma 25, Lemma 26, Proposition 27.

### D022 `pieceAffineImage`

Statements.lean:881

`Core.lean:9678` · Used by: Theorem 21, Lemma 24, Lemma 25, Lemma 26, Proposition 27 (indirectly).

```
887 def pieceAffineImage {d : ℕ} (u : Fin d → ℤ) (Φ : Matrix (Fin d) (Fin d) ℤ)
888     (S : Set (Fin d → ℤ)) : Set (Fin d → ℤ) :=
889   (fun y => u + Φ *ᵥ y) '' S
```

Library terms: G37 `f '' S`; G63 `Matrix.mulVec`.

### D023 `matInftyNorm`

Statements.lean:896

`Core.lean:9664` · Used by: Theorem 21, Lemma 24, Lemma 26, Proposition 27 (indirectly).

```
902 def matInftyNorm {d : ℕ} (Φ : Matrix (Fin d) (Fin d) ℤ) : ℕ :=
903   Finset.univ.sup fun ij : Fin d × Fin d => (Φ ij.1 ij.2).natAbs
```

Library terms: G29 `Finset.sup`; G46 `Int.natAbs`.

### D024 `transitionSize`

Statements.lean:910

`Core.lean:9695` · Used by: Lemma 24, Lemma 26, Proposition 27.

```
915 def transitionSize {d : ℕ} (Φ : Matrix (Fin d) (Fin d) ℤ) : ℕ := matInftyNorm Φ
```

### D025 `Paper.definition3`

Statements.lean:920

`Compatibility.lean:62` · Used by no displayed statement.

The paper's two displayed disjoint unions $P = \bigsqcup_i \mathscr{P}(L_i)$ and $Q = \bigsqcup_i (A_i\,\mathscr{P}(L_i) + b_i)$ are each stated as a set equality together with pairwise disjointness of the summands. `det.natAbs = 1` is $A_i \in \mathrm{GL}_k(\mathbb{Z})$; `transitionSize` (D024) is the paper's $|A_i|_\infty$. The predicate does not require P, Q to be $\mathbb{Z}$-polytopes or $R \geq 1$; for those it reads, by inspection, as the paper's definition (for $R = 0$ it says $P = Q = \varnothing$), a harmless generalization.

```
932 def definition3 (k R : ℕ) (P Q : Set (Fin k → ℤ)) : Prop :=
933   ∃ (r : ℕ) (L : Fin r → Finset ((Fin k → ℤ) × ℤ))
934       (A : Fin r → Matrix (Fin k) (Fin k) ℤ) (b : Fin r → Fin k → ℤ),
935     r ≤ R ∧
936     (∀ i, (L i).card ≤ R) ∧
937     (∀ i, (A i).det.natAbs = 1) ∧
938     (∀ i, transitionSize (A i) ≤ R) ∧
939     (∀ i, ∀ p ∈ L i, (∑ j, |p.1 j|) ≤ (R : ℤ)) ∧
940     P = ⋃ i : Fin r, polyPZ k (L i) ∧
941     (∀ i i' : Fin r, i ≠ i' → Disjoint (polyPZ k (L i)) (polyPZ k (L i'))) ∧
942     Q = ⋃ i : Fin r, pieceAffineImage (b i) (A i) (polyPZ k (L i)) ∧
943     ∀ i i' : Fin r, i ≠ i' →
944       Disjoint (pieceAffineImage (b i) (A i) (polyPZ k (L i)))
945         (pieceAffineImage (b i') (A i') (polyPZ k (L i')))
```

Library terms: G28 `Finset.card`; G38 `⋃ i, S i`; G39 `Disjoint`; G46 `Int.natAbs`; G62 `Matrix.det`.

LEAN · #CHECK line 952

```
952 #check @_root_.Paper.definition3
```

**D025w** **`Paper.definition3Weak`** Statements.lean:955

`Compatibility.lean:91` · Used by no displayed statement.

**Not the paper's Definition 3.** This is the superseded encoding, kept only so that the counterexamples in the `affine` context name a real declaration: it has the first five clauses of D025 verbatim, but replaces the two disjoint unions by unique-membership clauses, which force neither the pieces into P nor their images into Q.

```
966 def definition3Weak (k R : ℕ) (P Q : Set (Fin k → ℤ)) : Prop :=
967   ∃ (r : ℕ) (L : Fin r → Finset ((Fin k → ℤ) × ℤ))
968       (A : Fin r → Matrix (Fin k) (Fin k) ℤ) (b : Fin r → Fin k → ℤ),
969     r ≤ R ∧
970     (∀ i, (L i).card ≤ R) ∧
971     (∀ i, (A i).det.natAbs = 1) ∧
972     (∀ i, transitionSize (A i) ≤ R) ∧
973     (∀ i, ∀ p ∈ L i, (∑ j, |p.1 j|) ≤ (R : ℤ)) ∧
974     (∀ x, x ∈ P ↔ ∃! i : Fin r, x ∈ polyPZ k (L i)) ∧
975     ∀ y, y ∈ Q ↔
976       ∃! i : Fin r, y ∈ pieceAffineImage (b i) (A i) (polyPZ k (L i))
```

Library terms: G28 `Finset.card`; G41 `∃!`; G46 `Int.natAbs`; G62 `Matrix.det`.

LEAN · #CHECK line 982

```
982 #check @_root_.Paper.definition3Weak
```

# Proposition 4 `prop:UNmap`

Statements.lean:985

coverage: direct

PAPER CONTEXT `simplex` Statements.lean:989

## The simplex, rectangular index set, and admissible maps

Here k = K−1. Lean's `boxB` indexes s from 0, so its N/(k − s) is the paper's N/(k+1−s).

The paper defines B_N only when lcm_k | N; `boxB` is total (ℕ-division), so the divisibility is not built in. It is stated explicitly in Proposition 4, `PropUNmap`, `UniformBoundUnit.PropUNmap` and the Theorem 5 endpoints that use `boxB`; Proposition 10 uses N − N % lcmUpTo k, a multiple of lcm_k. Proposition 9 has no divisibility hypothesis, but with M = N−b its hypothesis `Set.BijOn U (simplexS k M) (boxB k M)` forces lcm_k | M: a bijection needs equal cardinalities, and #boxB k M = ∏_{s=1}^{k} (⌊M/s⌋+1) ≤ ∏_{s=1}^{k} (M/s+1) = choose(M+k,k) = #simplexS k M, with equality iff every s ≤ k divides M.

`IsUNmap` has ℕ-valued vectors, so it uses `polyP`, not `polyPZ`. Its source-piece clause has the unique-membership weakness of `Paper.definition3Weak`, but its `Set.BijOn` is a true bijection. It occurs as a hypothesis (`hU` of `theorem5OrthonormalBasis`, the U-hypotheses of `theorem5UniformBound` and `proposition10`, and inside both `PropUNmaps`), where its looseness can only strengthen them; the existence conjuncts of `theorem5UniformBound` and `proposition10` only show that their bounds are not vacuous. It is stricter in one respect: its affine clause must hold on all of `polyP k (L i)`, even outside S_N. When lcm_k | N, Proposition 4's witness, converted to ℕ-values with `toNat`, satisfies it (`basis` context).

> ORIGINAL PAPER DEFINITION/CONTEXT article.txt:424-426 · typeset from the verbatim TeX at lines 1017–1019
>
> $$\mathrm{lcm}_k := \mathrm{lcm}(1, 2, \ldots, k)$$

> ORIGINAL PAPER DEFINITION/CONTEXT article.txt:444-450 · typeset from the verbatim TeX at lines 1024–1030
>
> Define the following $\mathbb{Z}$-polytopes
>
> $$\mathcal{S}_N := \left\{ j \in \mathbb{Z}^k_{\geq 0} : j_1 + \cdots + j_k \leq N \right\} \quad \text{and} \quad \mathcal{B}_N := \prod_{s=1}^{k} \left\{ 0, 1, \ldots, \frac{N}{k+1-s} \right\}, \qquad \text{(eq:originalSimplexAndBox)}$$
>
> where $\mathcal{B}_N$ is only defined when $\mathrm{lcm}_k \mid N$.

Lean definitions for this comparison: `lcmUpTo`, `simplexS`, `boxB`, `IsUNmap`. Statements depending on these definitions: Theorem 5, Proposition 9, Proposition 10, Proposition 28.

### D026 `lcmUpTo`

Statements.lean:1038

`Core.lean:619` · Used by: Theorem 5, Proposition 10.

```
1043  def lcmUpTo (k : ℕ) : ℕ := (Finset.Icc 1 k).lcm id
```

Library terms: G25 `Finset.Icc`; G31 `Finset.lcm`.

### D027 simplexS

Statements.lean:1050

`Core.lean:633` · Used by: Proposition 9, Proposition 28; Theorem 5, Proposition 10 (indirectly).

```
def simplexS (k N : ℕ) : Finset (Fin k → ℕ) :=
  (Fintype.piFinset fun _ : Fin k => Finset.range (N + 1)).filter fun j => ∑ s, j s ≤ N
```

Library terms: G24 `Finset.range`; G26 `Finset.filter`; G33 `Fintype.piFinset`.

### D028 boxB

Statements.lean:1064

`Core.lean:636` · Used by: Theorem 5, Proposition 9, Proposition 10.

```
def boxB (k N : ℕ) : Finset (Fin k → ℕ) :=
  Fintype.piFinset fun s : Fin k => Finset.range (N / (k - s.val) + 1)
```

Library terms: G15 `x.val`; G24 `Finset.range`; G33 `Fintype.piFinset`.

### D029 IsUNmap

Statements.lean:1077

`Core.lean:642` · Used by: Theorem 5, Proposition 10.

```
def IsUNmap (k R N : ℕ) (U : (Fin k → ℕ) → Fin k → ℕ) : Prop :=
  Set.BijOn U (simplexS k N : Set (Fin k → ℕ)) (boxB k N : Set (Fin k → ℕ)) ∧
    ∃ (r : ℕ) (L : Fin r → Finset ((Fin k → ℤ) × ℤ))
      (A : Fin r → Matrix (Fin k) (Fin k) ℤ) (c : Fin r → Fin k → ℤ),
      r ≤ R ∧
      (∀ i, (L i).card ≤ R) ∧
      (∀ i, ∀ p ∈ L i, ∑ s, |p.1 s| ≤ (R : ℤ)) ∧
      (∀ i, (A i).det = 1 ∨ (A i).det = -1) ∧
      (∀ i s t, |A i s t| ≤ (R : ℤ)) ∧
      (∀ x : Fin k → ℕ, x ∈ simplexS k N ↔ ∃! i : Fin r, x ∈ polyP k (L i)) ∧
      ∀ i, ∀ x ∈ polyP k (L i), ∀ s,
        (U x s : ℤ) = Matrix.mulVec (A i) (fun t => (x t : ℤ)) s + c i s
```

Library terms: G28 `Finset.card`; G40 `Set.BijOn`; G41 `∃!`; G62 `Matrix.det`; G63 `Matrix.mulVec`.

**PAPER STATEMENT** typeset from the verbatim TeX at lines 1103–1114

**Proposition 4.** *Let $N \geq 0$ with $\mathrm{lcm}_k \mid N$. Then the $\mathbb{Z}$-polytopes $\mathcal{S}_N$ and $\mathcal{B}_N$ are $Aff_k(\mathbb{Z})$-equidecomposable with complexity $\ll_k 1$. Specifically, for each $N$, there exists a decomposition,*

$$\mathcal{S}_N := \bigsqcup_{i=1}^{r} \mathcal{P}(L_i)$$

*with $r \ll_k 1$, $\#L_i \ll_k 1$ for each $i$, and*

$$\sup_{\substack{1\leq i\leq r\\(v,b)\in L_i}} |v|_1 \ll_k 1,$$

*and a bijection $U_N : \mathcal{S}_N \to \mathcal{B}_N$ such that $U_{N|\mathcal{P}(L_i)} = A_i x + b_i$ with $|A_i|_\infty \ll_k 1$ for each $i$.*

Coverage: `direct`.

Reading note: the paper's image partition of B_N (the Q-side of Definition 3) follows, since the pieces partition S_N and U N is a bijection S_N→B_N equal to x ↦ A i x + c i on piece i. Hence for lcm_k | N the sets S_N, B_N ⊆ ℤ^k of the `Set.BijOn` clause satisfy `Paper.definition3 k R` (same r, L, A; b = c; bounds by inspection), with R depending only on k, as the proposition's first sentence asserts.

Library terms: G15 `x.val`; G25 `Finset.Icc`; G28 `Finset.card`; G31 `Finset.lcm`; G40 `Set.BijOn`; G62 `Matrix.det`; G63 `Matrix.mulVec`.

LEAN STATEMENT `proposition4` lines 1131–1156

```
theorem proposition4 :
  ∀ k : ℕ, 1 ≤ k →
      ∃ (R : ℕ) (U : ℕ → (Fin k → ℤ) → Fin k → ℤ),
        ∀ N : ℕ, (Finset.Icc 1 k).lcm id ∣ N →
          Set.BijOn (U N)
            {x : Fin k → ℤ | (∀ s, 0 ≤ x s) ∧ ∑ s, x s ≤ (N : ℤ)}
            {l : Fin k → ℤ | ∀ s : Fin k, 0 ≤ l s ∧ l s ≤ ((N / (k - s.val) : ℕ) : ℤ)} ∧
          ∃ (r : ℕ) (L : Fin r → Finset ((Fin k → ℤ) × ℤ))
            (A : Fin r → Matrix (Fin k) (Fin k) ℤ) (c : Fin r → Fin k → ℤ),
            r ≤ R ∧
            (∀ i, (L i).card ≤ R) ∧
            (∀ i, ∀ p ∈ L i, ∑ s, |p.1 s| ≤ (R : ℤ)) ∧
            (∀ i, (A i).det = 1 ∨ (A i).det = -1) ∧
            (∀ i s t, |A i s t| ≤ (R : ℤ)) ∧
            (∀ i, ∀ x : Fin k → ℤ,
              (∀ p ∈ L i, 0 ≤ (∑ s, p.1 s * x s) + p.2) →
                (∀ s, 0 ≤ x s) ∧ ∑ s, x s ≤ (N : ℤ)) ∧
            (∀ i j, i ≠ j → ∀ x : Fin k → ℤ,
              ¬ ((∀ p ∈ L i, 0 ≤ (∑ s, p.1 s * x s) + p.2) ∧
                 (∀ p ∈ L j, 0 ≤ (∑ s, p.1 s * x s) + p.2))) ∧
            (∀ x : Fin k → ℤ, (∀ s, 0 ≤ x s) → ∑ s, x s ≤ (N : ℤ) →
              ∃ i, ∀ p ∈ L i, 0 ≤ (∑ s, p.1 s * x s) + p.2) ∧
            ∀ i, ∀ x : Fin k → ℤ,
              (∀ p ∈ L i, 0 ≤ (∑ s, p.1 s * x s) + p.2) →
                U N x = Matrix.mulVec (A i) x + c i :=
  @_root_.prop_UNmap_explicit
```

# Theorem 5 `thm:onbexplicit`

Statements.lean:1159

coverage: all-degrees-conclusion

PAPER STATEMENT typeset from the verbatim TeX at lines 1165–1198

**Theorem 5.** *Let $K \geq 2$, and put $k = K - 1$. Let $a \geq 1$, and let $\alpha = [a_0; a_1, a_2, \ldots]$ be an irrational number with $a_i \leq a$ for all $i \geq 1$. For each $N \geq 1$, with $\mathrm{lcm}_k \mid N$, let $U_N$ be the map given by Proposition 4. Then:*

*(1) Let $D_N = \mathrm{diag}(\frac{1}{N/k+1}, \frac{1}{N/(k-1)+1}, \ldots, \frac{1}{N+1})$ and define for each $l \in \mathcal{B}_N$*

$$\varphi_l(z) := \sum_{\substack{j \in \mathbb{Z}^K_{\geq 0} \\ j_1+\cdots+j_K=N}} \binom{N}{j}^{1/2} z^j e\left(\alpha \sum_{s=1}^{K} j_s^2 + U_N(j_1, \ldots, j_k) \cdot D_N l\right), \tag{eq:originalBasis}$$

*where $\binom{N}{j} = N!/(j_1! \cdots j_K!)$. Then $\{\varphi_l : l \in \mathcal{B}_N\}$ is an orthonormal basis of $\mathrm{H}_N$.*

*(2) Assuming the estimate in Theorem 2,*

$$\sup_{\substack{N \geq 1 \\ \mathrm{lcm}_k \mid N}} \sup_{l \in \mathcal{B}_N} \|\varphi_l\|_{L^\infty(\mathbb{S}^{2K-1})} \ll_{K,a} 1.$$

*(3) The divisibility condition can be removed: assuming Theorem 2, then, for every $N \geq 0$, the space $\mathrm{H}_N$ admits an orthonormal basis $(\psi_\nu)_{1 \leq \nu \leq \dim \mathrm{H}_N}$ satisfying*

$$\sup_{1 \leq \nu \leq \dim \mathrm{H}_N} \|\psi_\nu\|_{L^\infty(\mathbb{S}^{2K-1})} \ll_{K,a} 1.$$

Coverage: `all-degrees-conclusion`.

Cross-references: `prop:UNmap` is Proposition 4; `thm:box` is Theorem 2.

Reading note: `theorem5OrthonormalBasis`, `theorem5UniformBound` and `theorem5` are parts 1, 2 and 3. Parts 1 and 2 quantify over every family U with `IsUNmap k R M (U M)` for all M ≥ 1 with lcm_k | M (part 1 also over every R): at least as strong as a statement for one fixed U_N. In part 2 the R and A, in part 3 the C, depend only on (k, a). Part 3 indexes its basis by `Fin ((N + k).choose k)`, of size dim H_N = binom(N+K−1,K−1) (K = k+1).

PAPER CONTEXT `basis` Statements.lean:1216

## Polynomial coefficients and theorem-valued hypotheses

In `basisPhi` (the coefficient of z^j in φ_l) the entries of D_N use the ℕ-division `N / (k - s.val)` (s.val = s−1), exact when lcm_k | N.

`ThmBox` and `PropUNmap` (root and `UniformBoundUnit` versions) are propositions packaging Theorem 2 and Proposition 4, not extra assumptions of the paper. Core proves `UniformBoundUnit.ThmBox` and `UniformBoundUnit.PropUNmap k` (k ≥ 1) as the private lemmas `agent4_onb_box_bridge` (Core.lean:17621) and `agent4_onb_UN_bridge` (Core.lean:17646); `thm_onb` (Core.lean:17675) applies `UniformBoundUnit.thm_onbexplicit_allDegrees` to them with a = 2, α = √2 (`bddPartQuot_two_sqrtTwo`, Core.lean:4377). So Theorem 1 depends on no unproved hypothesis, while Theorem 5 (`UniformBoundUnit` versions) and Proposition 10 (root versions) are stated conditionally on them (`theorem5OrthonormalBasis` assumes an `IsUNmap` family `hU` instead). Core also proves the root `ThmBox` (private `a4boxOct14_new_finished_v2`, Core.lean:14755), whence `thm_box`; the root `PropUNmap` occurs in Core only in its definition and as the hypothesis `hUN` of `prop_shiftedbound` (Core.lean:3119), and no declaration proves it.

All four follow from displayed theorems, and the derivations are now exported from `Compatibility.lean` as `Paper.theorem2_to_thmBox`, `Paper.theorem2_to_unitThmBox`, `Paper.proposition4_to_propUNmap` and `Paper.proposition4_to_unitPropUNmap`:

- root `ThmBox` unfolds to the type of `theorem2`;
- `UniformBoundUnit.ThmBox` repackages it (`BddPartQuot`, binder order, `*Z` box definitions) and omits `∀ s, lo s ≤ hi s`, harmlessly: for an empty box both bounds read 0 ≤ 0;

- `PropUNmap` and `UniformBoundUnit.PropUNmap k` ($k \geq 1$): keep the R, L, A, c of `proposition4` and take U′ N x = (U N x).toNat componentwise. The pieces lie in S_N, which U N maps onto the nonnegative B_N, so `toNat` is exact there, and the bijection, the `∃!` clause (from cover and disjointness) and the affine clause on `polyP` carry over.

`Paper.proposition4_to_unitPropUNmap` is derived from `Paper.proposition4_to_propUNmap`, not from Core's private `agent4_onb_UN_bridge`: `private` is not visible across the module boundary, and going through `Paper.proposition4` puts the content in the displayed statement anyway.

So `hbox`, `hUN` and `hU` are satisfiable and the Theorem 5 and Proposition 10 endpoints are not vacuous; likewise the first conjunct of `theorem5UniformBound` and of `proposition10` shows that the U-classes they bound are nonempty. Their second conjunct gives the bound for **every** complexity R, with A depending on k, a and R, rather than for one R chosen together with A; that is what lets them be applied to whatever map `proposition4` produces. Each implies its former `∃ R, ∃ A` form, by taking A at the R the first conjunct supplies.

Lean definitions for this comparison: `basisPhi`, `ThmBox`, `PropUNmap`, `UniformBoundUnit.ThmBox`, `UniformBoundUnit.PropUNmap`. Statements depending on these definitions: Theorem 5, Proposition 10.

### D030 `basisPhi`

Statements.lean:1270

`Core.lean:655` · Used by: Theorem 5.

```
noncomputable def basisPhi (k N : ℕ) (α : ℝ) (U : (Fin k → ℕ) → Fin k → ℕ)
    (l : Fin k → ℕ) : PolySpaceH (k + 1) N :=
  fun n =>
    (Real.sqrt (Nat.multinomial Finset.univ (n : Fin (k + 1) → ℕ)) : ℂ) *
      expTwoPiI
        (α * ∑ s, ((n : Fin (k + 1) → ℕ) s : ℝ) ^ 2 +
          ∑ s : Fin k,
            (U (fun t => (n : Fin (k + 1) → ℕ) t.castSucc) s : ℝ) * (l s : ℝ)
              / ((N / (k - s.val) : ℕ) + 1))
```

Library terms: G15 `x.val`; G17 `Fin.castSucc`; G53 `Nat.multinomial`; G57 `Real.sqrt`.

### D031 `ThmBox`

Statements.lean:1291

`Core.lean:1999` · Used by: Proposition 10.

```
def ThmBox : Prop :=
  ∀ (k r V h : ℕ) (a : ℝ), 1 ≤ k → 1 ≤ V → 1 ≤ h → 1 ≤ a →
    ∃ C > 0, ∀ α : ℝ, Irrational α →
      (∀ (i : ℕ) (x : ℝ), (GenContFract.of α).partDens.get? i = some x → x ≤ a) →
      ∀ c : Fin k → ℕ, (∀ j, 1 ≤ c j) → (∀ j, c j ≤ h) →
      ∀ lo hi : Fin k → ℤ, (∀ s, lo s ≤ hi s) →
      ∀ L : Finset ((Fin k → ℤ) × ℤ), L.card ≤ r →
        (∀ p ∈ L, ∑ s, |p.1 s| ≤ (V : ℤ)) →
        ∀ θ : Fin k → ℝ,
          ‖quadExpSum k α c θ (polyBoxZ k L lo hi)‖ ≤
              C * Real.sqrt (boxVol k lo hi) ∧
            ∀ (w₀ : Fin k → ℤ) (b₀ : ℤ), w₀ ≠ 0 → (w₀, b₀) ∈ L → (-w₀, -b₀) ∈ L →
              ‖quadExpSum k α c θ (polyBoxZ k L lo hi)‖ ≤
                C * Real.sqrt (boxSect k w₀ lo hi)
```

Library terms: G28 `Finset.card`; G55 `Irrational`; G57 `Real.sqrt`; G76 `GenContFract.of`.

### D032 PropUNmap

Statements.lean:1317

Core.lean:2014 · Used by: Proposition 10.

```
def PropUNmap : Prop :=
  ∀ k : ℕ, 1 ≤ k → ∃ (R : ℕ) (U : ℕ → (Fin k → ℕ) → Fin k → ℕ),
    ∀ M : ℕ, 1 ≤ M → lcmUpTo k | M → IsUNmap k R M (U M)
```

### D033 UniformBoundUnit.ThmBox

Statements.lean:1329

Core.lean:3206 · Used by: Theorem 5.

```
def ThmBox : Prop :=
  ∀ (k r V h : ℕ), 1 ≤ k → 1 ≤ V → 1 ≤ h → ∀ a : ℝ, 1 ≤ a →
    ∃ C : ℝ, 0 < C ∧
      ∀ α : ℝ, BddPartQuot a α →
        ∀ c : Fin k → ℕ, (∀ j, 1 ≤ c j ∧ c j ≤ h) →
        ∀ lo hi : Fin k → ℤ, ∀ L : Finset ((Fin k → ℤ) × ℤ), L.card ≤ r →
          (∀ p ∈ L, ∑ s, |p.1 s| ≤ (V : ℤ)) →
          ∀ θ : Fin k → ℝ,
            ‖∑ n ∈ latticePtsZ k L lo hi,
                expTwoPiI (α * ∑ j, (c j : ℝ) * (n j : ℝ) ^ 2 +
                  ∑ j, θ j * (n j : ℝ))‖
              ≤ C * Real.sqrt (boxVolZ k lo hi) ∧
            ∀ (w₀ : Fin k → ℤ) (b₀ : ℤ), w₀ ≠ 0 →
              (w₀, b₀) ∈ L → (-w₀, -b₀) ∈ L →
              ‖∑ n ∈ latticePtsZ k L lo hi,
                  expTwoPiI (α * ∑ j, (c j : ℝ) * (n j : ℝ) ^ 2 +
                    ∑ j, θ j * (n j : ℝ))‖
                ≤ C * Real.sqrt (boxSectZ k w₀ lo hi)
```

Library terms: G28 `Finset.card`; G57 `Real.sqrt`.

### D034 UniformBoundUnit.PropUNmap

Statements.lean:1358

Core.lean:3225 · Used by: Theorem 5.

```
def PropUNmap (k : ℕ) : Prop :=
  ∃ R : ℕ, ∀ N : ℕ, 1 ≤ N → lcmUpTo k | N →
    ∃ U : (Fin k → ℕ) → Fin k → ℕ, IsUNmap k R N U
```

Definitions used: degIdx (D002), PolySpaceH (D003), polyEval (D004), polyInner (D006), UniformBoundUnit.cSphere (D005), UniformBoundUnit.BddPartQuot (D021), expTwoPiI (D001), UniformBoundUnit.latticePtsZ (D017), UniformBoundUnit.boxVolZ (D018), UniformBoundUnit.boxLenZ (D019), UniformBoundUnit.boxSectZ (D020), UniformBoundUnit.ThmBox (D033), lcmUpTo (D026), simplexS (D027), boxB (D028), polyP (D008), IsUNmap (D029), UniformBoundUnit.PropUNmap (D034). Library terms: G52 Nat.choose; G67 Module.Basis ι R M.

LEAN STATEMENT theorem5 lines 1379–1388

```
theorem theorem5 :
  ∀ (k : ℕ) (hk : 1 ≤ k) (a : ℝ) (ha : 1 ≤ a)
    (hbox : _root_.UniformBoundUnit.ThmBox) (hUN : ∀ j : ℕ, 1 ≤ j → j ≤ k →
        ↪ _root_.UniformBoundUnit.PropUNmap j),
∃ C : ℝ, 0 < C ∧
     ∀ α : ℝ, _root_.UniformBoundUnit.BddPartQuot a α →
       ∀ N : ℕ,
         ∃ ψ : Module.Basis (Fin ((N + k).choose k)) ℂ (PolySpaceH (k + 1) N),
           (∀ i j, polyInner (k + 1) N (ψ i) (ψ j) = if i = j then 1 else 0) ∧
           ∀ ν, ∀ z ∈ _root_.UniformBoundUnit.cSphere (k + 1), ‖polyEval (k + 1) N (ψ ν) z‖
                ↪ ≤ C :=
  @_root_.UniformBoundUnit.thm_onbexplicit_allDegrees
```

Definitions used: lcmUpTo (D026), degIdx (D002), PolySpaceH (D003), polyInner (D006), boxB (D028), simplexS (D027), polyP (D008), IsUNmap (D029), expTwoPiI (D001), basisPhi (D030). Library terms: G37 f '' S; G55 Irrational; G69 Submodule.span; G76 GenContFract.of.

LEAN STATEMENT theorem5OrthonormalBasis lines 1396–1407

```
theorem theorem5OrthonormalBasis :
  ∀ (k : ℕ) (hk : 1 ≤ k)
    (a α : ℝ) (ha : 1 ≤ a) (hα : Irrational α)
    (hbpq : ∀ (i : ℕ) (x : ℝ), (GenContFract.of α).partDens.get? i = some x → x ≤ a)
    (R : ℕ) (U : ℕ → (Fin k → ℕ) → Fin k → ℕ)
    (hU : ∀ M : ℕ, 1 ≤ M → lcmUpTo k ∣ M → IsUNmap k R M (U M))
    (N : ℕ) (hN : 1 ≤ N) (hlcm : lcmUpTo k ∣ N),
(∀ l ∈ boxB k N, ∀ l' ∈ boxB k N,
       polyInner (k + 1) N (basisPhi k N α (U N) l) (basisPhi k N α (U N) l') =
         if l = l' then 1 else 0) ∧
     Submodule.span ℂ (basisPhi k N α (U N) '' (boxB k N : Set (Fin k → ℕ))) = ⊤ :=
  @_root_.thm_onbexplicit_orthonormalBasis
```

Definitions used: `lcmUpTo` (D026), `degIdx` (D002), `PolySpaceH` (D003), `polyEval` (D004), `boxB` (D028), `simplexS` (D027), `polyP` (D008), `IsUNmap` (D029), `expTwoPiI` (D001), `basisPhi` (D030), `UniformBoundUnit.cSphere` (D005), `UniformBoundUnit.BddPartQuot` (D021), `UniformBoundUnit.latticePtsZ` (D017), `UniformBoundUnit.boxVolZ` (D018), `UniformBoundUnit.boxLenZ` (D019), `UniformBoundUnit.boxSectZ` (D020), `UniformBoundUnit.ThmBox` (D033), `UniformBoundUnit.PropUNmap` (D034).

**LEAN STATEMENT** `theorem5UniformBound` lines 1418–1430

```
theorem theorem5UniformBound :
  ∀ (k : ℕ) (hk : 1 ≤ k) (a : ℝ) (ha : 1 ≤ a)
    (hbox : _root_.UniformBoundUnit.ThmBox) (hUN : _root_.UniformBoundUnit.PropUNmap k),
(∃ R : ℕ, ∀ N : ℕ, 1 ≤ N → lcmUpTo k | N →
        ∃ U₀ : (Fin k → ℕ) → Fin k → ℕ, IsUNmap k R N U₀) ∧
      ∀ R : ℕ, ∃ A : ℝ, 0 < A ∧
        ∀ α : ℝ, _root_.UniformBoundUnit.BddPartQuot a α →
          ∀ U : ℕ → (Fin k → ℕ) → Fin k → ℕ,
            (∀ M : ℕ, 1 ≤ M → lcmUpTo k | M → IsUNmap k R M (U M)) →
            ∀ N : ℕ, 1 ≤ N → lcmUpTo k | N →
              ∀ l ∈ boxB k N, ∀ z ∈ _root_.UniformBoundUnit.cSphere (k + 1),
                ‖polyEval (k + 1) N (basisPhi k N α (U N) l) z‖ ≤ A :=
  @_root_.Paper.Theorem5.uniformBound
```

# Lemma 6 `lem:poisson1d`

Statements.lean:1433

coverage: direct

PAPER CONTEXT `poisson` Statements.lean:1437

## Poisson weights, scales, and forward differences

`fwdDiff (1 : ℤ) (poissonQ lam) n` is Δq_λ(n) = q_λ(n+1)−q_λ(n). The definitions are total; λ > 0 is a hypothesis of Lemmas 6 and 7. The excerpt's a_{N,r}, c_N and c(n) do not occur in the Lemma 6 type.

ORIGINAL PAPER DEFINITION/CONTEXT article.txt:530-563 · typeset from the verbatim TeX at lines 1445–1478

for $\varphi \in \mathrm{H}_N$, it will be useful to define the following auxiliary function. For $\lambda > 0$, define the square root of the Poisson weight, extended by zero to the negative integers, by

$$q_\lambda(n) = \begin{cases} e^{-\lambda/2}\dfrac{\lambda^{n/2}}{\sqrt{n!}}, & n \geq 0, \\ 0, & n < 0. \end{cases}$$

Writing $r = (r_1, \ldots, r_K)$, for $N \geq 1$ define for every $n \in \mathbb{Z}^K$ the normalized multinomial weights

$$a_{N,r}(n) := \begin{cases} \binom{N}{n}^{1/2} \prod_{s=1}^K r_s^{n_s/2}, & n_1, \ldots, n_K \geq 0 \text{ and } n_1 + \cdots + n_K = N, \\ 0, & \text{otherwise.} \end{cases}$$

It is straightforward to see that when $a_{N,r}(n) \neq 0$ we have

$$a_{N,r}(n) = c_N \prod_{s=1}^K q_{Nr_s}(n_s) \quad \text{with} \quad c_N := e^{N/2}\frac{\sqrt{N!}}{N^{N/2}} \asymp N^{1/4}, \qquad \text{(eq:poissonFactorizationMultinomial)}$$

where the estimate for $c_N$ follows from Stirling's [DLMF, Section 5.11]. Note that any $\varphi \in \mathrm{H}_N$, in the new coordinates $z_s = \sqrt{r_s}e(\theta_s)$, can be expressed as

$$\varphi(z_1, \ldots, z_K) = \sum_{\substack{n \in \mathbb{Z}^K_{\geq 0} \\ n_1 + \cdots + n_K = N}} c(n) a_{N,r}(n) e(\theta \cdot n),$$

for some coefficients $c(n)$, where $\theta = (\theta_1, \ldots, \theta_K)$. In particular, $\|\varphi\|^2 = \binom{N+K-1}{K-1}^{-1} \sum |c(n)|^2$. Put

$$H_\lambda := (1+\lambda)^{1/2} \quad \text{and} \quad \rho_\lambda(n) := \frac{|n-\lambda|}{H_\lambda}.$$

Also set $\Delta b(j) = b(j+1) - b(j)$, for a sequence $b : \mathbb{Z} \to \mathbb{C}$. The next lemma produces the necessary bounds for the components of $a_{N,r}(n)$.

Lean definitions for this comparison: `poissonQ`, `poissonH`, `poissonRho`. Statements depending on these definitions: Lemma 6, Lemma 7.

### D035 poissonQ

Statements.lean:1486

Core.lean:930 · Used by: Lemma 6, Lemma 7.

```
noncomputable def poissonQ (lam : ℝ) (n : ℤ) : ℝ :=
  if 0 ≤ n then
    Real.exp (-lam / 2) * lam ^ ((n : ℝ) / 2) / Real.sqrt (Nat.factorial n.toNat)
  else 0
```

Library terms: G45 `Int.toNat`; G54 `Nat.factorial`; G56 `x ^ y`; G57 `Real.sqrt`; G58 `Real.exp`.

### D036 poissonH

Statements.lean:1502

Core.lean:935 · Used by: Lemma 6, Lemma 7.

```
noncomputable def poissonH (lam : ℝ) : ℝ := Real.sqrt (1 + lam)
```

Library terms: G57 `Real.sqrt`.

### D037 poissonRho

Statements.lean:1514

Core.lean:937 · Used by: Lemma 6.

```
noncomputable def poissonRho (lam : ℝ) (n : ℤ) : ℝ :=
  |(n : ℝ) - lam| / poissonH lam
```

PAPER STATEMENT typeset from the verbatim TeX at lines 1527–1534

**Lemma 6.** *We have, for all $n \in \mathbb{Z}$,*

$$q_\lambda(n) \ll H_\lambda^{-1/2} e^{-\rho_\lambda(n)/4} \quad \text{and} \quad |\Delta q_\lambda(n)| \ll H_\lambda^{-3/2} e^{-\rho_\lambda(n)/4}. \tag{eq:poissonPointwiseBound}$$

Coverage: `direct`.

Definitions used: `poissonQ` (D035), `poissonH` (D036), `poissonRho` (D037). Library terms: G56 `x ^ y`; G58 `Real.exp`; G75 `fwdDiff`.

LEAN STATEMENT `lemma6` lines 1544–1550

```
theorem lemma6 :
  ∃ C > 0, ∀ lam : ℝ, 0 < lam → ∀ n : ℤ,
      |poissonQ lam n| ≤
          C * poissonH lam ^ (-(1 : ℝ) / 2) * Real.exp (-poissonRho lam n / 4) ∧
        |fwdDiff (1 : ℤ) (poissonQ lam) n| ≤
          C * poissonH lam ^ (-(3 : ℝ) / 2) * Real.exp (-poissonRho lam n / 4) :=
  @_root_.lem_poisson1d
```

# Lemma 7 `lem:fouriermass`

Statements.lean:1553

coverage: direct

PAPER CONTEXT `fourier` Statements.lean:1557

## The hat function and Fourier transform convention

`seqFourier` takes real-valued sequences (the paper allows complex ones; real suffices here). Its `∑'` (0 for a non-summable family) cannot make Lemma 7 trivial: by the support conjunct each `poissonQLoc lam m` is finitely supported, so the `∑'` is the paper's finite sum (a continuous trigonometric polynomial in u), and the m-series is asserted `Summable` before its `∑'` is bounded.

> ORIGINAL PAPER DEFINITION/CONTEXT article.txt:609-618 · typeset from the verbatim TeX at lines 1568–1577
>
> Fix the hat function $\eta(x) = (1 - |x|)_+$. It is supported in $[-1, 1]$, bounded by one, 1-Lipschitz and satisfies
>
> $$\sum_{n\in\mathbb{Z}} \eta(x - n) = 1, \qquad \text{(eq:shiftFreePartitionUnity)}$$
>
> for every $x \in \mathbb{R}$. For a finitely supported $b : \mathbb{Z} \to \mathbb{C}$, write
>
> $$\widehat{b}(u) = \sum_{j\in\mathbb{Z}} b(j)e(-uj) \quad \text{for} \quad u \in \mathbb{R}.$$

Lean definitions for this comparison: `hatEta`, `poissonQLoc`, `seqFourier`. Statements depending on these definitions: Lemma 7.

### D038 `hatEta`

Statements.lean:1585

`Core.lean:1219` · Used by: Lemma 7 (indirectly).

```
1590  noncomputable def hatEta (x : ℝ) : ℝ := max (1 - |x|) 0
```

### D039 `poissonQLoc`

Statements.lean:1595

`Core.lean:1221` · Used by: Lemma 7.

```
1600  noncomputable def poissonQLoc (lam : ℝ) (m : ℤ) (n : ℤ) : ℝ :=
1601    poissonQ lam n * hatEta ((n : ℝ) / poissonH lam - (m : ℝ))
```

### D040 `seqFourier`

Statements.lean:1606

`Core.lean:1224` · Used by: Lemma 7.

```
1611  noncomputable def seqFourier (b : ℤ → ℝ) (u : ℝ) : ℂ :=
1612    ∑' j : ℤ, (b j : ℂ) * expTwoPiI (-(u * (j : ℝ)))
```

Library terms: G71 `Summable`.

PAPER STATEMENT typeset from the verbatim TeX at lines 1621–1634

**Lemma 7.** *For $m \in \mathbb{Z}$ put*

$$q_{\lambda,m}(n) := q_\lambda(n)\eta(n/H_\lambda - m). \tag{eq:localizedMultinomialSequence}$$

*Then $q_\lambda = \sum_m q_{\lambda,m}$, each $q_{\lambda,m}$ is supported on $\{n : |n/H_\lambda - m| < 1\}$ and*

$$\sum_{m\in\mathbb{Z}} \int_0^1 |\widehat{q_{\lambda,m}}(u)|\,\mathrm{d}u \ll H_\lambda^{-1/2}, \tag{eq:multinomialTotalFourierMass}$$

*where the implied constant is independent of $\lambda > 0$.*

Coverage: `direct`.

Definitions used: `poissonQ` (D035), `poissonH` (D036), `hatEta` (D038), `poissonQLoc` (D039), `expTwoPiI` (D001), `seqFourier` (D040). Library terms: G56 `x ^ y`; G71 `Summable`; G72 `∫ x, f x`.

LEAN STATEMENT `lemma7` lines 1645–1652

```
1645 theorem lemma7 :
1646   ∃ C > 0, ∀ lam : ℝ, 0 < lam →
1647       (∀ n : ℤ, HasSum (fun m : ℤ => poissonQLoc lam m n) (poissonQ lam n)) ∧
1648       (∀ m n : ℤ, 1 ≤ |(n : ℝ) / poissonH lam - (m : ℝ)| → poissonQLoc lam m n = 0) ∧
1649       (Summable fun m : ℤ => ∫ u in (0:ℝ)..1, ‖seqFourier (poissonQLoc lam m) u‖) ∧
1650       (∑' m : ℤ, ∫ u in (0:ℝ)..1, ‖seqFourier (poissonQLoc lam m) u‖) ≤
1651         C * poissonH lam ^ (-(1 : ℝ) / 2) :=
1652   @_root_.lem_fouriermass
```

# Definition 8 `def:shiftedbasis`

Statements.lean:1655

coverage: direct-definition

PAPER STATEMENT typeset from the verbatim TeX at lines 1661–1682

**Definition 8** (Shifted basis)**.** *Let $N \geq 0$ and set*

$$b := N - \operatorname{lcm}_k\left\lfloor \frac{N}{\operatorname{lcm}_k} \right\rfloor \quad \text{and} \quad N' = N - b.$$

*Also write*

$$D := \binom{N+k}{k} = \dim \mathrm{H}_N \quad \text{and} \quad D' := \binom{N'+k}{k} = \dim \mathrm{H}_{N'}. \tag{eq:dimensions}$$

*For $l \in \mathcal{B}_{N'}$ put*

$$\varphi_l^b(z) = \left(\frac{D}{D'}\right)^{1/2} \sum_{\substack{n \in \mathbb{Z}^K_{\geq 0},\\ n_1+\cdots+n_K=N \\ n_K \geq b}} \binom{N}{n}^{1/2} z^n \, e\Big(\alpha \sum_{s=1}^{K} n_s^2 + U_{N'}(n_1,\ldots,n_k)\cdot D_{N'} l\Big). \tag{eq:shiftedBasis}$$

Coverage: `direct-definition`.

Conformance with the statements: Definition 8 is encoded by `shiftedBasisPhi` (D041), the coefficients of φ^b_l in `PolySpaceH (k+1) N` (K = k+1), 0 when n_K < b. b, α, U, l are free arguments: `proposition9Orthogonality` allows any b ≤ N, real α and bijection U (which forces b ≡ N mod lcm_k, see Proposition 9); `proposition10` takes b = `N % lcmUpTo k`, the paper's residue, U_{N′} = `U N′` for any `IsUNmap` family U, l ∈ B_{N′} and α as in the paper. There the body is `eq:shiftedBasis`; the ℕ-division `(N - b) / (k - s.val)` (s.val = s−1) in D_{N′} is exact as lcm_k | N′. For b = 0 the values are those of `basisPhi` (D030), as the paper remarks.

PAPER CONTEXT `shifted` Statements.lean:1702

## Shifted coefficients, residue, and dimensions

Encoding and the hypotheses on b: see the Definition 8 conformance note.

Lean definitions for this comparison: `shiftedBasisPhi`. Statements depending on these definitions: Proposition 9, Proposition 10.

**D041** **shiftedBasisPhi** Statements.lean:1711

Core.lean:2018 · Used by: Proposition 9, Proposition 10.

```
1716  noncomputable def shiftedBasisPhi (k N b : ℕ) (α : ℝ)
1717      (U : (Fin k → ℕ) → Fin k → ℕ) (l : Fin k → ℕ) : PolySpaceH (k + 1) N :=
1718    fun n =>
1719      if b ≤ (n : Fin (k + 1) → ℕ) (Fin.last k) then
1720        (Real.sqrt (((N + k).choose k : ℝ) / ((N - b + k).choose k : ℝ)) : ℂ) *
1721          (Real.sqrt (Nat.multinomial Finset.univ (n : Fin (k + 1) → ℕ)) : ℂ) *
1722          expTwoPiI
1723            (α * ∑ s, ((n : Fin (k + 1) → ℕ) s : ℝ) ^ 2 +
1724              ∑ s : Fin k,
1725                (U (fun t => (n : Fin (k + 1) → ℕ) t.castSucc) s : ℝ) * (l s : ℝ)
1726                  / (((N - b) / (k - s.val) : ℕ) + 1))
1727      else 0
```

Library terms: G15 `x.val`; G16 `Fin.last`; G17 `Fin.castSucc`; G52 `Nat.choose`; G53 `Nat.multinomial`; G57 `Real.sqrt`.

LEAN · #CHECK line 1734

```
1734  #check @_root_.shiftedBasisPhi
```

# Proposition 9 `prop:shiftedONS`

Statements.lean:1737

coverage: component-only

**PAPER STATEMENT** typeset from the verbatim TeX at lines 1743–1747

**Proposition 9.** *The functions* $\{\varphi_l^b : l \in \mathcal{B}_{N'}\}$ *form an orthonormal basis of the subspace spanned by the monomials* $z^n \in \mathrm{H}_N$ *with* $n_K \geq b$.

Coverage: `component-only`.

Reading note: there is no divisibility hypothesis, but `hU` forces lcm_k | N−b (cardinality count in the `simplex` context), i.e. b ≡ N (mod lcm_k); the paper's residue is the case b < lcm_k. Not displayed, hence `component-only`: that the φ^b_l, l ∈ B_{N−b} (which lie in the span of the z^n with n_K ≥ b), span it; the paper gets this by counting dimensions (`Compatibility.lean` exports only the index bijection for that count, `Paper.Proposition9.indexEquiv`). `Compatibility.lean` deliberately exports no `Paper.proposition9`. `assemble_faces_clean` (Core.lean:4076) combines `shifted_orth` with the face lifts of Lemma 11 into binom(N+k,k) = dim H_{K,N} orthonormal polynomials of degree N.

Definitions used: `degIdx` (D002), `PolySpaceH` (D003), `polyInner` (D006), `simplexS` (D027), `boxB` (D028), `expTwoPiI` (D001), `shiftedBasisPhi` (D041). Library terms: G40 `Set.BijOn`.

**LEAN STATEMENT** `proposition9Orthogonality` lines 1769–1776

```
1769 theorem proposition9Orthogonality :
1770   ∀ (k N b : ℕ) (hb : b ≤ N) (α : ℝ)
1771     (U : (Fin k → ℕ) → Fin k → ℕ)
1772     (hU : Set.BijOn U (simplexS k (N-b) : Set _) (boxB k (N-b) : Set _))
1773     (l l' : Fin k → ℕ) (hl : l ∈ boxB k (N-b)) (hl' : l' ∈ boxB k (N-b)),
1774 polyInner (k+1) N (shiftedBasisPhi k N b α U l)
1775       (shiftedBasisPhi k N b α U l') = if l=l' then 1 else 0 :=
1776   @_root_.AllDegreesProof.shifted_orth
```

# Proposition 10 `prop:shiftedbound`

Statements.lean:1779

coverage: direct

**PAPER STATEMENT** typeset from the verbatim TeX at lines 1785–1792

**Proposition 10.** *Let $a \geq 1$ and let $\alpha = [a_0; a_1, a_2, \ldots]$ be an irrational number with $a_i \leq a$ for all $i \geq 1$. Let $N \geq 0$, with $b \in [0, \mathrm{lcm}_k)$ the residue of $N$ modulo $\mathrm{lcm}_k$ and $N' = N - b$, and let $\varphi_l^b(z)$ be defined as in* (eq:shiftedBasis)*. Then there is a constant $A = A(K, a)$ such that*

$$\sup_{l \in \mathcal{B}_{N'}} \sup_{z \in \mathbb{S}^{2K-1}} \left|\varphi_l^b(z)\right| \leq A.$$

Coverage: `direct`.

Cross-references: `eq:shiftedBasis` is the display in Definition 8.

Reading note: A and R depend only on k and a, so A = A(K,a). The bound holds for every family U with `IsUNmap k R M (U M)` for all M ≥ 1 with lcm_k | M (the paper uses one U_{N′}). If N < lcm_k, then N′ = 0 and `U 0` is unconstrained, but `boxB k 0` = {0} forces l = 0, and the polynomial is √binom(N+k,k)·e(αN²)·z_K^N, bounded on the sphere in terms of k. `hbox`, `hUN` stand for the paper's standing assumption here ("Throughout this section we will assume Theorem 2 and Proposition 4", `article.txt:678`).

Definitions used: `lcmUpTo` (D026), `degIdx` (D002), `PolySpaceH` (D003), `polyEval` (D004), `boxB` (D028), `simplexS` (D027), `polyP` (D008), `IsUNmap` (D029), `boxSide` (D011), `boxVol` (D012), `boxLen` (D013), `boxSect` (D014), `boxZ` (D010), `polyBoxZ` (D015), `expTwoPiI` (D001), `quadExpSum` (D016), `ThmBox` (D031), `PropUNmap` (D032), `shiftedBasisPhi` (D041). Library terms: G55 `Irrational`; G76 `GenContFract.of`.

**LEAN STATEMENT** `proposition10` lines 1816–1831

```
1816 theorem proposition10 :
1817   ∀ (hbox : _root_.ThmBox) (hUN : _root_.PropUNmap) (k : ℕ) (hk : 1 ≤ k)
1818     (a : ℝ) (ha : 1 ≤ a),
1819 (∃ (R : ℕ) (U : ℕ → (Fin k → ℕ) → Fin k → ℕ),
1820         ∀ M : ℕ, 1 ≤ M → lcmUpTo k ∣ M → IsUNmap k R M (U M)) ∧
1821       ∀ R : ℕ, ∃ A > 0,
1822         ∀ U : ℕ → (Fin k → ℕ) → Fin k → ℕ,
1823           (∀ M : ℕ, 1 ≤ M → lcmUpTo k ∣ M → IsUNmap k R M (U M)) →
1824           ∀ α : ℝ, Irrational α →
1825             (∀ (i : ℕ) (x : ℝ), (GenContFract.of α).partDens.get? i = some x → x ≤ a) →
1826           ∀ N : ℕ, ∀ l ∈ boxB k (N - N % lcmUpTo k),
1827           ∀ z : Fin (k + 1) → ℂ, ∑ s, ‖z s‖ ^ 2 = 1 →
1828             ‖polyEval (k + 1) N
1829                 (shiftedBasisPhi k N (N % lcmUpTo k) α
1830                   (U (N - N % lcmUpTo k)) l) z‖ ≤ A :=
1831   @_root_.Paper.proposition10
```

# Lemma 11 `lem:facenorm`

Statements.lean:1834

coverage: component-only

PAPER CONTEXT `faces` Statements.lean:1838

## The face subspaces and multiplication by the last coordinate

`faceLift k N i c` maps into z_K^i H_{K−1,N−i} (last exponent exactly i), which for i < N is smaller than the “at least i” space H_{K,N}^{(i)}; `faceScale`: see Lemma 11.

ORIGINAL PAPER DEFINITION/CONTEXT article.txt:836-843 · typeset from the verbatim TeX at lines 1846–1853

Let $N \geq 0$ and put

$$\mathrm{H}^{(i)}_{K,N} = \operatorname{span}\{z^n : n \in \mathbb{Z}^K_{\geq 0},\ n_1 + \cdots + n_K = N,\ n_K \geq i\}$$

and set $\mathrm{H}_{K,N} := \mathrm{H}^{(0)}_{K,N} = \mathrm{H}_N$. Let $b \in [0, \mathrm{lcm}_k)$ be the residue of $N$ modulo $\mathrm{lcm}_k$ and $N' = N - b$. We start by noticing that

$$\mathrm{H}_{K,N} = \mathrm{H}^{(b)}_{K,N} \bigoplus \Big( \bigoplus_{0 \leq i < b} z_K^i \mathrm{H}_{K-1,N-i} \Big), \qquad \text{(eq:Hfacesdecomp)}$$

Lean definitions for this comparison: `AllDegreesProof.faceLift`, `AllDegreesProof.faceScale`. Statements depending on these definitions: Lemma 11.

### D042 `AllDegreesProof.faceLift`

Statements.lean:1861

`Core.lean:3644` · Used by: Lemma 11.

```
1866 noncomputable def faceLift (k N i : ℕ) (c : ℂ) (v : PolySpaceH k (N-i)) :
1867     PolySpaceH (k+1) N := fun n =>
1868   if h : n.val (Fin.last k) = i then
1869     c * v ⟨Fin.init n.val, by
1870       apply (onb_mem_deg _ _ _).2
1871       have hn := (onb_mem_deg _ _ _).1 n.property
1872       rw [Fin.sum_univ_castSucc, h] at hn
1873       change (∑ j : Fin k, n.val j.castSucc) = N-i
1874       omega⟩
1875   else 0
```

Library terms: G15 `x.val`; G16 `Fin.last`; G17 `Fin.castSucc`; G18 `Fin.init`.

### D043 `AllDegreesProof.faceScale`

Statements.lean:1882

`Core.lean:3717` · Used by: Lemma 11.

```
1887 noncomputable def faceScale (k N i : ℕ) : ℝ :=
1888   Real.sqrt (((N+k).choose k : ℝ) * (N.choose i : ℝ) /
1889     ((N-i+k-1).choose (k-1) : ℝ))
```

Library terms: G52 `Nat.choose`; G57 `Real.sqrt`.

PAPER STATEMENT typeset from the verbatim TeX at lines 1898–1922

**Lemma 11** (The faces). *Let $K \geq 2$, $N \geq 1$ and $0 \leq i \leq N$ be integers. Put*

$$c_{N,i}^2 := \frac{\binom{N+K-1}{K-1}\binom{N}{i}}{\binom{N-i+K-2}{K-2}}. \tag{eq:facenorm}$$

*Then $c_{N,i} \asymp_{K,i} N^{(i+1)/2}$. Moreover:*

*(1) The map $\psi \mapsto c_{N,i}\, z_K^i \psi$ is a linear isometry from $\mathrm{H}_{K-1,N-i} \subset L^2(\mathbb{S}^{2K-3})$ onto $z_K^i \mathrm{H}_{K-1,N-i} \subset L^2(\mathbb{S}^{2K-1})$.*

*(2) If $\mathrm{H}_{K-1,N-i}$ has an orthonormal basis $\{\psi_1, \ldots, \psi_{\dim \mathrm{H}_{K-1,N-i}}\}$ with $\|\psi_\nu\|_{L^\infty(\mathbb{S}^{2K-3})} \leq C$, then the functions $c_{N,i} z_K^i \psi_\nu$ form an orthonormal basis of $z_K^i \mathrm{H}_{K-1,N-i}$ satisfying*

$$\left\| c_{N,i} z_K^i \psi_\nu \right\|_{L^\infty(\mathbb{S}^{2K-1})} \ll_{K,i} C\, N^{1/2}. \tag{eq:facesup}$$

Coverage: `component-only`.

Reading note: for k ≥ 1 and i ≤ N the ℕ-subtractions in `faceScale` are exact and `faceScale k N i`$^2$ is the paper's c_{N,i}$^2$ (K = k+1). `lemma11Isometry` assumes only i ≤ N (no k ≥ 1, N ≥ 1): at k = 0 the `k-1` of `faceScale` and the `K - 1` of `polyInner k (N-i)` truncate to 0. `lemma11SphereBound` needs no basis: it bounds the lift of any v with |v| ≤ C on the smaller sphere by C·√(binom(N+k,k)/binom(N−i+k−1,k−1)), a ratio asymptotic to N/k for fixed k, i; for N ≥ 1 this is `eq:facesup`. `lemma11Evaluation` identifies `faceLift k N i c` (i ≤ N) with the paper's map ψ ↦ c·z_K^i·ψ. Not displayed: c_{N,i} ≍_{K,i} N^{(i+1)/2}; that the map is onto z_K^i H_{K−1,N−i} (part 1); that the c_{N,i}·z_K^i·ψ_ν form an orthonormal basis of it (part 2). The last two follow from the isometry by linear algebra, given the evaluation formula and the `pairing` context. `Compatibility.lean` deliberately exports no `Paper.lemma11` (the asymptotics of the normalizing factor are not bundled as a theorem).

Definitions used: `degIdx` (D002), `PolySpaceH` (D003), `polyInner` (D006), `AllDegreesProof.faceLift` (D042), `AllDegreesProof.faceScale` (D043).

LEAN STATEMENT `lemma11Isometry` lines 1947–1952

```
1947 theorem lemma11Isometry :
1948   ∀ (k N i : ℕ) (hi : i ≤ N)
1949     (v w : PolySpaceH k (N-i)),
1950 polyInner (k+1) N (_root_.AllDegreesProof.faceLift k N i (_root_.AllDegreesProof.faceScale k
       ↪ N i) v)
1951       (_root_.AllDegreesProof.faceLift k N i (_root_.AllDegreesProof.faceScale k N i) w) =
             ↪ polyInner k (N-i) v w :=
1952   @_root_.AllDegreesProof.face_norm_inner
```

Definitions used: degIdx (D002), PolySpaceH (D003), polyEval (D004), UniformBoundUnit.cSphere (D005), AllDegreesProof.faceLift (D042), AllDegreesProof.faceScale (D043). Library terms: G52 Nat.choose; G57 Real.sqrt.

LEAN STATEMENT lemma11SphereBound lines 1960–1967

```
1960 theorem lemma11SphereBound :
1961   ∀ (k N i : ℕ) (hk : 1 ≤ k) (hi : i ≤ N)
1962     (v : PolySpaceH k (N-i)) (C : ℝ) (hC : 0 ≤ C)
1963     (hv : ∀ z ∈ UniformBoundUnit.cSphere k, ‖polyEval k (N-i) v z‖ ≤ C)
1964     (z : Fin (k+1) → ℂ) (hz : z ∈ UniformBoundUnit.cSphere (k+1)),
1965 ‖polyEval (k+1) N (_root_.AllDegreesProof.faceLift k N i (_root_.AllDegreesProof.faceScale k
       ↳ N i) v) z‖ ≤
1966       C * Real.sqrt (((N+k).choose k : ℝ) / ((N-i+k-1).choose (k-1) : ℝ)) :=
1967   @_root_.AllDegreesProof.face_eval_bound
```

Definitions used: degIdx (D002), PolySpaceH (D003), polyEval (D004), AllDegreesProof.faceLift (D042). Library terms: G16 Fin.last; G18 Fin.init.

LEAN STATEMENT lemma11Evaluation lines 1974–1979

```
1974 theorem lemma11Evaluation :
1975   ∀ (k N i : ℕ) (hi : i ≤ N) (c : ℂ)
1976     (v : PolySpaceH k (N-i)) (z : Fin (k+1) → ℂ),
1977 polyEval (k+1) N (_root_.AllDegreesProof.faceLift k N i c v) z =
1978       c * z (Fin.last k)^i * polyEval k (N-i) v (Fin.init z) :=
1979   @_root_.AllDegreesProof.faceLift_eval
```

# Lemma 12 `lem:absorption`

Statements.lean:1982

coverage: direct

**PAPER STATEMENT** typeset from the verbatim TeX at lines 1988–2005

**Lemma 12** (Absorption Lemma)**.** *Let $D \geq 2$. Let $X$ be a set. Let $V$ be a $D$-dimensional complex vector space of functions $f : X \to \mathbb{C}$. Let $\varphi_1, \ldots, \varphi_D$ be an orthonormal basis of $V$ with respect to an inner product $\langle \cdot, \cdot \rangle : V \times V \to \mathbb{C}$. Let $A, B \geq 0$ and let $D_2$ be an integer with $1 \leq D_2 \leq D/2$. Suppose*

$$\sup_{x \in X} |\varphi_j(x)| \leq B \quad (1 \leq j \leq D_2) \quad \text{and} \quad \sup_{x \in X} |\varphi_j(x)| \leq A \quad (D_2 < j \leq D).$$

*Then $V$ has an orthonormal basis $\psi_1, \ldots, \psi_D$ with respect to the inner product $\langle \cdot, \cdot \rangle$ such that*

$$\max_{1 \leq s \leq D} \sup_{x \in X} |\psi_s(x)| \leq 10A + 10B\sqrt{D_2/D}.$$

Coverage: `direct`.

Reading note: `ip` is only sesquilinear (no positivity or conjugate symmetry); the paper's construction ($\psi = Q\varphi$, Q real orthogonal, $QQ^T = I$) needs no more. Indices are 0-based: `(j : ℕ) < D₂` is $1 \leq j \leq D_2$, `D₂ ≤ (j : ℕ)` is $D_2 < j \leq D$, and `2 * D₂ ≤ D` is $D_2 \leq D/2$. So the Lean hypotheses follow from the paper's; the conclusion is the same.

Library terms: G15 `x.val`; G57 `Real.sqrt`; G67 `Module.Basis ι R M`; G69 `Submodule.span`; G70 `V →ₗ[ℂ] V →ₗ⋆[ℂ] ℂ`.

**LEAN STATEMENT** `lemma12` lines 2021–2035

```
2021 theorem lemma12 :
2022   ∀ {X : Type*} {D : ℕ} (hD : 2 ≤ D)
2023     (V : Submodule ℂ (X → ℂ))
2024     (ip : V →ₗ[ℂ] V →ₗ⋆[ℂ] ℂ)
2025     (φ : Module.Basis (Fin D) ℂ V)
2026     (hφ : ∀ i j, ip (φ i) (φ j) = if i = j then 1 else 0)
2027     (A B : ℝ) (hA : 0 ≤ A) (hB : 0 ≤ B)
2028     (D₂ : ℕ) (hD₂ : 1 ≤ D₂) (hD₂' : 2 * D₂ ≤ D)
2029     (hbigB : ∀ j : Fin D, (j : ℕ) < D₂ → ∀ x : X, ‖(φ j : X → ℂ) x‖ ≤ B)
2030     (hbigA : ∀ j : Fin D, D₂ ≤ (j : ℕ) → ∀ x : X, ‖(φ j : X → ℂ) x‖ ≤ A),
2031 ∃ ψ : Module.Basis (Fin D) ℂ V,
2032       (∀ i j, ip (ψ i) (ψ j) = if i = j then 1 else 0) ∧
2033       ∀ s : Fin D, ∀ x : X,
2034         ‖(ψ s : X → ℂ) x‖ ≤ 10 * A + 10 * B * Real.sqrt ((D₂ : ℝ) / (D : ℝ)) :=
2035   @_root_.absorption_lemma
```

# Definition 13 `def:balanced`

Statements.lean:2038

coverage: direct-definition

PAPER STATEMENT typeset from the verbatim TeX at lines 2044–2056

**Definition 13.** *Let $k \geq 1$ and $R \geq 1$. Let $I = \prod_{j=1}^{k} I_j$ be a box and $L \subset \mathbb{Z}^k \times \mathbb{Z}$ a finite set of constraints. We say that the pair $(L, I)$ is* $R$-balanced *if*

$$\#L \leq R \quad \text{and} \quad 0 < |v|_1 \leq R \quad \text{for all } (v, b) \in L,$$

*and*

$$\#I_i \leq R\,\#I_j \quad \text{for all } i, j.$$

*The locations of the intervals and the constants $b$ are unrestricted.*

Coverage: `direct-definition`.

Conformance with the statements: Definition 13 is encoded by `IsBalancedPair` (D044). Only `theorem14` uses it, as a hypothesis on (L, I), I = `boxZ k lo hi`; the body is the paper's three conditions with #I_j = `boxSide k lo hi j` (0 for an empty interval). k ≥ 1 and R ≥ 1 are not part of the predicate (unlike `IsBalancedWrt`, Definition 20) but are `hk`, `hR` of `theorem14`. R : ℕ and `lo ≤ hi`: see the `balanced` context.

PAPER CONTEXT `balanced` Statements.lean:2072

## Global balance is not componentwise balance

This is global balance (all pairs of sides), unlike Definition 20 (only coordinates connected in the constraint graph). R : ℕ, while Definition 13 allows real R ≥ 1, loses nothing: the paper's Theorem 14 takes an integer R. `theorem14` adds `∀ s, lo s ≤ hi s`, excluding only empty boxes, where the sum is empty and `boxVol` = 0, so the bound would read 0 ≤ 0.

Lean definitions for this comparison: `IsBalancedPair`. Statements depending on these definitions: Theorem 14.

### D044 `IsBalancedPair`

Statements.lean:2085

`Core.lean:4392` · Used by: Theorem 14.

```
2090 def IsBalancedPair (k R : ℕ) (L : Finset ((Fin k → ℤ) × ℤ)) (lo hi : Fin k → ℤ) : Prop :=
2091   L.card ≤ R ∧
2092     (∀ p ∈ L, 0 < ∑ s, |p.1 s| ∧ ∑ s, |p.1 s| ≤ (R : ℤ)) ∧
2093     ∀ i j : Fin k, boxSide k lo hi i ≤ R * boxSide k lo hi j
```

Library terms: G28 `Finset.card`.

LEAN · #CHECK line 2099

```
2099 #check @_root_.IsBalancedPair
```

# Theorem 14 `thm:general`

Statements.lean:2102

coverage: direct

**PAPER STATEMENT** typeset from the verbatim TeX at lines 2108–2117

**Theorem 14.** *Let $k, R \geq 1$ be integers. Let $(L, I)$ be an $R$-balanced pair. Let $\alpha = [a_0; a_1, a_2, \ldots]$ be an irrational number with $a_i \leq R$ for all $i \geq 1$ and $c_1, \ldots, c_k$ be positive integers with $c_j \leq R$. Then*

$$\sup_{\theta \in \mathbb{R}^k} \left| \sum_{n \in \mathcal{P}(L) \cap I} e\Big(\alpha \sum_{j=1}^{k} c_j n_j^2 + \theta \cdot n\Big) \right| \ll_{k,R} \mathrm{vol}(I)^{1/2}.$$

Coverage: `direct`.

Definitions used: `boxSide` (D011), `boxVol` (D012), `boxZ` (D010), `polyBoxZ` (D015), `expTwoPiI` (D001), `quadExpSum` (D016), `IsBalancedPair` (D044). Library terms: G55 `Irrational`; G57 `Real.sqrt`; G76 `GenContFract.of`.

**LEAN STATEMENT** `theorem14` lines 2128–2139

```
2128 theorem theorem14 :
2129   ∀ (k R : ℕ) (hk : 1 ≤ k) (hR : 1 ≤ R),
2130 ∃ C > 0, ∀ α : ℝ, Irrational α →
2131       (∀ (i : ℕ) (x : ℝ),
2132         (GenContFract.of α).partDens.get? i = some x → x ≤ (R : ℝ)) →
2133       ∀ c : Fin k → ℕ, (∀ j, 1 ≤ c j) → (∀ j, c j ≤ R) →
2134       ∀ lo hi : Fin k → ℤ, (∀ s, lo s ≤ hi s) →
2135       ∀ L : Finset ((Fin k → ℤ) × ℤ), IsBalancedPair k R L lo hi →
2136       ∀ θ : Fin k → ℝ,
2137         ‖quadExpSum k α c θ (polyBoxZ k L lo hi)‖ ≤
2138           C * Real.sqrt (boxVol k lo hi) :=
2139   @_root_.thm_general
```

# Proposition 15 `prop:multiwall`

Statements.lean:2142

coverage: direct

PAPER CONTEXT `cutoff` Statements.lean:2146

## Smooth cutoffs, signs, and the multiwall summand

`IsSmoothCutoff` asks only C∞ and support in [−2,2] (no nonnegativity or |U| ≤ 1); ‖U^(m)‖₁ = ∫ℝ |U^(m)|, U^(0) = U. Each sign factor of `multiwallTerm` is ±1, as 2t+1 ≠ 0 for integer t.

> **ORIGINAL PAPER DEFINITION/CONTEXT** article.txt:1088-1098 · typeset from the verbatim TeX at lines 2154–2164
>
> Let $U$ be smooth and supported in $[-2,2]$, and let $\beta \in \mathbb{R}^k$. Then
>
> $$\Big|\sum_{h\in\mathbb{Z}^k}\prod_{i=1}^{r}\operatorname{sgn}(2(w_i\cdot h-B_i)+1)\prod_{j=1}^{k}U\Big(\frac{h_j}{P}\Big)e\Big(\alpha\sum_{j=1}^{k}c_jh_j^2+\beta\cdot h\Big)\Big| \ll q^{k/2},$$
>
> where the implied constant depends only on $A$, $k$, $r$, $V$, $C$ and $\|U^{(m)}\|_{L^1}$ for $0\le m\le 2$.

Lean definitions for this comparison: `IsSmoothCutoff`, `multiwallTerm`, `multiwallT`. Statements depending on these definitions: Proposition 15, Lemma 17.

### D045 `IsSmoothCutoff`

Statements.lean:2172

`Core.lean:5120` · Used by: Proposition 15, Lemma 17.

```
2177 def IsSmoothCutoff (U : ℝ → ℂ) : Prop :=
2178   ContDiff ℝ ((⊤ : ℕ∞) : WithTop ℕ∞) U ∧
2179     Function.support U ⊆ Set.Icc (-2 : ℝ) 2
```

Library terms: G36 `Set.Icc`; G43 `Function.support`; G74 `ContDiff ℝ ((⊤ : ℕ∞) : WithTop ℕ∞)`.

### D046 `multiwallTerm`

Statements.lean:2187

`Core.lean:7142` · Used by: Proposition 15.

```
2192 noncomputable def multiwallTerm (k r : ℕ) (α : ℝ) (c : Fin k → ℕ) (β : Fin k → ℝ)
2193     (U : ℝ → ℂ) (P : ℕ) (w : Fin r → Fin k → ℤ) (B : Fin r → ℤ)
2194     (h : Fin k → ℤ) : ℂ :=
2195   (∏ i, ((Int.sign (2 * ((∑ s, w i s * h s) - B i) + 1) : ℤ) : ℂ)) *
2196     (∏ j, U ((h j : ℝ) / (P : ℝ))) *
2197     expTwoPiI (α * ∑ j, (c j : ℝ) * (h j : ℝ) ^ 2 + ∑ j, β j * (h j : ℝ))
```

Library terms: G47 `Int.sign`.

### D047 `multiwallT`

Statements.lean:2204

`Core.lean:7149` · Used by: Proposition 15.

```
2209 noncomputable def multiwallT (k r : ℕ) (α : ℝ) (c : Fin k → ℕ) (β : Fin k → ℝ)
2210     (U : ℝ → ℂ) (P : ℕ) (w : Fin r → Fin k → ℤ) (B : Fin r → ℤ) : ℂ :=
2211   ∑' h : Fin k → ℤ, multiwallTerm k r α c β U P w B h
```

Library terms: G71 `Summable`.

PAPER STATEMENT typeset from the verbatim TeX at lines 2220–2243

**Proposition 15.** *Let $A, C, V, k \geq 1$ and $r \geq 0$. Let $\alpha = [a_0; a_1, a_2, \ldots]$ be an irrational number with $a_i \leq A$ for all $i \geq 1$, and let $a/q$ be a convergent of $\alpha$. Let $c_1, \ldots, c_k$ be integers with $1 \leq c_j \leq C$, let $1 \leq P \leq q$ be an integer, let $w_1, \ldots, w_r \in \mathbb{Z}^k \setminus \{0\}$ with $|w_i|_1 \leq V$, and let $B_1, \ldots, B_r$ be integers such that*

$$|w_i \cdot h - B_i| \leq \frac{q}{4} \qquad \text{whenever } |h_j| \leq 2P \text{ for all } j.$$

*Let $U$ be smooth and supported in $[-2, 2]$, and let $\beta \in \mathbb{R}^k$. Then*

$$\Big|\sum_{h \in \mathbb{Z}^k} \prod_{i=1}^{r} \operatorname{sgn}(2(w_i \cdot h - B_i) + 1) \prod_{j=1}^{k} U\Big(\frac{h_j}{P}\Big)\, e\Big(\alpha \sum_{j=1}^{k} c_j h_j^2 + \beta \cdot h\Big)\Big| \ll q^{k/2},$$

*where the implied constant depends only on $A$, $k$, $r$, $V$, $C$ and $\|U^{(m)}\|_{L^1}$ for $0 \leq m \leq 2$.*

Coverage: `direct`.

Reading note: Lean's `h` is the paper's C (bound on the c_j), Lean's `C` the implied constant; `V`, `h` : ℕ lose nothing (|w_i|₁ and the c_j are integers). `Lam` bounds ∫‖U‖, ∫‖U′‖, ∫‖U″‖ and `C` is chosen after it, so `C` is uniform in such U: at least as strong as the paper. The `nums s` and `dens s` equations make a/q the s-th convergent (nums 0 = ⌊α⌋, dens 0 = 1), so the extra `0 < q` and `Int.gcd a q = 1` hold automatically. The added `Summable` holds as the summand vanishes unless every |h_j| ≤ 2P.

Definitions used: `IsSmoothCutoff` (D045), `expTwoPiI` (D001), `multiwallTerm` (D046), `multiwallT` (D047). Library terms: G48 `Int.gcd`; G55 `Irrational`; G57 `Real.sqrt`; G71 `Summable`; G72 `∫ x, f x`; G73 `deriv`; G76 `GenContFract.of`.

LEAN STATEMENT `proposition15` lines 2262–2285

```
2262 theorem proposition15 :
2263   ∀ (A : ℝ) (k r V h : ℕ) (Lam : ℝ) (hA : 1 ≤ A) (hk : 1 ≤ k)
2264     (hV : 1 ≤ V) (hh : 1 ≤ h) (hLam : 1 ≤ Lam),
2265 ∃ C > 0, ∀ α : ℝ, Irrational α →
2266       (∀ (i : ℕ) (x : ℝ), (GenContFract.of α).partDens.get? i = some x → x ≤ A) →
2267       ∀ (s : ℕ) (a : ℤ) (q : ℕ), 0 < q →
2268         (a : ℝ) = (GenContFract.of α).nums s →
2269         ((q : ℕ) : ℝ) = (GenContFract.of α).dens s →
2270         Int.gcd a q = 1 →
2271         ∀ c : Fin k → ℕ, (∀ j, 1 ≤ c j) → (∀ j, c j ≤ h) →
2272         ∀ P : ℕ, 1 ≤ P → P ≤ q →
2273         ∀ w : Fin r → Fin k → ℤ, (∀ i, w i ≠ 0) →
2274           (∀ i, ∑ j, |w i j| ≤ (V : ℤ)) →
2275         ∀ B : Fin r → ℤ,
2276           (∀ x : Fin k → ℤ, (∀ j, |x j| ≤ 2 * (P : ℤ)) →
2277             ∀ i, 4 * |(∑ t, w i t * x t) - B i| ≤ (q : ℤ)) →
2278         ∀ U : ℝ → ℂ, IsSmoothCutoff U →
2279           (∫ t : ℝ, ‖U t‖) ≤ Lam →
2280           (∫ t : ℝ, ‖deriv U t‖) ≤ Lam →
2281           (∫ t : ℝ, ‖deriv (deriv U) t‖) ≤ Lam →
2282         ∀ β : Fin k → ℝ,
2283           Summable (multiwallTerm k r α c β U P w B) ∧
2284             ‖multiwallT k r α c β U P w B‖ ≤ C * Real.sqrt (q : ℝ) ^ k :=
2285   @_root_.prop_multiwall
```

# Lemma 16 `lem:sgn`

Statements.lean:2288

coverage: direct

PAPER STATEMENT typeset from the verbatim TeX at lines 2294–2304

**Lemma 16.** *Let $L \geq 4$ be an even integer. Then for every integer $h$ with $|h| < L/2$,*

$$\operatorname{sgn}(h) = \sum_{\substack{0<|\ell|<L/2 \\ \ell \text{ odd}}} \widehat{c}_L(\ell)\, e\Big(\frac{\ell h}{L}\Big) \quad \text{with} \quad \widehat{c}_L(\ell) = -\frac{2i}{L}\cot\Big(\frac{\pi\ell}{L}\Big). \tag{eq:saw}$$

*In particular, $|\widehat{c}_L(\ell)| \leq \frac{2}{\pi|\ell|}$ for $0 < |\ell| < L/2$.*

Coverage: `direct`.

`lemma16` is `eq:saw` (2|h| < L is |h| < L/2; for even L, `sgnIndex L` is the paper's index set); `lemma16OddCoefficient` is the cotangent formula for odd $\ell$ and every L; `lemma16CoefficientBound` (`norm_sgnCoeff_le`, Core.lean:4798) has the paper's $L \geq 4$ and $0 < |\ell| < L/2$ but not L even, a harmless generalization. `sgnCoeff` is 0 at even $\ell$ (`lemma16EvenCoefficient`); for even $\ell$ with $0 < |\ell| < L/2$ the paper's ĉ_L(ℓ) ≠ 0, so there the displayed bound is about 0. This is harmless: only odd $\ell$ occur in the expansion, and the proof of Proposition 15 uses ĉ_{2q}(v) = 0 for even v, as Lean does.

The last sentence of the lemma -- the bound at every $\ell$ with $0 < |\ell| < L/2$, even $\ell$ included -- is therefore exported on its own, as `Paper.lemma16CotWeightBound` (displayed below, after the other endpoints): it bounds `|sgnCotWeight L ℓ| = |ĉ_L(ℓ)|`, so it covers the even case, and it wraps Core's `abs_sgnCotWeight_le` (Core.lean:4764). On odd $\ell$ the two bounds agree, by `Lemma16.oddCoefficientFormula` together with ‖-i·z‖ = ‖z‖.

PAPER CONTEXT `sign` Statements.lean:2329

## The finite sign expansion

The definitions are total, with no condition on L; for the index set and even $\ell$ see Lemma 16's note.

Lean definitions for this comparison: `sgnCotWeight`, `sgnCoeff`, `sgnIndex`. Statements depending on these definitions: Lemma 16.

### D048 `sgnCotWeight`

Statements.lean:2339

`Core.lean:4452` · Used by: Lemma 16 (indirectly).

```
2344 noncomputable def sgnCotWeight (L : ℕ) (x : ℝ) : ℝ :=
2345   2 / (L : ℝ) * Real.cot (Real.pi * x / (L : ℝ))
```

Library terms: G58 `Real.exp`; G60 `Real.cot`.

### D049 `sgnCoeff`

Statements.lean:2352

`Core.lean:4455` · Used by: Lemma 16.

```
2357 noncomputable def sgnCoeff (L : ℕ) (l : ℤ) : ℂ :=
2358   if Odd l then -Complex.I * (sgnCotWeight L (l : ℝ) : ℂ) else 0
```

Library terms: G51 `Odd`; G58 `Real.exp`.

### D050 `sgnIndex`

Statements.lean:2365

`Core.lean:4458` · Used by: Lemma 16.

```
2370 def sgnIndex (L : ℕ) : Finset ℤ :=
2371   (Finset.Ioo (-((L : ℤ) / 2)) ((L : ℤ) / 2)).filter fun l => Odd l
```

Library terms: G25 `Finset.Icc`; G26 `Finset.filter`; G51 `Odd`.

Definitions used: `expTwoPiI` (D001), `sgnCotWeight` (D048), `sgnCoeff` (D049), `sgnIndex` (D050). Library terms: G47 `Int.sign`; G51 `Odd`.

LEAN STATEMENT `lemma16` lines 2381–2385

```
2381 theorem lemma16 :
2382   ∀ (L : ℕ) (hL : 4 ≤ L) (hLe : Even L) (h : ℤ) (hh : 2 * |h| < (L : ℤ)),
2383 (Int.sign h : ℂ) =
2384       ∑ l ∈ sgnIndex L, sgnCoeff L l * expTwoPiI (((l * h : ℤ) : ℝ) / (L : ℝ)) :=
2385   @_root_.lem_sgn
```

Definitions used: `sgnCotWeight` (D048), `sgnCoeff` (D049). Library terms: G51 `Odd`; G58 `Real.exp`; G60 `Real.cot`.

LEAN STATEMENT `lemma16OddCoefficient` lines 2391–2395

```
2391 theorem lemma16OddCoefficient :
2392   ∀ (L : ℕ) (l : ℤ) (hl : Odd l),
2393 sgnCoeff L l =
2394       -(2 * Complex.I / (L : ℂ)) * (Real.cot (Real.pi * (l : ℝ) / (L : ℝ)) : ℂ) :=
2395   @_root_.sgnCoeff_of_odd
```

Definitions used: `sgnCotWeight` (D048), `sgnCoeff` (D049). Library terms: G51 `Odd`.

LEAN STATEMENT `lemma16EvenCoefficient` lines 2401–2404

```
2401 theorem lemma16EvenCoefficient :
2402   ∀ (L : ℕ) (l : ℤ) (hl : Even l),
2403 sgnCoeff L l = 0 :=
2404   @_root_.sgnCoeff_of_even
```

Definitions used: `sgnCotWeight` (D048), `sgnCoeff` (D049). Library terms: G58 `Real.exp`.

LEAN STATEMENT `lemma16CoefficientBound` lines 2410–2414

```
2410 theorem lemma16CoefficientBound :
2411   ∀ (L : ℕ) (hL : 4 ≤ L) (l : ℤ) (hl : l ≠ 0)
2412     (hlL : 2 * |l| < (L : ℤ)),
2413 ‖sgnCoeff L l‖ ≤ 2 / (Real.pi * |(l : ℝ)|) :=
2414   @_root_.norm_sgnCoeff_le
```

Definitions used: `sgnCotWeight` (D048).

**LEAN STATEMENT** `lemma16CotWeightBound` lines 2419–2423

```
2419 theorem lemma16CotWeightBound :
2420   ∀ (L : ℕ) (hL : 4 ≤ L) (hLe : Even L) (l : ℤ) (hl : 0 < |l|)
2421     (hlL : |l| < (L : ℤ) / 2),
2422 |sgnCotWeight L (l : ℝ)| ≤ 2 / (Real.pi * |(l : ℝ)|) :=
2423   @_root_.Paper.lemma16CotWeightBound
```

# Lemma 17 `lem:poisson`

Statements.lean:2426

coverage: component

PAPER STATEMENT typeset from the verbatim TeX at lines 2432–2453

**Lemma 17.** *Let $U:\mathbb{R}\to\mathbb{C}$ be a smooth function supported in $[-2,2]$. Given $\lambda,\xi\in\mathbb{R}$ set*

$$W_U(\lambda;\xi):=\int_{\mathbb{R}} U(t)\,e(\lambda t^2+\xi t)\,dt$$

*and, for an integer $q\geq 1$ and $a,b\in\mathbb{Z}$, set*

$$G(a,b;q):=\sum_{x \bmod q} e\Big(\frac{ax^2+bx}{q}\Big).$$

*Then for any $N,q\geq 1$, $\delta,\theta\in\mathbb{R}$ and $a,b\in\mathbb{Z}$,*

$$\sum_{n\in\mathbb{Z}} e\Big(\Big(\frac{a}{q}+\delta\Big)n^2+\Big(\frac{b}{q}+\theta\Big)n\Big)\,U\Big(\frac{n}{N}\Big)=\frac{N}{q}\sum_{\ell\in\mathbb{Z}} G(a,b+\ell;q)\,W_U\Big(\delta N^2;\,N\Big(\theta-\frac{\ell}{q}\Big)\Big). \tag{eq:poisson}$$

*Moreover, for $\Lambda\geq 1$, $|\lambda|\leq\Lambda$ and $\xi\in\mathbb{R}$,*

$$|W_U(\lambda;\xi)|\ll_\Lambda (1+\|U'\|_1+\|U''\|_1)\cdot\frac{1}{1+|\xi|^2}.$$

Coverage: `component`.

`lemma17Summation` (the identity) and `lemma17Decay` (the bound on W_U, in the paper's form) cover the whole lemma. N is real, N ≥ 1; the ℓ-series is asserted `Summable`, and the n-sum is a `∑'` with finite support.

PAPER CONTEXT `integrals` Statements.lean:2464

## Oscillatory integrals and complete Gauss sums

`quadGaussSum` sums over the representatives 0,…,q−1 and takes a proof of q > 0; `quadPhase` is one summand.

Lean definitions for this comparison: `quadPhase`, `quadGaussSum`, `oscIntegralW`. Statements depending on these definitions: Lemma 17, Lemma 18.

### D051 `quadPhase`

Statements.lean:2474

`Core.lean:5040` · Used by: Lemma 17, Lemma 18 (indirectly).

```
2479 noncomputable def quadPhase (a b : ℤ) (q : ℕ) (_hq : 0 < q) (x : ℤ) : ℂ :=
2480   expTwoPiI (((a * x ^ 2 + b * x : ℤ) : ℝ) / (q : ℝ))
```

### D052 `quadGaussSum`

Statements.lean:2485

`Core.lean:5043` · Used by: Lemma 17, Lemma 18.

```
2490 noncomputable def quadGaussSum (a b : ℤ) (q : ℕ) (hq : 0 < q) : ℂ :=
2491   ∑ x ∈ Finset.range q, quadPhase a b q hq (x : ℤ)
```

Library terms: G24 `Finset.range`.

### D053 `oscIntegralW`

Statements.lean:2498

`Core.lean:5126` · Used by: Lemma 17.

```
2503 noncomputable def oscIntegralW (U : ℝ → ℂ) (lam xi : ℝ) : ℂ :=
2504   ∫ t : ℝ, U t * expTwoPiI (lam * t ^ 2 + xi * t)
```

Library terms: G72 `∫ x, f x`.

Definitions used: `expTwoPiI` (D001), `quadPhase` (D051), `quadGaussSum` (D052), `IsSmoothCutoff` (D045), `oscIntegralW` (D053). Library terms: G71 `Summable`.

**LEAN STATEMENT** `lemma17Summation` lines 2514–2528

```
2514 theorem lemma17Summation :
2515   ∀ (U : ℝ → ℂ) (hU : IsSmoothCutoff U)
2516     (N : ℝ) (hN : 1 ≤ N) (q : ℕ) (hq : 0 < q)
2517     (δ θ : ℝ) (a b : ℤ),
2518 Summable (fun l : ℤ =>
2519         quadGaussSum a (b + l) q hq *
2520           oscIntegralW U (δ * N ^ 2) (N * (θ - (l : ℝ) / (q : ℝ)))) ∧
2521       ∑' n : ℤ,
2522           expTwoPiI (((a : ℝ) / (q : ℝ) + δ) * (n : ℝ) ^ 2 +
2523             ((b : ℝ) / (q : ℝ) + θ) * (n : ℝ)) * U ((n : ℝ) / N) =
2524         ((N : ℂ) / (q : ℂ)) *
2525           ∑' l : ℤ,
2526             quadGaussSum a (b + l) q hq *
2527               oscIntegralW U (δ * N ^ 2) (N * (θ - (l : ℝ) / (q : ℝ))) :=
2528   @_root_.lem_poisson_summation
```

Definitions used: `IsSmoothCutoff` (D045), `expTwoPiI` (D001), `oscIntegralW` (D053). Library terms: G72 `∫ x, f x`; G73 `deriv`.

**LEAN STATEMENT** `lemma17Decay` lines 2534–2541

```
2534 theorem lemma17Decay :
2535   ∀ (Λ : ℝ) (hΛ : 1 ≤ Λ),
2536 ∃ C > 0, ∀ U : ℝ → ℂ, IsSmoothCutoff U →
2537       ∀ lam : ℝ, |lam| ≤ Λ → ∀ xi : ℝ,
2538         ‖oscIntegralW U lam xi‖ ≤
2539           C * (1 + (∫ t : ℝ, ‖deriv U t‖) + (∫ t : ℝ, ‖deriv (deriv U) t‖)) /
2540             (1 + |xi| ^ 2) :=
2541   @_root_.lem_poisson_decay
```

# Lemma 18 `lem:gaussphase`

Statements.lean:2544

coverage: direct

PAPER CONTEXT `gaussphase` Statements.lean:2548

## The periodic Gauss-phase factor

φ_{a,q} is defined in the paper's proof (excerpt below); `gaussPhasePhi` takes ā as an explicit argument. (Not the basis φ_l of Theorem 5.)

ORIGINAL PAPER DEFINITION/CONTEXT article.txt:1226-1229 · typeset from the verbatim TeX at lines 2555–2558

Let

$$\phi_{a,q}(x;c) := \frac{1}{\sqrt{q}} \cdot G(ca, x; q) e\Big(\frac{\overline{a}x^2}{4cq}\Big).$$

Lean definitions for this comparison: `gaussPhasePhi`. Statements depending on these definitions: Lemma 18.

**D054** **`gaussPhasePhi`** Statements.lean:2566

`Core.lean:5750` · Used by: Lemma 18.

```
2571 noncomputable def gaussPhasePhi (a abar : ℤ) (c q : ℕ) (hq : 0 < q) (x : ℤ) : ℂ :=
2572   (Real.sqrt (q : ℝ) : ℂ)⁻¹ * quadGaussSum ((c : ℤ) * a) x q hq *
2573     expTwoPiI ((abar : ℝ) * (x : ℝ) ^ 2 / (4 * (c : ℝ) * (q : ℝ)))
```

Library terms: G57 `Real.sqrt`.

PAPER STATEMENT typeset from the verbatim TeX at lines 2582–2590

**Lemma 18.** *Let $c, q \geq 1$ and $(a, q) = 1$. Denote by $\overline{a}$ the inverse of $a$ modulo $q$. Then, for all $b \in \mathbb{Z}$,*

$$G(ca, b; q) = \sqrt{q} \cdot \phi_{a,q}(b; c) \cdot e\Big(-\frac{\overline{a}\, b^2}{4cq}\Big),$$

*where $x \mapsto \phi_{a,q}(x; c)$ is periodic modulo $2c$ and $|\phi_{a,q}(x; c)| \leq \sqrt{2c}$ for all $x \in \mathbb{Z}$.*

Coverage: `direct`.

Reading note: `gaussPhasePhi` is the paper's φ_{a,q}, so the first conjunct is essentially its definition; the content is the 2c-periodicity and the bound √(2c). φ depends on ā through e(āx²/(4cq)); Lean states all three conjuncts for every integer `abar` with a·abar ≡ 1 mod q (the lemma fixes no representative), so Lean is at least as strong.

Definitions used: expTwoPiI (D001), quadPhase (D051), quadGaussSum (D052), gaussPhasePhi (D054). Library terms: G48 Int.gcd; G49 a ≡ b [ZMOD n]; G57 Real.sqrt.

LEAN STATEMENT lemma18 lines 2607–2617

```
theorem lemma18 :
  ∀ (a abar : ℤ) (c q : ℕ) (hc : 1 ≤ c) (hq : 0 < q)
    (ha : Int.gcd a q = 1) (habar : a * abar ≡ 1 [ZMOD (q : ℤ)]),
(∀ b : ℤ,
        quadGaussSum ((c : ℤ) * a) b q hq =
          (Real.sqrt (q : ℝ) : ℂ) * gaussPhasePhi a abar c q hq b *
            expTwoPiI (-((abar : ℝ) * (b : ℝ) ^ 2) / (4 * (c : ℝ) * (q : ℝ)))) ∧
      (∀ x : ℤ,
        gaussPhasePhi a abar c q hq (x + 2 * (c : ℤ)) = gaussPhasePhi a abar c q hq x) ∧
      (∀ x : ℤ, ‖gaussPhasePhi a abar c q hq x‖ ≤ Real.sqrt (2 * (c : ℝ))) :=
  @_root_.lem_gaussphase
```

# Lemma 19 `lem:weylgauss`

Statements.lean:2620

coverage: direct

PAPER CONTEXT `weyl` Statements.lean:2624

## Nearest-integer distance and progression sums

The paper's ‖x‖ (distance to the nearest integer) occurs in no displayed statement (Core's `intDist`, Core.lean:5928, only in auxiliary lemmas and proofs). `weylProgIco N R v r` is $\{h \in \mathbb{Z} : N \le h < N+R,\ h \equiv v \bmod r\}$.

> **ORIGINAL PAPER DEFINITION/CONTEXT** article.txt:1263-1266 · typeset from the verbatim TeX at lines 2632–2635
>
> Throughout this section $\alpha = [a_0; a_1, a_2, \ldots]$ is an irrational number with $a_i \le A$ for all $i \ge 1$, and $a/q$ is a convergent of $\alpha$, and for $x \in \mathbb{R}$ we write $\|x\|$ for the distance from $x$ to the nearest integer.

Lean definitions for this comparison: `weylProgIco`, `weylProgSum`. Statements depending on these definitions: Lemma 19.

### D055 `weylProgIco`

Statements.lean:2643

`Core.lean:7074` · Used by: Lemma 19 (indirectly).

```
2648 noncomputable def weylProgIco (N : ℤ) (R : ℝ) (v r : ℤ) : Finset ℤ :=
2649   (Finset.Ico N (N + ⌈R⌉)).filter fun h => (h : ℝ) < (N : ℝ) + R ∧ h ≡ v [ZMOD r]
```

Library terms: G25 `Finset.Icc`; G26 `Finset.filter`; G49 `a ≡ b [ZMOD n]`; G50 `⌈x⌉`.

### D056 `weylProgSum`

Statements.lean:2657

`Core.lean:7079` · Used by: Lemma 19.

```
2662 noncomputable def weylProgSum (alpha' beta : ℝ) (N : ℤ) (R : ℝ) (v r : ℤ) : ℂ :=
2663   ∑ h ∈ weylProgIco N R v r, expTwoPiI (alpha' * (h : ℝ) ^ 2 + beta * (h : ℝ))
```

> **PAPER STATEMENT** typeset from the verbatim TeX at lines 2670–2678
>
> **Lemma 19.** *Let $A, M \ge 1$. Let $\alpha = [a_0; a_1, a_2, \ldots]$ be an irrational number with $a_i \le A$ for all $i \ge 1$. Let $a/q$ be a convergent of $\alpha$ and denote by $\overline{a} \in [0, q)$ the modular inverse of $a$ modulo $q$. Then, for all $1 \le R \le 2q$,*
>
> $$\sup_{\beta \in \mathbb{R}} \sup_{\substack{1 \le |m|,|n| \le M \\ m,n,N \in \mathbb{Z} \\ v,r \ge 1}} \left| \sum_{\substack{N \le h < N+R \\ h \equiv v \pmod r}} e\Big(\frac{m}{n} \cdot \frac{\overline{a}}{4q} \cdot h^2 + \beta h\Big) \right| \ll_{M,A} \sqrt{R \log(2R)}.$$

Coverage: `direct`.

Definitions used: expTwoPiI (D001), weylProgIco (D055), weylProgSum (D056). Library terms: G49 a ≡ b [ZMOD n]; G55 Irrational; G57 Real.sqrt; G59 Real.log; G76 GenContFract.of.

LEAN STATEMENT lemma19 lines 2689–2702

```
theorem lemma19 :
  ∀ (A M : ℝ) (hA : 1 ≤ A) (hM : 1 ≤ M),
∃ C > 0, ∀ α : ℝ, Irrational α →
      (∀ (i : ℕ) (x : ℝ), (GenContFract.of α).partDens.get? i = some x → x ≤ A) →
      ∀ s : ℕ, ∀ a q : ℤ,
        (a : ℝ) = (GenContFract.of α).nums s →
        (q : ℝ) = (GenContFract.of α).dens s →
        ∀ abar : ℤ, a * abar ≡ 1 [ZMOD q] → 0 ≤ abar → abar < q →
        ∀ R : ℝ, 1 ≤ R → R ≤ 2 * (q : ℝ) →
        ∀ (beta : ℝ) (m n : ℤ), 1 ≤ |m| → (|m| : ℝ) ≤ M → 1 ≤ |n| → (|n| : ℝ) ≤ M →
        ∀ N v r : ℤ, 1 ≤ v → 1 ≤ r →
          ‖weylProgSum ((m : ℝ) / (n : ℝ) * ((abar : ℝ) / (4 * (q : ℝ)))) beta N R v r‖ ≤
            C * Real.sqrt (R * Real.log (2 * R)) :=
  @_root_.lem_weylgauss
```

# Definition 20 `def:balanced-W`

Statements.lean:2705

coverage: direct-definition

PAPER CONTEXT `graph` Statements.lean:2709

## Constraint weight and the coordinate-interaction graph

ρ is a constraint-complexity measure, not the Poisson deviation ρ_λ.

ORIGINAL PAPER DEFINITION/CONTEXT article.txt:1553-1558 · typeset from the verbatim TeX at lines 2715–2720

For a finite $W \subset \mathbb{Z}^k \times \mathbb{Z}$ we write

$$\rho(W) = \max\Big\{\#W,\ \max_{(w,b)\in W} |w|_1\Big\},$$

and associate to $W$ the undirected graph $\Gamma_k(W)$ with vertices $\{1,\dots,k\}$, in which $i \neq j$ are joined by an edge whenever they occur together in some inequality of $W$, i.e. $w_i w_j \neq 0$ for some $(w,b) \in W$.

Lean definitions for this comparison: `constraintRho`, `constraintGraph`. Statements depending on these definitions: Theorem 21, Lemma 24, Lemma 25, Lemma 26, Proposition 27.

### D057 `constraintRho`

Statements.lean:2728

`Core.lean:9508` · Used by: Theorem 21, Lemma 24, Lemma 25, Lemma 26, Proposition 27 (indirectly).

```
2734 def constraintRho (k : ℕ) (W : Finset ((Fin k → ℤ) × ℤ)) : ℕ :=
2735   max W.card (W.sup fun p => (∑ s, |p.1 s|).toNat)
```

Library terms: G28 `Finset.card`; G29 `Finset.sup`; G45 `Int.toNat`.

### D058 `constraintGraph`

Statements.lean:2742

`Core.lean:9511` · Used by: Theorem 21, Proposition 27 (indirectly).

```
2747 def constraintGraph (k : ℕ) (W : Finset ((Fin k → ℤ) × ℤ)) : SimpleGraph (Fin k) :=
2748   SimpleGraph.fromRel fun i j => ∃ p ∈ W, p.1 i * p.1 j ≠ 0
```

Library terms: G77 `SimpleGraph`.

PAPER STATEMENT typeset from the verbatim TeX at lines 2757–2765

**Definition 20** (Componentwise balanced boxes)**.** *Let $k, R \geq 1$ and let $W \subset (\mathbb{Z}^k \setminus \{0\}) \times \mathbb{Z}$ be finite. A box $J = \prod_{j=1}^k J_j$ is* $R$-balanced with respect to $W$ *if*

$$\#J_i \leq R\#J_j \quad \text{whenever } i, j \text{ belong to the same component of } \Gamma_k(W).$$

Coverage: `direct-definition`.

Conformance with the statements: Definition 20 is `IsBalancedWrt` (D059): Γ_k(W) is `constraintGraph` (D058), "same component" is equality of `connectedComponentMk`, #J_i ≤ R#J_j is compared in ℕ via `boxSide` (k, R natural), and the standing hypotheses 1 ≤ k, 1 ≤ R and nonzero normals are conjuncts, so each conclusion `IsBalancedWrt …` asserts them. No displayed statement assumes it; it is concluded in Proposition 27 (last conjunct; for the normals see its reading note) and in Theorem 21 through `BalancedDecompData` (D060), with J_m = `boxZ k (lo' m) (hi' m)` (there the normals repeat its clause `∀ m, ∀ p ∈ W' m, p.1 ≠ 0`). `constraintRho` (D057) is not part of Definition 20 (it enters via `BalancedDecompData` and the field `rho_le_A` of `Piece`).

PAPER CONTEXT `components` Statements.lean:2786

## Balance along connected components

`IsBalancedWrt` bounds neither #W nor $|w|_1$ (separate conjuncts of the conclusions) and does not require nonempty intervals.

Lean definitions for this comparison: `IsBalancedWrt`. Statements depending on these definitions: Theorem 21, Proposition 27.

### D059 `IsBalancedWrt`

Statements.lean:2796

`Core.lean:9552` · Used by: Proposition 27; Theorem 21 (indirectly).

```
2801 def IsBalancedWrt (k R : ℕ) (W : Finset ((Fin k → ℤ) × ℤ)) (lo hi : Fin k → ℤ) : Prop :=
2802   1 ≤ k ∧ 1 ≤ R ∧ (∀ p ∈ W, p.1 ≠ 0) ∧
2803     ∀ i j : Fin k,
2804       (constraintGraph k W).connectedComponentMk i =
2805           (constraintGraph k W).connectedComponentMk j →
2806         boxSide k lo hi i ≤ R * boxSide k lo hi j
```

Library terms: G78 `SimpleGraph.connectedComponentMk`.

LEAN · #CHECK line 2812

```
2812 #check @_root_.IsBalancedWrt
```

# Theorem 21 `thm:decomp`

Statements.lean:2815

coverage: direct

PAPER STATEMENT typeset from the verbatim TeX at lines 2821–2845

**Theorem 21** (Decomposition with balanced enclosing boxes)**.** *Let $k, V, h \geq 1$ and $r \geq 0$ be integers, and let $c_1, \dots, c_k$ be positive integers with $c_j \leq h$. Let $W \subset \mathbb{Z}^k \times \mathbb{Z}$ with $\#W \leq r$ be such that $0 < |w|_1 \leq V$ for all $(w, b) \in W$. Suppose that $P = \mathcal{P}(W)$ is finite and nonempty, and let $I$ be the smallest box containing $P$. Then, there exists an integer $R = R(k, r, V, h)$ and a disjoint decomposition*

$$P = \bigsqcup_{m=1}^{M} (u_m + \Phi_m \mathcal{P}(W_m)) \tag{eq:decomposition}$$

*with the property that $M \leq R$, $u_m \in \mathbb{Z}^k$, $\Phi_m \in \mathbb{Z}^{k \times k}$ is non-singular, each $W_m \subset \mathbb{Z}^k \setminus \{0\} \times \mathbb{Z}$ is finite, the boxes $J_m$ are $R$-balanced with respect to $W_m$ and*

$$\mathcal{P}(W_m) \subseteq J_m \quad \text{and} \quad \operatorname{vol}(J_m) \leq R \operatorname{vol}(I). \tag{eq:boxed-conclusion}$$

*Furthermore each $\Phi_m^{\mathsf{T}} \operatorname{diag}(c_j) \Phi_m$ is diagonal, and*

$$\max\Big(|\Phi_m|_\infty, |\Phi_m^{\mathsf{T}} \operatorname{diag}(c_j) \Phi_m|_\infty, \rho(W_m)\Big) \leq R \qquad (1 \leq m \leq M).$$

*If moreover both $(w_0, b_0)$ and $(-w_0, -b_0)$ belong to $W$, then the decomposition* (eq:decomposition) *can be chosen so that, in addition,*

$$\operatorname{vol}(J_m) \leq R \frac{\operatorname{vol}(I)}{\operatorname{len}(w_0; I)} \qquad (1 \leq m \leq M). \tag{eq:decomposition-section}$$

Coverage: `direct`.

PAPER CONTEXT `decomposition` Statements.lean:2852

## The packaged decomposition conclusion

In `theorem21`, I is `boxZ k lo hi`: it contains P(W) and each `lo j`, `hi j` is attained on P(W), so they are the coordinatewise extrema. The image clause `⊆ polyPZ k W` is implied by the union equality; as y ↦ u_m+Φ_m y is injective (det Φ_m ≠ 0), union plus disjointness is the paper's disjoint union. The sectional bound reads vol(J_m)·len(w₀;I) ≤ R·vol(I), equivalent to the quotient as `boxLen` ≥ 1 (w₀ ≠ 0 since 0 < |w|₁ on W).

Lean definitions for this comparison: `BalancedDecompData`. Statements depending on these definitions: Theorem 21.

### D060 BalancedDecompData

Statements.lean:2867

`Core.lean:13945` · Used by: Theorem 21.

```
def BalancedDecompData (k R : ℕ) (c : Fin k → ℕ) (W : Finset ((Fin k → ℤ) × ℤ))
    (lo hi : Fin k → ℤ) (M : ℕ) (u : Fin M → Fin k → ℤ)
    (Φ : Fin M → Matrix (Fin k) (Fin k) ℤ) (W' : Fin M → Finset ((Fin k → ℤ) × ℤ))
    (lo' hi' : Fin M → Fin k → ℤ) : Prop :=
  M ≤ R ∧
    (∀ m, (Φ m).det ≠ 0) ∧
    (∀ m, pieceAffineImage (u m) (Φ m) (polyPZ k (W' m)) ⊆ polyPZ k W) ∧
    (∀ m m' : Fin M, m ≠ m' →
      Disjoint (pieceAffineImage (u m) (Φ m) (polyPZ k (W' m)))
        (pieceAffineImage (u m') (Φ m') (polyPZ k (W' m')))) ∧
    polyPZ k W = ⋃ m : Fin M, pieceAffineImage (u m) (Φ m) (polyPZ k (W' m)) ∧
    (∀ m, ∀ p ∈ W' m, p.1 ≠ 0) ∧
    (∀ m, IsBalancedWrt k R (W' m) (lo' m) (hi' m)) ∧
    (∀ m, polyPZ k (W' m) ⊆ (boxZ k (lo' m) (hi' m) : Set (Fin k → ℤ))) ∧
    (∀ m, boxVol k (lo' m) (hi' m) ≤ R * boxVol k lo hi) ∧
    (∀ m, ((Φ m).transpose * Matrix.diagonal (fun j => (c j : ℤ)) * Φ m).IsDiag) ∧
    (∀ m, matInftyNorm (Φ m) ≤ R) ∧
    (∀ m, matInftyNorm ((Φ m).transpose * Matrix.diagonal (fun j => (c j : ℤ)) * Φ m) ≤ R) ∧
    (∀ m, constraintRho k (W' m) ≤ R)
```

Library terms: G38 `⋃ i, S i`; G39 `Disjoint`; G62 `Matrix.det`; G64 `Matrix.transpose`; G65 `Matrix.diagonal`.

Definitions used: boxZ (D010), boxSide (D011), boxVol (D012), boxLen (D013), polyPZ (D007), constraintRho (D057), constraintGraph (D058), IsBalancedWrt (D059), matInftyNorm (D023), pieceAffineImage (D022), BalancedDecompData (D060). Library terms: G28 `Finset.card`; G42 `Set.Finite`.

LEAN STATEMENT theorem21 lines 2903–2922

```
theorem theorem21 :
  ∀ (k V h r : ℕ) (hk : 1 ≤ k) (hV : 1 ≤ V) (hh : 1 ≤ h),
∃ R : ℕ, 1 ≤ R ∧
      ∀ c : Fin k → ℕ, (∀ j, 1 ≤ c j) → (∀ j, c j ≤ h) →
      ∀ W : Finset ((Fin k → ℤ) × ℤ), W.card ≤ r →
        (∀ p ∈ W, 0 < ∑ s, |p.1 s| ∧ ∑ s, |p.1 s| ≤ (V : ℤ)) →
        (polyPZ k W).Finite → (polyPZ k W).Nonempty →
      ∀ lo hi : Fin k → ℤ,
        polyPZ k W ⊆ (boxZ k lo hi : Set (Fin k → ℤ)) →
        (∀ j, ∃ x ∈ polyPZ k W, x j = lo j) →
        (∀ j, ∃ x ∈ polyPZ k W, x j = hi j) →
        (∃ (M : ℕ) (u : Fin M → Fin k → ℤ) (Φ : Fin M → Matrix (Fin k) (Fin k) ℤ)
            (W' : Fin M → Finset ((Fin k → ℤ) × ℤ)) (lo' hi' : Fin M → Fin k → ℤ),
            BalancedDecompData k R c W lo hi M u Φ W' lo' hi') ∧
        ∀ (w₀ : Fin k → ℤ) (b₀ : ℤ), (w₀, b₀) ∈ W → (-w₀, -b₀) ∈ W →
          ∃ (M : ℕ) (u : Fin M → Fin k → ℤ) (Φ : Fin M → Matrix (Fin k) (Fin k) ℤ)
            (W' : Fin M → Finset ((Fin k → ℤ) × ℤ)) (lo' hi' : Fin M → Fin k → ℤ),
            BalancedDecompData k R c W lo hi M u Φ W' lo' hi' ∧
            ∀ m, boxVol k (lo' m) (hi' m) * boxLen k w₀ lo hi ≤ R * boxVol k lo hi :=
  @_root_.thm_decomp
```

# Definition 22 `def:piece`

Statements.lean:2925

coverage: direct-structure

PAPER STATEMENT typeset from the verbatim TeX at lines 2931–2942

**Definition 22** (Pieces). *A* piece *of dimension $d \geq 1$ is a quadruple $(W, J, Q, A)$, where $W \subset \mathbb{Z}^d \times \mathbb{Z}$ is finite, $J = \prod_{j=1}^{d} J_j$ is a box, $Q(x) = \sum_{j=1}^{d} \gamma_j x_j^2$ with integers $\gamma_j \geq 1$, and $A \geq 1$ is an integer, such that*

$$\mathcal{P}(W) \subseteq J \qquad \text{and} \qquad \max\{d, \rho(W), |(\gamma_j)|_\infty\} \leq A.$$

*Let $H = \min_{1 \leq j \leq d} \#J_j$. A coordinate $j$ is said to be* short *if $\#J_j \leq AH$ and* long *otherwise. We define $B := \{j : \#J_j \leq AH\}$ and $B^c = \{1, \dots, d\} \setminus B$ as the sets of short and long coordinates. An inequality $(w, b) \in W$ is said to be* mixed *if* supp $w$ *has nonempty intersection with both $B$ and $B^c$. The* index *of the piece is $(d, n_{\text{long}}, n_{\text{mix}})$, where $n_{\text{long}} = \#B^c$ and $n_{\text{mix}}$ is the number of mixed inequalities in $W$. Indices are ordered lexicographically.*

Coverage: `direct-structure`.

Conformance with the statements: Definition 22 is the structure `Piece` (D061): data fields W, lo, hi, gamma, A give (W, J, Q, A) with J = `boxZ d lo hi`, Q = `Piece.quad` (D062); the proof fields are the paper's conditions (max{d, ρ(W), |(γ_j)|∞} ≤ A as three fields), with d, γ_j, A natural; the derived notions are D063–D071 (index ordered via `toLex`). Uses: Lemma 24 only through transitions, decompositions and `Piece.prod`; Lemma 25 assumes `p.IsMixed (w, b)`, concludes `pPlus.index < p.index` and the bound $A^2H$, and states "… are pieces" as existence of a `Piece` (see `split`); Lemma 26 constrains `p.A`, `p.minSide`, `p.side` and the outputs' `A`, `gamma`, sides and intervals; Proposition 27 assumes `p.A = A`, `p.nLong = nlong`, `p.nMix = nmix` (index (d, n_long, n_mix), d the type parameter) and bounds the outputs' `A` and box volume. Theorem 21 does not use `Piece`. The structure does not require `lo j ≤ hi j`; Lemmas 25, 26 and Proposition 27 assume it, Lemma 24 does not.

PAPER CONTEXT `pieces`

Statements.lean:2967

## All fields of a piece and its lexicographic index

Core.lean's `Piece.IsShort`, `Piece.IsLong` (Core.lean:9595, 9598) occur in no statement and are not shown.

Lean definitions for this comparison: `Piece`, `Piece.quad`, `Piece.side`, `Piece.minSide`, `Piece.shortCoords`, `Piece.longCoords`, `Piece.IsMixed`, `Piece.mixedIneqs`, `Piece.nLong`, `Piece.nMix`, `Piece.index`. Statements depending on these definitions: Lemma 24, Lemma 25, Lemma 26, Proposition 27.

### D061 `Piece`

Statements.lean:2977

`Core.lean:9572` · Used by: Lemma 24, Lemma 25, Lemma 26, Proposition 27.

```
2982 structure Piece (d : ℕ) where
2983   W : Finset ((Fin d → ℤ) × ℤ)
2984   lo : Fin d → ℤ
2985   hi : Fin d → ℤ
2986   gamma : Fin d → ℕ
2987   A : ℕ
2988   one_le_dim : 1 ≤ d
2989   one_le_gamma : ∀ j, 1 ≤ gamma j
2990   one_le_A : 1 ≤ A
2991   poly_subset_box : polyPZ d W ⊆ (boxZ d lo hi : Set (Fin d → ℤ))
2992   dim_le_A : d ≤ A
2993   rho_le_A : constraintRho d W ≤ A
2994   gamma_le_A : ∀ j, gamma j ≤ A
```

### D062 `Piece.quad`

Statements.lean:2999

`Core.lean:9586` · Used by: Lemma 24, Lemma 25, Lemma 26, Proposition 27 (indirectly).

```
3005  def Piece.quad {d : ℕ} (p : Piece d) (x : Fin d → ℤ) : ℤ :=
3006    ∑ j, (p.gamma j : ℤ) * x j ^ 2
```

### D063 `Piece.side`

Statements.lean:3011

`Core.lean:9589` · Used by: Lemma 26; Lemma 25, Proposition 27 (indirectly).

```
3016  def Piece.side {d : ℕ} (p : Piece d) (j : Fin d) : ℕ := boxSide d p.lo p.hi j
```

### D064 `Piece.minSide`

Statements.lean:3021

`Core.lean:9591` · Used by: Lemma 25, Lemma 26; Proposition 27 (indirectly).

```
3026  def Piece.minSide {d : ℕ} (p : Piece d) : ℕ :=
3027    Finset.univ.inf'
3028      (Finset.univ_nonempty_iff.mpr (Fin.pos_iff_nonempty.mp p.one_le_dim)) p.side
```

Library terms: G30 `Finset.inf'`.

### D065 `Piece.shortCoords`

Statements.lean:3035

`Core.lean:9600` · Used by: Lemma 25, Proposition 27 (indirectly).

```
3040  def Piece.shortCoords {d : ℕ} (p : Piece d) : Finset (Fin d) :=
3041    Finset.univ.filter fun j => p.side j ≤ p.A * p.minSide
```

Library terms: G26 `Finset.filter`.

### D066 `Piece.longCoords`

Statements.lean:3048

`Core.lean:9603` · Used by: Lemma 25, Proposition 27 (indirectly).

```
3053  def Piece.longCoords {d : ℕ} (p : Piece d) : Finset (Fin d) :=
3054    Finset.univ.filter fun j => ¬ p.side j ≤ p.A * p.minSide
```

Library terms: G26 `Finset.filter`.

### D067 `Piece.IsMixed`

Statements.lean:3061

`Core.lean:9606` · Used by: Lemma 25; Proposition 27 (indirectly).

```
3066  def Piece.IsMixed {d : ℕ} (p : Piece d) (q : (Fin d → ℤ) × ℤ) : Prop :=
3067    (vecSupport q.1 ∩ p.shortCoords).Nonempty ∧
3068      (vecSupport q.1 ∩ p.longCoords).Nonempty
```

Library terms: G32 `insert`; G42 `Set.Finite`.

### D068 `Piece.mixedIneqs` Statements.lean:3075

`Core.lean:9610` · Used by: Lemma 25, Proposition 27 (indirectly).

```
3080  noncomputable def Piece.mixedIneqs {d : ℕ} (p : Piece d) :
3081      Finset ((Fin d → ℤ) × ℤ) := by
3082    classical
3083    exact p.W.filter fun q => p.IsMixed q
```

Library terms: G26 `Finset.filter`.

### D069 `Piece.nLong` Statements.lean:3090

`Core.lean:9615` · Used by: Proposition 27; Lemma 25 (indirectly).

```
3095  def Piece.nLong {d : ℕ} (p : Piece d) : ℕ := p.longCoords.card
```

Library terms: G28 `Finset.card`.

### D070 `Piece.nMix` Statements.lean:3102

`Core.lean:9617` · Used by: Proposition 27; Lemma 25 (indirectly).

```
3107  noncomputable def Piece.nMix {d : ℕ} (p : Piece d) : ℕ := p.mixedIneqs.card
```

Library terms: G28 `Finset.card`.

### D071 `Piece.index` Statements.lean:3114

`Core.lean:9619` · Used by: Lemma 25.

```
3119  noncomputable def Piece.index {d : ℕ} (p : Piece d) : ℕ ×ₗ (ℕ ×ₗ ℕ) :=
3120    toLex (d, toLex (p.nLong, p.nMix))
```

Library terms: G22 `toLex`.

LEAN · #CHECK lines 3126–3132

```
3126  #check @_root_.Piece
3127
3128  #check @_root_.Piece.quad
3129
3130  #check @_root_.Piece.side
3131
3132  #check @_root_.Piece.minSide
```

# Definition 23 `def:transition`

Statements.lean:3135

coverage: direct-structure

**PAPER STATEMENT** typeset from the verbatim TeX at lines 3141–3153

**Definition 23** (Transitions, size and decompositions)**.** *Let* $(W, J, Q, A)$ *and* $(W', J', Q', A')$ *be pieces of dimension* $d$*. A* transition *from* $(W, J, Q, A)$ *to* $(W', J', Q', A')$ *is a substitution* $x = u + \Phi y$*, with* $u \in \mathbb{Z}^d$ *and* $\Phi \in \mathbb{Z}^{d\times d}$ *non-singular, such that*

$$u + \Phi\,\mathcal{P}(W') \subseteq \mathcal{P}(W) \qquad \text{and} \qquad Q'(y) = Q(\Phi y).$$

*We define the size of a transition by* $|\Phi|_\infty$*. A* decomposition *of* $(W, J, Q, A)$ *is a finite family of transitions* $x = u_m + \Phi_m y$ *from* $(W, J, Q, A)$ *to pieces* $(W_m, J_m, Q_m, A_m)$ *such that*

$$\mathcal{P}(W) = \bigsqcup_m (u_m + \Phi_m \mathcal{P}(W_m)).$$

Coverage: `direct-structure`.

Conformance with the statements: Definition 23 is `IsTransition` (D072), whose fields are the paper's conditions (`quad_eq`, for all y ∈ ℤ^d, is equivalent to the polynomial identity, since a polynomial vanishing on ℤ^d is zero), with size `transitionSize` (D024) = `matInftyNorm` (D023) = |Φ|∞ and decompositions `IsDecomposition` (D073), indexed by `Fin M`. Uses: Lemma 24 composes transitions and decompositions and forms products (its two size bounds concern integer matrices only); Lemma 25 concludes `IsTransition p pPlus 0 1`, `IsTransition p pMinus 0 1` and `IsDecomposition (M := 2) …` (u = 0, Φ = 1: the identity map); Lemma 26 and Proposition 27 conclude `IsDecomposition p q …` and `transitionSize … ≤ C`. Theorem 21 uses none of these: `BalancedDecompData` repeats det Φ_m ≠ 0, image, disjointness and union for explicit u, Φ, W′, but has no pieces and no Q′(y) = Q(Φy) (instead $\Phi_m^{\mathrm{T}}$ diag(c) Φ_m is diagonal), and bounds |Φ_m|∞ by `matInftyNorm` directly.

PAPER CONTEXT `transitions` Statements.lean:3177

## Transitions versus unimodular equidecompositions

Non-singular means det Φ ≠ 0; unlike Definition 3, det Φ = ±1 is not required.

Lean definitions for this comparison: `IsTransition`, `IsDecomposition`. Statements depending on these definitions: Lemma 24, Lemma 25, Lemma 26, Proposition 27.

### D072 `IsTransition`

Statements.lean:3187

`Core.lean:9688` · Used by: Lemma 24, Lemma 25; Lemma 26, Proposition 27 (indirectly).

```
3193 structure IsTransition {d : ℕ} (p p' : Piece d) (u : Fin d → ℤ)
3194     (Φ : Matrix (Fin d) (Fin d) ℤ) : Prop where
3195   det_ne_zero : Φ.det ≠ 0
3196   image_subset : pieceAffineImage u Φ (polyPZ d p'.W) ⊆ polyPZ d p.W
3197   quad_eq : ∀ y : Fin d → ℤ, p'.quad y = p.quad (Φ *ᵥ y)
```

Library terms: G62 `Matrix.det`; G63 `Matrix.mulVec`.

### D073 `IsDecomposition`

Statements.lean:3204

`Core.lean:9698` · Used by: Lemma 24, Lemma 25, Lemma 26, Proposition 27.

```
3209  structure IsDecomposition {d M : ℕ} (p : Piece d) (q : Fin M → Piece d)
3210      (u : Fin M → Fin d → ℤ) (Φ : Fin M → Matrix (Fin d) (Fin d) ℤ) : Prop where
3211    transitions : ∀ m, IsTransition p (q m) (u m) (Φ m)
3212    pieces_disjoint : ∀ m m' : Fin M, m ≠ m' →
3213      Disjoint (pieceAffineImage (u m) (Φ m) (polyPZ d (q m).W))
3214        (pieceAffineImage (u m') (Φ m') (polyPZ d (q m').W))
3215    iUnion_eq : polyPZ d p.W =
3216      ⋃ m : Fin M, pieceAffineImage (u m) (Φ m) (polyPZ d (q m).W)
```

Library terms: G38 `⋃ i, S i`; G39 `Disjoint`.

LEAN · #CHECK lines 3222–3226

```
3222  #check @_root_.IsTransition
3223
3224  #check @_root_.transitionSize
3225
3226  #check @_root_.IsDecomposition
```

# Lemma 24 `lem:compose`

Statements.lean:3229

coverage: decomposition-component

PAPER STATEMENT typeset from the verbatim TeX at lines 3235–3251

**Lemma 24** (Composing and multiplying transitions)**.** *The following holds true:*

*(1) If $x = u + \Phi y$ is a transition from $(W, J, Q, A)$ to $(W', J', Q', A')$ and $y = v + \Psi z$ is one from $(W', J', Q', A')$ to $(W'', J'', Q'', A'')$, then $x = (u + \Phi v) + \Phi\Psi z$ is a transition from $(W, J, Q, A)$ to $(W'', J'', Q'', A'')$ of size at most $d\,|\Phi|_\infty|\Psi|_\infty$. Consequently, a decomposition of a piece, followed by decompositions of each of the resulting pieces, gives a valid decomposition of the original piece.*

*(2) If $(W_i, J_i, Q_i, A_i)$ is a piece of dimension $d_i$ for $i = 1, 2$, then $(W_1 \oplus W_2,\ J_1 \times J_2,\ Q_1 \oplus Q_2,\ A_1 + A_2)$ is a piece of dimension $d_1 + d_2$, where $W_1 \oplus W_2$ is the union of $W_1$ and $W_2$ with the vectors $w$ padded with zeros and $(Q_1 \oplus Q_2)(y_1, y_2) = Q_1(y_1) + Q_2(y_2)$. Transitions from $P_i := (W_i, J_i, Q_i, A_i)$ to $P_i' := (W_i', J_i', Q_i', A_i')$, $i = 1, 2$, combine coordinatewise into a transition from the product of $P_1$ and $P_2$ to the product of $P_1'$ and $P_2'$. Consequently, decompositions of two lower-dimensional pieces combine into a decomposition of their product.*

Coverage: `decomposition-component`.

PAPER CONTEXT `products` Statements.lean:3258

## Products, zero padding, and composition indices

`Piece.prod` (written `p₁.prod p₂`) witnesses that ($W_1 \oplus W_2$, $J_1 \times J_2$, $Q_1 \oplus Q_2$, $A_1 + A_2$) is a piece (Lemma 24(2)). Core.lean's block projections `leftPart`, `rightPart` (Core.lean:9907, 9910) occur only in auxiliary lemmas and proofs and are not shown. `sigmaPieceIndexEquiv` is only an indexing device. `lemma24` omits the size bound d|Φ|∞|Ψ|∞ for Φ_mΨ_{m,n}, which follows from `lemma24CompositionSizeBound`; `lemma24ProductSizeBound` is not in the paper.

Lean definitions for this comparison: `padLeftVec`, `padRightVec`, `oplusConstraints`, `blockDiagMat`, `Piece.prod`, `sigmaPieceIndexEquiv`. Statements depending on these definitions: Lemma 24.

### D074 `padLeftVec`

Statements.lean:3273

`Core.lean:9894` · Used by: Lemma 24 (indirectly).

```
3278 def padLeftVec (d₁ d₂ : ℕ) (w : Fin d₁ → ℤ) : Fin (d₁ + d₂) → ℤ :=
3279   Fin.append w 0
```

Library terms: G19 `Fin.append`.

### D075 `padRightVec`

Statements.lean:3286

`Core.lean:9897` · Used by: Lemma 24 (indirectly).

```
3291 def padRightVec (d₁ d₂ : ℕ) (w : Fin d₂ → ℤ) : Fin (d₁ + d₂) → ℤ :=
3292   Fin.append 0 w
```

Library terms: G19 `Fin.append`.

### D076 `oplusConstraints`

Statements.lean:3299

`Core.lean:9900` · Used by: Lemma 24 (indirectly).

```
3304  noncomputable def oplusConstraints (d₁ d₂ : ℕ)
3305      (W₁ : Finset ((Fin d₁ → ℤ) × ℤ)) (W₂ : Finset ((Fin d₂ → ℤ) × ℤ)) :
3306      Finset ((Fin (d₁ + d₂) → ℤ) × ℤ) := by
3307    classical
3308    exact W₁.image (fun p => (padLeftVec d₁ d₂ p.1, p.2)) ∪
3309      W₂.image (fun p => (padRightVec d₁ d₂ p.1, p.2))
```

Library terms: G27 `Finset.image`; G32 `insert`.

### D077 `blockDiagMat`

Statements.lean:3316

`Core.lean:9913` · Used by: Lemma 24.

```
3321  def blockDiagMat {d₁ d₂ : ℕ} (Φ₁ : Matrix (Fin d₁) (Fin d₁) ℤ)
3322      (Φ₂ : Matrix (Fin d₂) (Fin d₂) ℤ) :
3323      Matrix (Fin (d₁ + d₂)) (Fin (d₁ + d₂)) ℤ :=
3324    Matrix.reindex finSumFinEquiv finSumFinEquiv (Matrix.fromBlocks Φ₁ 0 0 Φ₂)
```

Library terms: G20 `finSumFinEquiv`; G66 `Matrix.fromBlocks`.

### D078 `Piece.prod`

Statements.lean:3331

`Core.lean:9971` · Used by: Lemma 24.

```
noncomputable def Piece.prod {d₁ d₂ : ℕ} (p : Piece d₁) (p' : Piece d₂) :
    Piece (d₁ + d₂) where
  W := oplusConstraints d₁ d₂ p.W p'.W
  lo := Fin.append p.lo p'.lo
  hi := Fin.append p.hi p'.hi
  gamma := Fin.append p.gamma p'.gamma
  A := p.A + p'.A
  one_le_dim := by have := p.one_le_dim; omega
  one_le_gamma := by
    rw [Fin.forall_fin_add]
    simpa using And.intro p.one_le_gamma p'.one_le_gamma
  one_le_A := by have := p.one_le_A; omega
  poly_subset_box := by
    intro x hx
    rw [polyPZ_oplusConstraints] at hx
    exact (mem_boxZ_append _ _ _ _ _ _ _).2
      ⟨p.poly_subset_box hx.1, p'.poly_subset_box hx.2⟩
  dim_le_A := add_le_add p.dim_le_A p'.dim_le_A
  rho_le_A := (constraintRho_oplusConstraints_le _ _ _ _).trans
    (add_le_add p.rho_le_A p'.rho_le_A)
  gamma_le_A := by
    rw [Fin.forall_fin_add]
    constructor
    · intro j
      simpa using (p.gamma_le_A j).trans (Nat.le_add_right _ _)
    · intro j
      simpa using (p'.gamma_le_A j).trans (Nat.le_add_left _ _)
```

Library terms: G19 `Fin.append`.

### D079 `sigmaPieceIndexEquiv`

Statements.lean:3369

`Core.lean:9793` · Used by: Lemma 24.

```
def sigmaPieceIndexEquiv {M : ℕ} (N : Fin M → ℕ) :
    ((m : Fin M) × Fin (N m)) ≃ Fin (∑ m, N m) :=
  finSigmaFinEquiv
```

Library terms: G20 `finSumFinEquiv`; G21 `Equiv`.

Definitions used: boxZ (D010), constraintRho (D057), polyPZ (D007), Piece (D061), pieceAffineImage (D022), Piece.quad (D062), IsTransition (D072), IsDecomposition (D073), sigmaPieceIndexEquiv (D079). Library terms: G21 Equiv; G63 Matrix.mulVec.

LEAN STATEMENT lemma24 lines 3388–3406

```
theorem lemma24 :
  ∀ {d M : ℕ} {p : Piece d} {q : Fin M → Piece d}
    {u : Fin M → Fin d → ℤ} {Φ : Fin M → Matrix (Fin d) (Fin d) ℤ}
    (h : IsDecomposition p q u Φ)
    (N : Fin M → ℕ) (q' : (m : Fin M) → Fin (N m) → Piece d)
    (v : (m : Fin M) → Fin (N m) → Fin d → ℤ)
    (Ψ : (m : Fin M) → Fin (N m) → Matrix (Fin d) (Fin d) ℤ)
    (h' : ∀ m, IsDecomposition (q m) (q' m) (v m) (Ψ m)),
IsDecomposition (M := ∑ m, N m) p
      (fun t =>
        q' ((sigmaPieceIndexEquiv N).symm t).1 ((sigmaPieceIndexEquiv N).symm t).2)
      (fun t =>
        u ((sigmaPieceIndexEquiv N).symm t).1 +
          Φ ((sigmaPieceIndexEquiv N).symm t).1 *ᵥ
            v ((sigmaPieceIndexEquiv N).symm t).1 ((sigmaPieceIndexEquiv N).symm t).2)
      (fun t =>
        Φ ((sigmaPieceIndexEquiv N).symm t).1 *
          Ψ ((sigmaPieceIndexEquiv N).symm t).1 ((sigmaPieceIndexEquiv N).symm t).2) :=
  @_root_.IsDecomposition.comp
```

Definitions used: boxZ (D010), constraintRho (D057), polyPZ (D007), Piece (D061), Piece.quad (D062), pieceAffineImage (D022), IsTransition (D072), matInftyNorm (D023), transitionSize (D024). Library terms: G63 Matrix.mulVec.

LEAN STATEMENT lemma24TransitionComposition lines 3414–3420

```
theorem lemma24TransitionComposition :
  ∀ {d : ℕ} {p p' p'' : Piece d} {u v : Fin d → ℤ}
    {Φ Ψ : Matrix (Fin d) (Fin d) ℤ}
    (h : IsTransition p p' u Φ) (h' : IsTransition p' p'' v Ψ),
IsTransition p p'' (u + Φ *ᵥ v) (Φ * Ψ) ∧
      transitionSize (Φ * Ψ) ≤ d * transitionSize Φ * transitionSize Ψ :=
  @_root_.IsTransition.comp
```

Definitions used: matInftyNorm (D023), transitionSize (D024).

LEAN STATEMENT lemma24CompositionSizeBound lines 3425–3428

```
theorem lemma24CompositionSizeBound :
  ∀ {d : ℕ} (Φ Ψ : Matrix (Fin d) (Fin d) ℤ),
transitionSize (Φ * Ψ) ≤ d * transitionSize Φ * transitionSize Ψ :=
  @_root_.transitionSize_mul_le
```

Definitions used: boxZ (D010), constraintRho (D057), polyPZ (D007), Piece (D061), Piece.quad (D062), pieceAffineImage (D022), IsTransition (D072), blockDiagMat (D077), padLeftVec (D074), padRightVec (D075), oplusConstraints (D076), Piece.prod (D078). Library terms: G19 Fin.append.

LEAN STATEMENT lemma24TransitionProduct lines 3436–3444

```
theorem lemma24TransitionProduct :
  ∀ {d₁ d₂ : ℕ}
    {p₁ p₁' : Piece d₁} {p₂ p₂' : Piece d₂}
    {u₁ : Fin d₁ → ℤ} {u₂ : Fin d₂ → ℤ}
    {Φ₁ : Matrix (Fin d₁) (Fin d₁) ℤ} {Φ₂ : Matrix (Fin d₂) (Fin d₂) ℤ}
    (h₁ : IsTransition p₁ p₁' u₁ Φ₁) (h₂ : IsTransition p₂ p₂' u₂ Φ₂),
IsTransition (p₁.prod p₂) (p₁'.prod p₂') (Fin.append u₁ u₂)
      (blockDiagMat Φ₁ Φ₂) :=
  @_root_.IsTransition.prod
```

Definitions used: boxZ (D010), constraintRho (D057), polyPZ (D007), Piece (D061), pieceAffineImage (D022), Piece.quad (D062), IsTransition (D072), IsDecomposition (D073), blockDiagMat (D077), padLeftVec (D074), padRightVec (D075), oplusConstraints (D076), Piece.prod (D078). Library terms: G19 Fin.append; G20 finSumFinEquiv; G21 Equiv.

LEAN STATEMENT lemma24DecompositionProduct lines 3453–3468

```
theorem lemma24DecompositionProduct :
  ∀ {d₁ d₂ M₁ M₂ : ℕ} {p₁ : Piece d₁} {p₂ : Piece d₂}
    {q₁ : Fin M₁ → Piece d₁} {q₂ : Fin M₂ → Piece d₂}
    {u₁ : Fin M₁ → Fin d₁ → ℤ} {u₂ : Fin M₂ → Fin d₂ → ℤ}
    {Φ₁ : Fin M₁ → Matrix (Fin d₁) (Fin d₁) ℤ}
    {Φ₂ : Fin M₂ → Matrix (Fin d₂) (Fin d₂) ℤ}
    (h₁ : IsDecomposition p₁ q₁ u₁ Φ₁) (h₂ : IsDecomposition p₂ q₂ u₂ Φ₂),
IsDecomposition (M := M₁ * M₂) (p₁.prod p₂)
      (fun t =>
        (q₁ (finProdFinEquiv.symm t).1).prod (q₂ (finProdFinEquiv.symm t).2))
      (fun t =>
        Fin.append (u₁ (finProdFinEquiv.symm t).1) (u₂ (finProdFinEquiv.symm t).2))
      (fun t =>
        blockDiagMat (Φ₁ (finProdFinEquiv.symm t).1)
          (Φ₂ (finProdFinEquiv.symm t).2)) :=
  @_root_.IsDecomposition.prod
```

Definitions used: matInftyNorm (D023), transitionSize (D024), blockDiagMat (D077).

LEAN STATEMENT lemma24ProductSizeBound lines 3474–3480

```
theorem lemma24ProductSizeBound :
  ∀ {d₁ d₂ : ℕ}
    (Φ₁ : Matrix (Fin d₁) (Fin d₁) ℤ)
    (Φ₂ : Matrix (Fin d₂) (Fin d₂) ℤ),
transitionSize (blockDiagMat Φ₁ Φ₂) ≤
      max (transitionSize Φ₁) (transitionSize Φ₂) :=
  @_root_.transitionSize_blockDiagMat_le
```

# Lemma 25 `lem:split`

Statements.lean:3483

coverage: decomposition-component

**PAPER STATEMENT** typeset from the verbatim TeX at lines 3489–3503

**Lemma 25** (Splitting along a mixed inequality)**.** *Let $(W, J, Q, A)$ be a piece of index $(d, n_{\text{long}}, n_{\text{mix}})$ with $B$ the set of short coordinates, and let $(w, b) \in W$ be mixed. Write $w = w_B + w_{B^c}$, where $w_B$ is obtained from $w$ by setting its entries outside $B$ to 0, and $w_{B^c} = w - w_B$, and let $\lambda_{w,b} = \min_{x \in J}(w_B \cdot x + b)$. Let $W^+$ be obtained from $W$ by replacing $(w, b)$ with $(w_{B^c}, \lambda_{w,b})$ and adjoining the $2d$ inequalities $x_j \geq \min J_j$ and $x_j \leq \max J_j$, and let $W^- = W \cup \{(-w_{B^c}, -\lambda_{w,b} - 1)\}$. Then $(W^+, J, Q, A + 2d)$ and $(W^-, J, Q, A + 1)$ are pieces with the following properties:*

*(1) The identity map gives transitions from $(W, J, Q, A)$ to $(W^+, J, Q, A + 2d)$ and from $(W, J, Q, A)$ to $(W^-, J, Q, A + 1)$, forming a decomposition of $(W, J, Q, A)$;*

*(2) The piece $(W^+, J, Q, A + 2d)$ has index strictly smaller than $(d, n_{\text{long}}, n_{\text{mix}})$;*

*(3) We have $|w_{B^c} \cdot (x - x')| < A^2 H$ for $x, x' \in \mathcal{P}(W^-)$.*

Coverage: `decomposition-component`.

PAPER CONTEXT `split`

Statements.lean:3510

## The two split pieces and the minimum defining lambda

`splitLambda` (via `boxMinAffine`) is total; under the nonempty-interval hypothesis `hJ` of all Lemma 25 statements it is the paper's minimum. "($W^+$,J,Q,A+2d) and ($W^-$,J,Q,A+1) are pieces" is `lemma25PlusIsPiece`, `lemma25MinusIsPiece` (a `Piece` with these data fields exists); `lemma25` and `lemma25PlusIndexDecreases` take such pieces with the field equations as hypotheses; `lemma25MinusSlabWidth` uses P(`p.splitMinusW w b`).

Lean definitions for this comparison: `restrictVec`, `coordVec`, `boxConstraints`, `Piece.shortPart`, `Piece.longPart`, `Piece.splitLambda`, `Piece.splitPlusW`, `Piece.splitMinusW`. Statements depending on these definitions: Lemma 25.

### D080 `restrictVec`

Statements.lean:3524

`Core.lean:10204` · Used by: Lemma 25 (indirectly).

```
3529  def restrictVec (d : ℕ) (S : Finset (Fin d)) (w : Fin d → ℤ) : Fin d → ℤ :=
3530    fun j => if j ∈ S then w j else 0
```

### D081 `coordVec`

Statements.lean:3535

`Core.lean:10239` · Used by: Lemma 25 (indirectly).

```
3540  def coordVec (d : ℕ) (j : Fin d) : Fin d → ℤ := fun s => if s = j then 1 else 0
```

### D082 boxConstraints

Statements.lean:3545

Core.lean:10241 · Used by: Lemma 25 (indirectly).

```
3550 noncomputable def boxConstraints (d : ℕ) (lo hi : Fin d → ℤ) :
3551     Finset ((Fin d → ℤ) × ℤ) := by
3552   classical
3553   exact (Finset.univ.image fun j : Fin d => (coordVec d j, -lo j)) ∪
3554     (Finset.univ.image fun j : Fin d => (-coordVec d j, hi j))
```

Library terms: G27 `Finset.image`; G32 `insert`.

### D083 Piece.shortPart

Statements.lean:3561

Core.lean:10335 · Used by: Lemma 25 (indirectly).

```
3566 noncomputable def Piece.shortPart {d : ℕ} (p : Piece d) (w : Fin d → ℤ) : Fin d → ℤ :=
3567   restrictVec d p.shortCoords w
```

### D084 Piece.longPart

Statements.lean:3572

Core.lean:10338 · Used by: Lemma 25.

```
3577 noncomputable def Piece.longPart {d : ℕ} (p : Piece d) (w : Fin d → ℤ) : Fin d → ℤ :=
3578   w - p.shortPart w
```

### D085 boxMinAffine

Statements.lean:3583

Core.lean:10298 · Used by: Lemma 25 (indirectly).

Implementation helper, no paper definition.

```
3590 noncomputable def boxMinAffine (d : ℕ) (lo hi : Fin d → ℤ) (w : Fin d → ℤ) (b : ℤ) : ℤ :=
3591   b + ∑ j, min (w j * lo j) (w j * hi j)
```

### D086 Piece.splitLambda

Statements.lean:3596

Core.lean:10345 · Used by: Lemma 25 (indirectly).

```
3601 noncomputable def Piece.splitLambda {d : ℕ} (p : Piece d) (w : Fin d → ℤ) (b : ℤ) : ℤ :=
3602   boxMinAffine d p.lo p.hi (p.shortPart w) b
```

### D087 `Piece.splitPlusW`

Statements.lean:3607

`Core.lean:10356` · Used by: Lemma 25.

```
noncomputable def Piece.splitPlusW {d : ℕ} (p : Piece d) (w : Fin d → ℤ) (b : ℤ) :
    Finset ((Fin d → ℤ) × ℤ) := by
  classical
  exact insert (p.longPart w, p.splitLambda w b) (p.W.erase (w, b)) ∪
    boxConstraints d p.lo p.hi
```

Library terms: G32 `insert`.

### D088 `Piece.splitMinusW`

Statements.lean:3623

`Core.lean:10362` · Used by: Lemma 25.

```
noncomputable def Piece.splitMinusW {d : ℕ} (p : Piece d) (w : Fin d → ℤ) (b : ℤ) :
    Finset ((Fin d → ℤ) × ℤ) := by
  classical
  exact insert (-p.longPart w, -p.splitLambda w b - 1) p.W
```

Library terms: G32 `insert`.

Definitions used: `polyPZ` (D007), `boxZ` (D010), `constraintRho` (D057), `Piece` (D061), `vecSupport` (D009), `boxSide` (D011), `Piece.side` (D063), `Piece.minSide` (D064), `Piece.shortCoords` (D065), `Piece.longCoords` (D066), `Piece.IsMixed` (D067), `Piece.quad` (D062), `pieceAffineImage` (D022), `IsTransition` (D072), `IsDecomposition` (D073), `coordVec` (D081), `boxConstraints` (D082), `restrictVec` (D080), `Piece.shortPart` (D083), `Piece.longPart` (D084), `boxMinAffine` (D085), `Piece.splitLambda` (D086), `Piece.splitPlusW` (D087), `Piece.splitMinusW` (D088).
Library terms: G23 `![a, b, …]`; G32 `insert`; G39 `Disjoint`.

**LEAN STATEMENT** `lemma25` lines 3648–3662

```
theorem lemma25 :
  ∀ {d : ℕ} (p pPlus pMinus : Piece d)
    (hJ : ∀ j, p.lo j ≤ p.hi j) (w : Fin d → ℤ) (b : ℤ)
    (hmem : (w, b) ∈ p.W) (hmix : p.IsMixed (w, b))
    (hPlusW : pPlus.W = p.splitPlusW w b) (hPluslo : pPlus.lo = p.lo)
    (hPlushi : pPlus.hi = p.hi) (hPlusgamma : pPlus.gamma = p.gamma)
    (hPlusA : pPlus.A = p.A + 2 * d)
    (hMinusW : pMinus.W = p.splitMinusW w b) (hMinuslo : pMinus.lo = p.lo)
    (hMinushi : pMinus.hi = p.hi) (hMinusgamma : pMinus.gamma = p.gamma)
    (hMinusA : pMinus.A = p.A + 1),
IsTransition p pPlus 0 1 ∧ IsTransition p pMinus 0 1 ∧
      Disjoint (polyPZ d pPlus.W) (polyPZ d pMinus.W) ∧
      polyPZ d p.W = polyPZ d pPlus.W ∪ polyPZ d pMinus.W ∧
      IsDecomposition (M := 2) p ![pPlus, pMinus] (fun _ => 0) (fun _ => 1) :=
  @_root_.Piece.split_identity_decomposition
```

Definitions used: boxZ (D010), constraintRho (D057), polyPZ (D007), Piece (D061), vecSupport (D009), boxSide (D011), Piece.side (D063), Piece.minSide (D064), Piece.shortCoords (D065), Piece.longCoords (D066), Piece.IsMixed (D067), coordVec (D081), boxConstraints (D082), restrictVec (D080), Piece.shortPart (D083), Piece.longPart (D084), boxMinAffine (D085), Piece.splitLambda (D086), Piece.splitPlusW (D087).

**LEAN STATEMENT** lemma25PlusIsPiece lines 3672–3677

```
3672 theorem lemma25PlusIsPiece :
3673   ∀ {d : ℕ} (p : Piece d) (hJ : ∀ j, p.lo j ≤ p.hi j)
3674     (w : Fin d → ℤ) (b : ℤ) (hmem : (w, b) ∈ p.W) (hmix : p.IsMixed (w, b)),
3675 ∃ p' : Piece d, p'.W = p.splitPlusW w b ∧ p'.lo = p.lo ∧ p'.hi = p.hi ∧
3676       p'.gamma = p.gamma ∧ p'.A = p.A + 2 * d :=
3677   @_root_.Piece.splitPlus_isPiece
```

Definitions used: boxZ (D010), constraintRho (D057), polyPZ (D007), Piece (D061), vecSupport (D009), boxSide (D011), Piece.side (D063), Piece.minSide (D064), Piece.shortCoords (D065), Piece.longCoords (D066), Piece.IsMixed (D067), restrictVec (D080), Piece.shortPart (D083), Piece.longPart (D084), boxMinAffine (D085), Piece.splitLambda (D086), Piece.splitMinusW (D088).

**LEAN STATEMENT** lemma25MinusIsPiece lines 3687–3692

```
3687 theorem lemma25MinusIsPiece :
3688   ∀ {d : ℕ} (p : Piece d) (hJ : ∀ j, p.lo j ≤ p.hi j)
3689     (w : Fin d → ℤ) (b : ℤ) (hmem : (w, b) ∈ p.W) (hmix : p.IsMixed (w, b)),
3690 ∃ p' : Piece d, p'.W = p.splitMinusW w b ∧ p'.lo = p.lo ∧ p'.hi = p.hi ∧
3691       p'.gamma = p.gamma ∧ p'.A = p.A + 1 :=
3692   @_root_.Piece.splitMinus_isPiece
```

Definitions used: boxZ (D010), constraintRho (D057), polyPZ (D007), Piece (D061), vecSupport (D009), boxSide (D011), Piece.side (D063), Piece.minSide (D064), Piece.shortCoords (D065), Piece.longCoords (D066), Piece.IsMixed (D067), Piece.mixedIneqs (D068), Piece.nLong (D069), Piece.nMix (D070), Piece.index (D071), coordVec (D081), boxConstraints (D082), restrictVec (D080), Piece.shortPart (D083), Piece.longPart (D084), boxMinAffine (D085), Piece.splitLambda (D086), Piece.splitPlusW (D087).

**LEAN STATEMENT** lemma25PlusIndexDecreases lines 3703–3710

```
3703 theorem lemma25PlusIndexDecreases :
3704   ∀ {d : ℕ} (p pPlus : Piece d)
3705     (hJ : ∀ j, p.lo j ≤ p.hi j) (w : Fin d → ℤ) (b : ℤ)
3706     (hmem : (w, b) ∈ p.W) (hmix : p.IsMixed (w, b))
3707     (hPlusW : pPlus.W = p.splitPlusW w b) (hlo : pPlus.lo = p.lo) (hhi : pPlus.hi = p.hi)
3708     (hgamma : pPlus.gamma = p.gamma) (hA : pPlus.A = p.A + 2 * d),
3709 pPlus.index < p.index :=
3710   @_root_.Piece.splitPlus_index_lt
```

Definitions used: polyPZ (D007), boxZ (D010), constraintRho (D057), Piece (D061), boxSide (D011), Piece.side (D063), Piece.minSide (D064), vecSupport (D009), Piece.shortCoords (D065), Piece.longCoords (D066), Piece.IsMixed (D067), restrictVec (D080), Piece.shortPart (D083), Piece.longPart (D084), boxMinAffine (D085), Piece.splitLambda (D086), Piece.splitMinusW (D088).

**LEAN STATEMENT** lemma25MinusSlabWidth lines 3720–3727

```
3720 theorem lemma25MinusSlabWidth :
3721   ∀ {d : ℕ} (p : Piece d)
3722     (hJ : ∀ j, p.lo j ≤ p.hi j) (w : Fin d → ℤ) (b : ℤ)
3723     (hmem : (w, b) ∈ p.W) (hmix : p.IsMixed (w, b))
3724     (x x' : Fin d → ℤ) (hx : x ∈ polyPZ d (p.splitMinusW w b))
3725     (hx' : x' ∈ polyPZ d (p.splitMinusW w b)),
3726 |∑ j, p.longPart w j * (x j - x' j)| < (p.A : ℤ) ^ 2 * (p.minSide : ℤ) :=
3727   @_root_.Piece.splitMinus_longPart_variation_lt
```

# Lemma 26 `lem:short-coordinate`

Statements.lean:3730

coverage: direct-strengthened

PAPER STATEMENT typeset from the verbatim TeX at lines 3736–3745

**Lemma 26** (A short linear combination as a coordinate)**.** *Let $(W, J, Q, R)$ be a piece of dimension $d$, let $H$ be an integer with $1 \le H \le \min_j \#J_j$, and let $a \in \mathbb{Z}^d \setminus \{0\}$ satisfy $|a|_1 \le R$ and $|a \cdot (x - x')| \le RH$ for $x, x' \in \mathcal{P}(W)$. Choose $q \in \operatorname{supp} a$ with largest $\#J_q$. Then there exist a set $U \subset \mathbb{Z}^d$ with $\#U \ll_R 1$ and a matrix $D \in \mathbb{Z}^{d\times d}$ such that the transitions $x = u + Dy$, $u \in U$, give a decomposition of $(W, J, Q, R)$ into pieces $(W'_u, J'_u, Q', R')$ with $R' \ll_R 1$ and*

$$\#J'_{u,q} = H, \qquad \#J_j \le \#J'_{u,j} \le 3\#J_j \ (j \ne q) \quad \text{and} \quad J'_{u,j} = J_j \ (j \notin \operatorname{supp} a).$$

Coverage: `direct-strengthened`.

PAPER CONTEXT `common_bound` Statements.lean:3752

## The common new complexity parameter

The outputs' common Q′ and R′ are encoded by equal `(q m).gamma` and `(q m).A = R'`. C is chosen before d, p, H, a, qc, so depends only on R. Stronger than the paper: the input needs only `p.A ≤ R` (paper: A = R); the conclusion holds for every qc ∈ supp a of maximal side (paper: "choose q"); `transitionSize D ≤ C` is added (the paper's proof gives |D|∞ ≪_R 1). The paper's translations form a *set* U ⊂ ℤ^d, one piece per u ∈ U; that the `Fin M`-indexed family `U : Fin M → Fin d → ℤ` enumerates such a set without repetition is the conjunct `Function.Injective U`, so no translation is used twice and M is the size of the paper's U. Form only: `∀ j, p.lo j ≤ p.hi j` follows from 1 ≤ H ≤ `p.minSide`; if P(W) = ∅, M = 0 satisfies the conclusion, as for the paper's decompositions.

Definitions used: vecSupport (D009), polyPZ (D007), boxZ (D010), constraintRho (D057), Piece (D061), boxSide (D011), Piece.side (D063), Piece.minSide (D064), matInftyNorm (D023), transitionSize (D024), pieceAffineImage (D022), Piece.quad (D062), IsTransition (D072), IsDecomposition (D073).

LEAN STATEMENT lemma26 lines 3774–3798

```
theorem lemma26 :
  ∀ (R : ℕ) (hR : 1 ≤ R),
∃ C : ℕ, 1 ≤ C ∧
      ∀ (d : ℕ) (p : Piece d), p.A ≤ R → (∀ j, p.lo j ≤ p.hi j) →
        ∀ H : ℕ, 1 ≤ H → H ≤ p.minSide →
          ∀ a : Fin d → ℤ, a ≠ 0 → (∑ j, |a j|) ≤ (R : ℤ) →
            (∀ x ∈ polyPZ d p.W, ∀ x' ∈ polyPZ d p.W,
              |∑ j, a j * (x j - x' j)| ≤ (R : ℤ) * (H : ℤ)) →
            ∀ qc : Fin d, qc ∈ vecSupport a →
              (∀ j ∈ vecSupport a, p.side j ≤ p.side qc) →
              ∃ (M : ℕ) (U : Fin M → Fin d → ℤ) (D : Matrix (Fin d) (Fin d) ℤ)
                (q : Fin M → Piece d) (R' : ℕ),
                M ≤ C ∧
                R' ≤ C ∧
                IsDecomposition p q U (fun _ => D) ∧
                Function.Injective U ∧
                transitionSize D ≤ C ∧
                (∀ m, (q m).A = R') ∧
                (∀ m m', (q m).gamma = (q m').gamma) ∧
                (∀ m, (q m).side qc = H) ∧
                (∀ (m : Fin M) (j : Fin d), j ≠ qc →
                  p.side j ≤ (q m).side j ∧ (q m).side j ≤ 3 * p.side j) ∧
                ∀ (m : Fin M) (j : Fin d), j ∉ vecSupport a →
                  (q m).lo j = p.lo j ∧ (q m).hi j = p.hi j :=
  @_root_.lem_short_coordinate
```

# Proposition 27 `prop:pieces`

Statements.lean:3801

coverage: direct

**PAPER STATEMENT** typeset from the verbatim TeX at lines 3807–3812

**Proposition 27.** *For all $d, n_{\text{long}}, n_{\text{mix}}, A$ there is $C = C(d, n_{\text{long}}, n_{\text{mix}}, A)$ such that every piece $(W, J, Q, A)$ of index $(d, n_{\text{long}}, n_{\text{mix}})$ has a decomposition by at most $C$ transitions $x = u_m + \Phi_m y$ of size at most $C$ to pieces $(W_m, J_m, Q_m, A_m)$ with $A_m \le C$, $\mathrm{vol}(J_m) \le C\,\mathrm{vol}(J)$ and $J_m$ $C$-balanced with respect to $W_m$.*

Coverage: `direct`.

Reading note: Lean adds `∀ j, p.lo j ≤ p.hi j` (the paper is silent); harmless, since otherwise J and P(W) are empty and M = 0 works, and the paper's proof of Theorem 21 applies Proposition 27 to the nonempty box I. The conclusion `IsBalancedWrt d C (q m).W …` also requires nonzero normals in each W_m (presupposed by Definition 20); this can be arranged, as a constraint (0,b) is vacuous if b ≥ 0 and empties P(W_m) if b < 0, so it can be deleted or the piece dropped.

Definitions used: `boxSide` (D011), `boxVol` (D012), `constraintGraph` (D058), `IsBalancedWrt` (D059), `boxZ` (D010), `constraintRho` (D057), `polyPZ` (D007), `Piece` (D061), `vecSupport` (D009), `Piece.side` (D063), `Piece.minSide` (D064), `Piece.shortCoords` (D065), `Piece.longCoords` (D066), `Piece.IsMixed` (D067), `Piece.mixedIneqs` (D068), `Piece.nLong` (D069), `Piece.nMix` (D070), `matInftyNorm` (D023), `transitionSize` (D024), `pieceAffineImage` (D022), `Piece.quad` (D062), `IsTransition` (D072), `IsDecomposition` (D073).

**LEAN STATEMENT** `proposition27` lines 3835–3848

```
3835 theorem proposition27 :
3836   ∀ (d nlong nmix A : ℕ),
3837 ∃ C : ℕ, 1 ≤ C ∧
3838       ∀ p : Piece d, p.A = A → p.nLong = nlong → p.nMix = nmix →
3839         (∀ j, p.lo j ≤ p.hi j) →
3840         ∃ (M : ℕ) (q : Fin M → Piece d) (u : Fin M → Fin d → ℤ)
3841           (Φ : Fin M → Matrix (Fin d) (Fin d) ℤ),
3842           M ≤ C ∧
3843           IsDecomposition p q u Φ ∧
3844           (∀ m, transitionSize (Φ m) ≤ C) ∧
3845           (∀ m, (q m).A ≤ C) ∧
3846           (∀ m, boxVol d (q m).lo (q m).hi ≤ C * boxVol d p.lo p.hi) ∧
3847           ∀ m, IsBalancedWrt d C (q m).W (q m).lo (q m).hi :=
3848   @_root_.prop_pieces
```

# Proposition 28 `prop:fold`

Statements.lean:3851

coverage: direct

PAPER CONTEXT `fold` Statements.lean:3855

## The fold's source pieces, target pieces, map, and inverse

The paper omits out-of-bounds coordinate expressions and empty blocks; `foldCoord` is a one-based lookup into `Fin k`, zero outside the range. `foldMapGlobal` picks a piece when one exists and is 0 otherwise; the theorem's hypotheses, not this default, identify it with the paper's map.

ORIGINAL PAPER DEFINITION/CONTEXT article.txt:2125-2180 · typeset from the verbatim TeX at lines 3864–3919

We now construct an explicit $\mathrm{Aff}_k(\mathbb{Z})$-equidecomposition of $\mathcal{S}_{k,N}$ and $\{0,1,\dots,N/k\}\times\mathcal{S}_{k-1,N}$. Set $\Phi_{1,N}=\mathrm{Id}$. For $k\ge 2$, $N\ge 0$ with $k\mid N$, and $1\le m\le k$, define

$$\mathcal{C}_{k,N,m}=\left\{x\in\mathcal{S}_{k,N}:\sum_{\nu=1}^{i}x_\nu>\frac{iN}{k}\ (1\le i<m),\quad \sum_{\nu=1}^{m}x_\nu\le\frac{mN}{k}\right\}. \tag{eq:sourcePieces}$$

These are disjoint $\mathbb{Z}$-polytopes that produce a partition of $\mathcal{S}_{k,N}$. In what follows, coordinate expressions or constraints containing out-of-bounds variables should be omitted in their entirety. Coordinate blocks with empty index ranges should also be omitted. For $x\in\mathcal{C}_{k,N,m}$, define the exact map directly by

$$\Phi_{k,N}(x)=\begin{cases}(x_1,x_1+x_2,x_3,\dots,x_k), & \text{if } m=1,\\ \left(\sum_{\nu=1}^{m}x_\nu-\frac{(m-1)N}{k},x_m,x_{m-1},\dots,x_3,\right.\\ \qquad\left.x_1-\frac{N}{k}-1,x_2+x_{m+1}+\frac{N}{k}+1,x_{m+2},\dots,x_k\right), & \text{if } 2\le m\le k.\end{cases} \tag{eq:fold}$$

It is now clear that $\Phi_{k,N}(\mathcal{S}_{k,N})\subset\{0,1,\dots,N/k\}\times\mathcal{S}_{k-1,N}$ and that $\Phi_{k,N}$ is piecewise affine, with integer matrices and translations. We now describe its inverse. Let

$$\mathcal{D}_{k,N,m}=\left\{y\in\{0,1,\dots,N/k\}\times\mathcal{S}_{k-1,N}:\sum_{\nu=2}^{i+1}y_\nu<y_1+\frac{(i-1)N}{k}\ (1\le i<m),\ \sum_{\nu=2}^{m+1}y_\nu\ge y_1+\frac{(m-1)N}{k}(m<k)\right\}, \tag{eq:targetPieces}$$

for $m=1,\dots,k$. We claim that $\Phi_{k,N}(\mathcal{C}_{k,N,m})=\mathcal{D}_{k,N,m}$ for each $m$ and that

$$\Phi_{k,N}^{-1}(y)=\begin{cases}(y_1,y_2-y_1,y_3,\dots,y_k), & \text{if } m=1,\\ \left(y_m+\frac{N}{k}+1,y_1+\frac{(m-2)N}{k}-1-\sum_{\nu=2}^{m}y_\nu,y_{m-1},\dots,y_2,\right.\\ \qquad\left.\sum_{\nu=2}^{m+1}y_\nu-y_1-\frac{(m-1)N}{k},y_{m+2},\dots,y_k\right), & \text{if } 2\le m\le k.\end{cases} \tag{eq:foldInverse}$$

Lean definitions for this comparison: `foldPrefixSum`, `foldSourcePiece`, `foldCoord`, `foldMap`, `foldMapGlobal`, `foldTargetBase`, `foldTargetPiece`, `foldInv`. Statements depending on these definitions: Proposition 28.

### D089 foldPrefixSum

Statements.lean:3927

Core.lean:14875 · Used by: Proposition 28 (indirectly).

```
3932  def foldPrefixSum (k : ℕ) (x : Fin k → ℕ) (i : ℕ) : ℕ :=
3933    ∑ ν ∈ Finset.univ.filter (fun ν : Fin k => ν.val < i), x ν
```

Library terms: G15 `x.val`; G26 `Finset.filter`.

### D090 foldSourcePiece

Statements.lean:3940

Core.lean:14881 · Used by: Proposition 28.

```
3945  noncomputable def foldSourcePiece (k N m : ℕ) : Finset (Fin k → ℕ) := by
3946    classical
3947    exact (simplexS k N).filter fun x =>
3948      (∀ i ∈ Finset.Ico 1 m, i * N < k * foldPrefixSum k x i) ∧
3949        k * foldPrefixSum k x m ≤ m * N
```

Library terms: G25 `Finset.Icc`; G26 `Finset.filter`.

### D091 foldCoord

Statements.lean:3956

Core.lean:14975 · Used by: Proposition 28 (indirectly).

```
3961  def foldCoord {k : ℕ} (x : Fin k → ℤ) (i : ℕ) : ℤ :=
3962    if h : 1 ≤ i ∧ i ≤ k then x ⟨i - 1, by omega⟩ else 0
```

### D092 foldMap

Statements.lean:3967

Core.lean:14978 · Used by: Proposition 28.

```
3972  def foldMap (k N m : ℕ) (x : Fin k → ℤ) : Fin k → ℤ := fun t =>
3973    let j := t.val + 1
3974    let q : ℤ := ((N / k : ℕ) : ℤ)
3975    if m = 1 then
3976      (if j = 2 then foldCoord x 1 + foldCoord x 2 else foldCoord x j)
3977    else if j = 1 then (∑ ν ∈ Finset.Icc 1 m, foldCoord x ν) - ((m : ℤ) - 1) * q
3978    else if j < m then foldCoord x (m + 2 - j)
3979    else if j = m then foldCoord x 1 - q - 1
3980    else if j = m + 1 then foldCoord x 2 + foldCoord x (m + 1) + q + 1
3981    else foldCoord x j
```

Library terms: G15 `x.val`; G25 `Finset.Icc`.

### D093 foldMapGlobal

Statements.lean:3988

Core.lean:14989 · Used by: Proposition 28.

```
noncomputable def foldMapGlobal (k N : ℕ) (x : Fin k → ℕ) : Fin k → ℤ := by
  classical
  exact
    if h : ∃ m, 1 ≤ m ∧ m ≤ k ∧ x ∈ foldSourcePiece k N m then
      foldMap k N (Nat.find h) (fun t => (x t : ℤ))
    else 0
```

Library terms: G44 Nat.find.

### D094 foldTargetBase

Statements.lean:4005

Core.lean:14996 · Used by: Proposition 28.

```
def foldTargetBase (k N : ℕ) : Set (Fin k → ℤ) :=
  {y | (∀ t, 0 ≤ y t) ∧ foldCoord y 1 ≤ ((N / k : ℕ) : ℤ) ∧
    ∑ j ∈ Finset.Icc 2 k, foldCoord y j ≤ (N : ℤ)}
```

Library terms: G25 Finset.Icc.

### D095 foldTargetPiece

Statements.lean:4019

Core.lean:15000 · Used by: Proposition 28.

```
def foldTargetPiece (k N m : ℕ) : Set (Fin k → ℤ) :=
  {y | y ∈ foldTargetBase k N ∧
    (∀ i ∈ Finset.Ico 1 m,
      ∑ ν ∈ Finset.Icc 2 (i + 1), foldCoord y ν <
        foldCoord y 1 + ((i : ℤ) - 1) * ((N / k : ℕ) : ℤ)) ∧
    (m < k →
      foldCoord y 1 + ((m : ℤ) - 1) * ((N / k : ℕ) : ℤ) ≤
        ∑ ν ∈ Finset.Icc 2 (m + 1), foldCoord y ν)}
```

Library terms: G25 Finset.Icc.

**D096** **foldInv** Statements.lean:4038

`Core.lean:15009` · Used by: Proposition 28.

```
def foldInv (k N m : ℕ) (y : Fin k → ℤ) : Fin k → ℤ := fun t =>
  let j := t.val + 1
  let q : ℤ := ((N / k : ℕ) : ℤ)
  if m = 1 then
    (if j = 2 then foldCoord y 2 - foldCoord y 1 else foldCoord y j)
  else if j = 1 then foldCoord y m + q + 1
  else if j = 2 then
    foldCoord y 1 + ((m : ℤ) - 2) * q - 1 - ∑ ν ∈ Finset.Icc 2 m, foldCoord y ν
  else if j ≤ m then foldCoord y (m + 2 - j)
  else if j = m + 1 then
    (∑ ν ∈ Finset.Icc 2 (m + 1), foldCoord y ν) - foldCoord y 1 - ((m : ℤ) - 1) * q
  else foldCoord y j
```

Library terms: G15 `x.val`; G25 `Finset.Icc`.

PAPER STATEMENT typeset from the verbatim TeX at lines 4063–4075

**Proposition 28** (The fold)**.** *Let $k \geq 2$ and let $N \geq 1$ be divisible by $k$. The map*

$$\Phi_{k,N} : \mathcal{S}_{k,N} \longrightarrow \{0, 1, \ldots, N/k\} \times \mathcal{S}_{k-1,N}$$

*induces an $Aff_k(\mathbb{Z})$-equidecomposition of $\mathcal{S}_{k,N}$ and $\{0, 1, \ldots, N/k\} \times \mathcal{S}_{k-1,N}$ given by* (eq:sourcePieces) *and* (eq:targetPieces) *(respectively) of complexity bounded by $2k + 1$. Its inverse is given by* (eq:foldInverse)*.*

Coverage: `direct`.

`eq:sourcePieces`, `eq:targetPieces`, `eq:foldInverse`: see the `fold` context.

Reading note: the partition of S_{k,N} into C_{k,N,1}, …, C_{k,N,k} is not a conjunct but follows from the first one: with s_i = x_1+⋯+x_i, x ∈ C_m needs s_m ≤ mN/k and x ∈ C_{m′} (m′ > m) needs s_m > mN/k, so the pieces are disjoint, and s_k ≤ N = kN/k puts x in C_m for the least m with s_m ≤ mN/k. The complexity bound is stated piece by piece (last conjunct), not via the Definition 3 predicate: k pieces, each source and target piece P(L) with #L ≤ 2k+1 and normals of $\ell^1$-norm ≤ k, each `foldMap k N m` affine with a 0/1 matrix of determinant ±1.

PAPER CONTEXT `fold_recursion` Statements.lean:4093

## The recursive simplex-to-box map

Core.lean implements U_{k,N} as `foldUN` (Core.lean:16464, totalized) with the auxiliary `foldStage` (Core.lean:16472); neither occurs in a `Compatibility.lean` statement, so neither is shown. The proof of Proposition 4 (`prop_UNmap_explicit`, Core.lean:17366) takes as U N the map x ↦ `foldUN k N` of the coordinatewise `toNat` of x, cast to ℤ (Core.lean:17060); `proposition4` only asserts that some U exists, and the Theorem 5 and Proposition 10 endpoints do not name `foldUN` (see their notes). The recursion is not part of Proposition 28.

ORIGINAL PAPER DEFINITION/CONTEXT article.txt:2031-2033 · typeset from the verbatim TeX at lines 4106–4108

$$U_{k,N}(x) = (l_1, U_{k-1,N}(l_2, \ldots, l_k)) \quad \text{if} \quad \Phi_{k,N}(x) = (l_1, l_2, \ldots, l_k). \tag{def:UN}$$

Lean definitions for this comparison: none displayed (see above). Statements depending on these definitions: none of the displayed theorem statements.

Definitions used: simplexS (D027), polyPZ (D007), foldPrefixSum (D089), foldSourcePiece (D090), foldCoord (D091), foldMap (D092), foldMapGlobal (D093), foldTargetBase (D094), foldTargetPiece (D095), foldInv (D096). Library terms: G15 x.val; G24 Finset.range; G25 Finset.Icc; G26 Finset.filter; G28 Finset.card; G37 f '' S; G40 Set.BijOn; G41 ∃!; G62 Matrix.det; G63 Matrix.mulVec.

LEAN STATEMENT proposition28 lines 4123–4182

```
theorem proposition28 :
  ∀ (k N : ℕ) (hk : 2 ≤ k) (hN : 1 ≤ N) (hkN : k ∣ N),
(∀ m, 1 ≤ m → m ≤ k → ∀ x : Fin k → ℕ,
      x ∈ foldSourcePiece k N m ↔
        (∑ s, x s ≤ N) ∧
        (∀ i ∈ Finset.Ico 1 m,
          i * (N / k) < ∑ ν ∈ Finset.univ.filter (fun ν : Fin k => ν.val < i), x ν) ∧
        ∑ ν ∈ Finset.univ.filter (fun ν : Fin k => ν.val < m), x ν ≤ m * (N / k)) ∧
    (∀ y : Fin k → ℤ, y ∈ foldTargetBase k N ↔
      (∀ t, 0 ≤ y t) ∧ y ⟨0, by omega⟩ ≤ ((N / k : ℕ) : ℤ) ∧
        ∑ t ∈ Finset.univ.filter (fun t : Fin k => 0 < t.val), y t ≤ (N : ℤ)) ∧
    (∀ m, 1 ≤ m → m ≤ k → ∀ y : Fin k → ℤ,
      let g : ℕ → ℤ := fun i => if h : i < k then y ⟨i, h⟩ else 0
      let q : ℤ := ((N / k : ℕ) : ℤ)
      (y ∈ foldTargetPiece k N m ↔
        y ∈ foldTargetBase k N ∧
        (∀ i ∈ Finset.Ico 1 m,
          ∑ j ∈ Finset.Ico 1 (i + 1), g j < g 0 + ((i : ℤ) - 1) * q) ∧
        (m < k → g 0 + ((m : ℤ) - 1) * q ≤ ∑ j ∈ Finset.Ico 1 (m + 1), g j)) ∧
      ∀ t : Fin k,
        foldMap k N m y t =
          (if m = 1 then (if t.val = 1 then g 0 + g 1 else y t)
          else if t.val = 0 then (∑ i ∈ Finset.range m, g i) - ((m : ℤ) - 1) * q
          else if t.val < m - 1 then g (m - t.val)
          else if t.val = m - 1 then g 0 - q - 1
          else if t.val = m then g 1 + g m + q + 1
          else y t) ∧
        foldInv k N m y t =
          (if m = 1 then (if t.val = 1 then g 1 - g 0 else y t)
          else if t.val = 0 then g (m - 1) + q + 1
          else if t.val = 1 then
            g 0 + ((m : ℤ) - 2) * q - 1 - ∑ i ∈ Finset.Ico 1 m, g i
          else if t.val < m then g (m - t.val)
          else if t.val = m then
            (∑ i ∈ Finset.Ico 1 (m + 1), g i) - g 0 - ((m : ℤ) - 1) * q
          else y t)) ∧
    (∀ m, 1 ≤ m → m ≤ k →
      foldMap k N m '' ((fun x : Fin k → ℕ => fun t => (x t : ℤ)) ''
          (foldSourcePiece k N m : Set (Fin k → ℕ))) = foldTargetPiece k N m) ∧
    (∀ m, 1 ≤ m → m ≤ k → foldTargetPiece k N m ⊆ foldTargetBase k N) ∧
    (∀ y ∈ foldTargetBase k N, ∃! m, 1 ≤ m ∧ m ≤ k ∧ y ∈ foldTargetPiece k N m) ∧
    (∀ m, 1 ≤ m → m ≤ k → ∀ y ∈ foldTargetPiece k N m,
      foldInv k N m y ∈ (fun x : Fin k → ℕ => fun t => (x t : ℤ)) ''
          (foldSourcePiece k N m : Set (Fin k → ℕ)) ∧
        foldMap k N m (foldInv k N m y) = y) ∧
    (∀ m, 1 ≤ m → m ≤ k → ∀ x ∈ foldSourcePiece k N m,
      foldInv k N m (foldMap k N m (fun t => (x t : ℤ))) = fun t => (x t : ℤ)) ∧
    Set.BijOn (foldMapGlobal k N) (simplexS k N : Set (Fin k → ℕ)) (foldTargetBase k N) ∧
    (∀ m, 1 ≤ m → m ≤ k →
      (∃ L : Finset ((Fin k → ℤ) × ℤ),
```

```
      (fun x : Fin k → ℕ => fun t => (x t : ℤ)) ''
          (foldSourcePiece k N m : Set (Fin k → ℕ)) = polyPZ k L ∧
      L.card ≤ 2 * k + 1 ∧ ∀ p ∈ L, ∑ s, |p.1 s| ≤ (k : ℤ)) ∧
    (∃ L : Finset ((Fin k → ℤ) × ℤ),
      foldTargetPiece k N m = polyPZ k L ∧
      L.card ≤ 2 * k + 1 ∧ ∀ p ∈ L, ∑ s, |p.1 s| ≤ (k : ℤ)) ∧
    ∃ (A : Matrix (Fin k) (Fin k) ℤ) (c : Fin k → ℤ),
      (A.det = 1 ∨ A.det = -1) ∧ (∀ i j, A i j = 0 ∨ A i j = 1) ∧
      ∀ x : Fin k → ℤ, foldMap k N m x = A.mulVec x + c) :=
@_root_.prop_fold
```

# Library glossary (Mathlib and Lean core)

Statements.lean:4185

Entries G15–G78: full entries for terms in theorem statements, one line for the others. Each definition and statement above lists the entries it uses.

### G15 — `x.val` (`Fin.val`, `Subtype.val`) and the coercions `(t : ℕ)`, `(n : Fin K → ℕ)`

Statements.lean:4190

Meaning: For t : Fin n, `t.val` (also `t.1`, `↑t`, `(t : ℕ)`) is the number t ∈ {0, …, n−1}. For n : ↑s = {x // x ∈ s}, `n.val` (also `n.1`, `↑n`, or an ascription such as `(n : Fin K → ℕ)`) is the underlying element and `n.property` (`n.2`) the proof that it lies in s. Both coercions are these projections, and both are injective.

Pinned by: structure fields `Fin.val`, `Subtype.val`; coercions `Fin.coeToNat` and `subtypeCoe`, both v ↦ v.val; `Subtype.ext`: val a1 = val a2 → a1 = a2.

Conventions: `t.val` is 0-based (paper index t.val + 1). Arithmetic on it is in ℕ, so `k - s.val` is truncated subtraction, though never truncated for s : Fin k (s.val ≤ k−1).

### G16 — `Fin.last`

Statements.lean:4205

Meaning: `Fin.last n : Fin (n+1)` is ⟨n, _⟩, the last of the indices 0, …, n (the argument is one less than the size). For x : Fin (k+1) → R, `x (Fin.last k)` is the last coordinate (paper coordinate k+1).

Pinned by: `def last (n : Nat) : Fin (n + 1) := ⟨n, n.lt_succ_self⟩`; `Fin.val_last`: (last n).1 = n.

### G18 — `Fin.init` and `Fin.snoc`

Statements.lean:4214

Meaning: For an (n+1)-tuple q, `Fin.init q` = (q 0, …, q (n−1)) drops the last entry; `Fin.snoc p x` = (p 0, …, p (n−1), x) appends x at `Fin.last n`. They are mutually inverse.

Pinned by: `Fin.init_def`: init q = fun k : Fin n ↦ q k.castSucc; `Fin.snoc_castSucc`: snoc p x i.castSucc = p i; `Fin.snoc_last`: snoc p x (last n) = x; `Fin.init_snoc`: init (snoc p x) = p; `Fin.snoc_init_self`: snoc (init q) (q (last n)) = q.

Conventions: init and snoc act at the END (index n); `Fin.tail` and `Fin.cons` at the FRONT (index 0).

### G19 — `Fin.append`, `Fin.castAdd`, `Fin.natAdd`

Statements.lean:4228

Meaning: `Fin.castAdd m i` has value i (first block); `Fin.natAdd n i` has value n + i (second block). `Fin.append a b` = (a 0, …, a (m−1), b 0, …, b (n−1)): the LEFT argument occupies the first coordinates.

Pinned by: `Fin.val_castAdd`: (castAdd m i : Nat) = i; `Fin.val_natAdd`: (natAdd n i : Nat) = n + i; `Fin.append_left`: append u v (castAdd n i) = u i; `Fin.append_right`: append u v (natAdd m i) = v i.

Conventions: In `Fin.castAdd d₂ j`, $d_2$ is the size of the block AFTER j's block; in `Fin.natAdd d₁ j`, $d_1$ is the size of the block BEFORE it (the shift).

### G20 — `finSumFinEquiv`, `finProdFinEquiv`, `finSigmaFinEquiv` (and `finCongr`)

Statements.lean:4241

Meaning: `finSumFinEquiv`: inl i ↦ i, inr j ↦ m + j. `finProdFinEquiv`: (i, j) ↦ j + n·i (row-major, second component fastest), so its inverse is t ↦ (t / n, t % n). `finSigmaFinEquiv`: ⟨i, j⟩ ↦ n 0 + ⋯ + n (i−1) + j (blocks Fin (n 0), Fin (n 1), … laid out consecutively). `finCongr h` keeps the value and changes only the bound.

Pinned by: `finSumFinEquiv_apply_left`: finSumFinEquiv (Sum.inl i) = Fin.castAdd n i; `finSumFinEquiv_apply_right`: finSumFinEquiv (Sum.inr i) = Fin.natAdd m i; definition of `finProdFinEquiv`: toFun x := ⟨x.2 + n * x.1, _⟩, invFun x := (x.divNat, x.modNat); `finSigmaFinEquiv_apply`: (finSigmaFinEquiv k : ℕ) = ∑ i : Fin k.1, n (Fin.castLE k.1.2.le i) + k.2; `finCongr_apply_coe`: (finCongr h k : ℕ) = k.

Conventions: `Fin.divNat`, `Fin.modNat` are in Batteries (not checked), but the inverse (t / n, t % n) is forced, since symm inverts toFun and j < n. `.1`, `.2` of `finProdFinEquiv.symm t` are the two factors; of `(sigmaPieceIndexEquiv N).symm t` they are the block index m and the position inside block m.

### G21 — `Equiv (α ≃ β)`, `e.symm`, `e.trans`, `Equiv.Perm`

Statements.lean:4261

Meaning: e : α ≃ β is a bijection: a function α → β (applied as `e x`) with a two-sided inverse `e.symm : β ≃ α`. `e.trans f : α ≃ γ` is x ↦ f (e x); `Equiv.Perm α` is α ≃ α. A `def … : α ≃ β where toFun … invFun … left_inv … right_inv …` gives both maps and proves them mutually inverse.

Pinned by: structure `Equiv` with `left_inv : LeftInverse invFun toFun`, `right_inv : RightInverse invFun toFun`; `Equiv.trans_apply`: (f.trans g) a = g (f a); `abbrev Equiv.Perm (α : Sort*) := Equiv α α`.

Conventions: `e.trans f` applies e FIRST (opposite to ∘). An Equiv into or out of a subtype ↥s also carries membership proofs; the map is read off `toFun`.

### G23 — `![a, b, …]` (vector notation)

Statements.lean:4275

Meaning: `![a₀, a₁, …]` is the function on Fin n with 0 ↦ $a_0$, 1 ↦ $a_1$, … (0-based): notation for nested `Matrix.vecCons a₀ (… Matrix.vecEmpty)`, with `Matrix.vecCons h t = Fin.cons h t`. Despite the namespace it is a vector, not a matrix.

Pinned by: `Matrix.cons_val_zero`: vecCons x u 0 = x; `Matrix.cons_val_one`: vecCons x u 1 = u 0 (both `rfl`).

### G24 — `Finset.range`

Statements.lean:4285

Meaning: `Finset.range n` = {m : ℕ | m < n} = {0, …, n−1}, with n elements; `∑ i ∈ Finset.range m, g i` = g 0 + ⋯ + g (m−1).

Pinned by: `Finset.mem_range`: m ∈ range n ↔ m < n; `Finset.card_range`: #(range n) = n.

Conventions: Starts at 0 and excludes n; `Finset.range_zero`: range 0 = ∅.

### G25 — `Finset.Icc`, `Finset.Ico`, `Finset.Ioo`, `Finset.Iic`

Statements.lean:4295

Meaning: Icc a b = {x | a ≤ x ≤ b}, Ico a b = {x | a ≤ x < b}, Ioo a b = {x | a < x < b}, Iic b = {x | x ≤ b}, with x in the type of the bounds (so on ℤ, Icc a b = [a, b] ∩ ℤ). Crossed bounds give the empty set.

Pinned by: `Finset.mem_Icc`: x ∈ Icc a b ↔ a ≤ x ∧ x ≤ b (likewise `Finset.mem_Ico`, `Finset.mem_Ioo`, `Finset.mem_Iic`); `Finset.Icc_eq_empty_iff`: Icc a b = ∅ ↔ ¬a ≤ b; `Finset.Ico_eq_empty_iff`: Ico a b = ∅ ↔ ¬a < b.

Conventions: Finset.Icc 1 k ⊆ ℕ, Finset.Icc (lo s) (hi s) ⊆ ℤ, Finset.Iic j ⊆ Fin d (= {0, …, j}; Fin compares values). Cardinalities: G28; `Fin.card_Iic`: #(Iic b) = b + 1.

### G26 — `Finset.filter` (`s.filter p`)

Statements.lean:4310

Meaning: `s.filter p` (also `Finset.filter p s`) is {x ∈ s | p x}.

Pinned by: `Finset.mem_filter`: a ∈ s.filter p ↔ a ∈ s ∧ p a.

Conventions: The predicate is everything after `fun x =>` up to the enclosing parenthesis, so `B.filter fun x => P x ∧ Q x` filters by P ∧ Q. The decidability instance (from `classical`) does not affect the result.

### G28 — `Finset.card` (`s.card`, `#s`)

Statements.lean:4320

Meaning: `s.card` (`#s`) is the number of distinct elements of s.

Pinned by: definition `card s = Multiset.card s.1`; `Int.card_Icc`: #(Icc a b) = (b + 1 - a).toNat; `Nat.card_Icc`: #(Icc a b) = b + 1 - a; `Fintype.card_piFinset`: #(piFinset s) = ∏ i, #(s i).

Conventions: On ℤ, #(Icc a b) is b − a + 1 if a ≤ b + 1 and 0 otherwise (`Int.toNat` sends negatives to 0); on ℕ the subtraction is truncated.

### G29 — `Finset.sup` (`s.sup f`)

Statements.lean:4331

Meaning: `s.sup f` = f $b_1$ ⊔ ⋯ ⊔ f $b_n$ over the elements of s, folded from ⊥. On ℕ (⊔ = max, ⊥ = 0) it is max_{b ∈ s} f b (attained) for nonempty s, and 0 for s = ∅.

Pinned by: definition `s.sup f = s.fold (· ⊔ ·) ⊥ f`; `Finset.sup_empty`: (∅ : Finset β).sup f = ⊥; `Finset.sup_le_iff`: s.sup f ≤ a ↔ ∀ b ∈ s, f b ≤ a; `Finset.exists_mem_eq_sup` (linear orders): s.Nonempty → ∃ i, i ∈ s ∧ s.sup f = f i; `Nat.bot_eq_zero`: ⊥ = 0.

Conventions: Requires a bottom element, so it is used here only with ℕ values (integers are first converted with natAbs or toNat).

### G31 — `Finset.lcm` (`s.lcm f`)

Statements.lean:4345

Meaning: `s.lcm f` folds `GCDMonoid.lcm` (on ℕ: `Nat.lcm`) over the f b, b ∈ s, starting from 1: the least common multiple.

Pinned by: definition `s.lcm f = s.fold GCDMonoid.lcm 1 f`; `Finset.lcm_dvd_iff`: s.lcm f | a ↔ ∀ b ∈ s, f b | a; `lcm_eq_nat_lcm`: lcm m n = Nat.lcm m n.

Conventions: The empty lcm is 1 (`Finset.lcm_empty`). The lcm is 0 iff some f b = 0 (`Finset.lcm_eq_zero_iff`), which does not happen for (Finset.Icc 1 k).lcm id.

### G32 — `insert`, `Finset.erase`, `∪`, `∩` (and `{a, b}`)

Statements.lean:4358

Meaning: `insert a s` is s with a added (unchanged if a ∈ s); `s.erase a` is s with a removed (unchanged if a ∉ s); ∪, ∩ are union and intersection, of finsets or of sets (as in Lemma 25's `polyPZ d p.W = polyPZ d pPlus.W ∪ polyPZ d pMinus.W`); `{a, b}` is insert a {b}.

Pinned by: `Finset.mem_insert`: a ∈ insert b s ↔ a = b ∨ a ∈ s; `Finset.mem_erase`: a ∈ erase s b ↔ a ≠ b ∧ a ∈ s; `Finset.mem_union`: a ∈ s ∪ t ↔ a ∈ s ∨ a ∈ t (`Set.mem_union` for sets); `Finset.mem_inter`: a ∈ $s_1$ ∩ $s_2$ ↔ a ∈ $s_1$ ∧ a ∈ $s_2$; `{x, xs,*}` expands to `insert x {xs,*}` (Init/NotationExtra.lean).

Conventions: Application binds tighter than ∪: `insert a (s.erase c) ∪ t` = (insert a (s.erase c)) ∪ t. For constraint sets, a union imposes both families of inequalities.

### G33 — `Fintype.piFinset`

Statements.lean:4375

Meaning: `Fintype.piFinset t` is the finset of all f with f i ∈ t i for every i: for ι = Fin k the box t 0 × ⋯ × t (k−1), with ∏ i, #(t i) elements.

Pinned by: `Fintype.mem_piFinset`: f ∈ piFinset t ↔ ∀ a, f a ∈ t a; `Fintype.card_piFinset`: #(piFinset s) = ∏ i, #(s i).

Conventions: Empty if some t i is empty. For k = 0 it is {the empty tuple} (`Fintype.piFinset_of_isEmpty`: piFinset s = univ).

### G37 — `f '' S` (`Set.image`)

Statements.lean:4386

Meaning: For `S : Set α`, `f '' S : Set β` is {f x | x ∈ S}; coinciding values merge, no injectivity is implied. A finset s is first coerced to `(s : Set α)`.

Pinned by: `Set.mem_image`: y ∈ f '' s ↔ ∃ x ∈ s, f x = y (by definition `Set.image f s := {f a | a ∈ s}`).

Conventions: `infixr:80`; application binds tighter, so `g a b '' S` is `(g a b) '' S`.

### G39 — `Disjoint`

Statements.lean:4397

Meaning: In a partial order with ⊥, `Disjoint a b` says every x with x ≤ a and x ≤ b satisfies x ≤ ⊥; for sets: no common element. Pairwise disjointness is written `∀ i i', i ≠ i' → Disjoint (S i) (S i')`.

Pinned by: definition `Disjoint a b := ∀ ⦃x⦄, x ≤ a → x ≤ b → x ≤ ⊥`; `Set.disjoint_iff_inter_eq_empty`: Disjoint s t ↔ s ∩ t = ∅.

Conventions: ∅ is disjoint from every set, including itself.

### G40 — `Set.BijOn`

Statements.lean:4408

Meaning: `Set.BijOn f s t`: f maps s into t, is injective on s, and hits every y ∈ t from s; i.e. f restricted to s is a bijection s → t. Values of f outside s are unconstrained.

Pinned by: definition `Set.BijOn f s t := MapsTo f s t ∧ InjOn f s ∧ SurjOn f s t` with `MapsTo f s t := ∀ ⦃x⦄, x ∈ s → f x ∈ t`, `InjOn f s := ∀ ⦃x₁⦄, x₁ ∈ s → ∀ ⦃x₂⦄, x₂ ∈ s → f x₁ = f x₂ → x₁ = x₂`, `SurjOn f s t := t ⊆ f '' s`.

Conventions: Finsets are coerced to sets, e.g. `(simplexS k N : Set (Fin k → ℕ))`.

### G41 — `∃!` (`ExistsUnique`)

Statements.lean:4421

Meaning: `∃! x, p x` holds iff some x satisfies p and every y satisfying p equals it. It binds one variable: `∃! m, A m ∧ B m` means exactly one m satisfies the conjunction.

Pinned by: definition `ExistsUnique p := ∃ x, p x ∧ ∀ y, p y → y = x`.

Conventions: `x ∈ P ↔ ∃! i, x ∈ S i` makes P the set of points in exactly one S i; it forces neither S i ⊆ P nor disjointness (points in two pieces are excluded from P). Over `Fin 0`, `∃! i, …` is false.

### G42 — `Set.Finite`, `Set.Nonempty` (and `Finset.Nonempty`)

Statements.lean:4433

Meaning: For `s : Set α`, `s.Finite` means ↥s is in bijection with some `Fin n`; `s.Nonempty` (for a Set or a Finset) means ∃ x, x ∈ s. A Finset is finite by construction.

Pinned by: `Set.Finite s := Finite s` (the subtype ↥s), with `class inductive Finite (α) | intro {n : ℕ} : α ≃ Fin n → Finite α`; `Set.Nonempty s := ∃ x, x ∈ s`; `Finset.Nonempty s := ∃ x : α, x ∈ s`.

Conventions: ∅ is finite and not nonempty.

### G47 — `Int.sign`

Statements.lean:4445

Meaning: sign z = 1, 0, −1 for z > 0, z = 0, z < 0; sign z · |z| = z.

Pinned by: definition `Int.sign`: ofNat (n+1) ↦ 1, ofNat 0 ↦ 0, negSucc _ ↦ −1; `Int.sign_mul_abs`: sign a * |a| = a.

Conventions: sign 0 = 0 (not 1). In this file the value is cast to ℂ.

### G48 — `Int.gcd`

Statements.lean:4454

Meaning: `Int.gcd m n` is the natural number Nat.gcd |m| |n|; Int.gcd m 0 = |m|, Int.gcd 0 0 = 0.

Pinned by: `Int.gcd_eq_natAbs_gcd_natAbs`: gcd m n = Nat.gcd m.natAbs n.natAbs (by rfl); `Int.isCoprime_iff_gcd_eq_one`: IsCoprime m n ↔ Int.gcd m n = 1.

Conventions: `Int.gcd a q = 1` is an equation in ℕ; signs of the arguments are ignored.

### G49 — `a ≡ b [ZMOD n]` (`Int.ModEq`)

Statements.lean:4465

Meaning: Congruence modulo n: equal Euclidean remainders a % n = b % n; equivalently n | b − a.

Pinned by: definition `Int.ModEq n a b := a % n = b % n`; `Int.modEq_iff_dvd`: a ≡ b [ZMOD n] ↔ n | b - a; `Int.modEq_zero_iff`: a ≡ b [ZMOD 0] ↔ a = b.

Conventions: Modulus 0 means equality; the sign of n is irrelevant (`Int.modEq_neg`). Precedence 50: `a * abar ≡ 1 [ZMOD q]` is Int.ModEq q (a * abar) 1. A ℕ modulus is cast: `[ZMOD (q : ℤ)]`.

### G51 — `Odd` and `Even`

Statements.lean:4477

Meaning: Odd n: n = 2m + 1 for some m of the same type; Even n: n = r + r for some r. On ℤ, Odd l ↔ l % 2 = 1 (so −1 is odd); on ℕ, Even L ↔ L % 2 = 0.

Pinned by: definitions `Odd a := ∃ k, a = 2 * k + 1`, `Even a := ∃ r, a = r + r`; `Int.odd_iff`: Odd n ↔ n % 2 = 1; `Nat.even_iff`: Even n ↔ n % 2 = 0; `Int.not_even_iff_odd`: ¬Even n ↔ Odd n.

Conventions: 0 is even and not odd.

### G52 — `Nat.choose` (also `n.choose k`)

Statements.lean:4488

Meaning: n.choose k = C(n, k) = n! / (k!·(n−k)!) for k ≤ n, and 0 for k > n (no error); defined by Pascal's rule.

Pinned by: definition: choose _ 0 = 1, choose 0 (k+1) = 0, choose (n+1) (k+1) = choose n k + choose n (k+1); `Nat.choose_eq_factorial_div_factorial` (k ≤ n): choose n k = n ! / (k ! * (n - k)!); `Nat.choose_eq_zero_of_lt`: n < k → choose n k = 0.

Conventions: Arguments are in ℕ, so truncated subtraction inside an argument is evaluated first.

### G55 — `Irrational`

Statements.lean:4501

Meaning: `Irrational x` (x : ℝ) means x ∉ ℚ; a hypothesis, not a computation.

Pinned by: definition `Irrational x := x ∉ Set.range ((↑) : ℚ → ℝ)`.

Conventions: For irrational α, `GenContFract.of α` never terminates (`GenContFract.terminates_iff_rat`), so every `(GenContFract.of α).partDens.get? i` is `some _` (G76).

### G56 — `x ^ y` with real exponent (`Real.rpow`)

Statements.lean:4511

Meaning: For real x and y, `x ^ y` is `Real.rpow x y`, the real part of (x : ℂ) ^ (y : ℂ); for x > 0 it is exp(y·log x) > 0. With a ℕ exponent (`t ^ 2`, `2 ^ n`, `Real.sqrt q ^ k` with k : ℕ), `^` is repeated multiplication (G09); the two agree on x ^ (n : ℝ).

Pinned by: `Real.rpow_def_of_pos`: 0 < x → x ^ y = exp (log x * y); `Real.rpow_natCast`: x ^ (n : ℝ) = x ^ n; `Real.rpow_neg`: 0 ≤ x → x ^ (-y) = $(x ^ y)^{-1}$.

Conventions: `Real.zero_rpow`: y ≠ 0 → 0 ^ y = 0; `Real.rpow_zero`: x ^ (0 : ℝ) = 1 for every x. For x < 0, x ^ y = exp (log x * y) * cos (y * π) (`Real.rpow_def_of_neg`; log x = log |x|), a convention, not a real root. `-(1 : ℝ) / 2` parses as (−1)/2.

### G57 — `Real.sqrt` (notation √x)

Statements.lean:4527

Meaning: For x ≥ 0, `Real.sqrt x` is the unique y ≥ 0 with y·y = x. It is `NNReal.sqrt (Real.toNNReal x)`, so a negative input is first replaced by 0.

Pinned by: `Real.sqrt_nonneg`: 0 ≤ √x; `Real.mul_self_sqrt`: 0 ≤ x → √x * √x = x; `Real.sqrt_eq_zero'`: √x = 0 ↔ x ≤ 0.

Conventions: √x = 0 for every x ≤ 0 (no error, no complex value); with a / 0 = 0 this gives √(a / 0) = 0. `Real.sqrt_eq_rpow`: √x = x ^ (1 / (2 : ℝ)) for all real x.

### G58 — `Real.exp`, `Complex.exp`, `Complex.I`, `Real.pi`

Statements.lean:4539

Meaning: `Complex.exp z` = Σ_{m≥0} z^m/m!; `Real.exp x` is its real part at x, the usual $e^x > 0$; `Complex.I` = ⟨0, 1⟩ = i; `Real.pi` = π, twice the zero of cos in [1, 2]. No junk values.

Pinned by: definitions `Complex.exp z := CauSeq.lim (exp' z)` (exp' z the partial sums Σ_{m<n} z^m / m!), `Real.exp x := (Complex.exp x).re`, `Complex.I := ⟨0, 1⟩`, `Real.pi := 2 * Classical.choose exists_cos_eq_zero`; `Complex.exp_mul_I`: exp (x * I) = cos x + sin x * I; `Complex.exp_two_pi_mul_I`: exp (2 * π * I) = 1.

Conventions: In `Complex.exp (2 * Real.pi * Complex.I * x)` with x : ℝ, `Real.pi` and x are cast to ℂ, so the argument is 2πix.

### G59 — `Real.log`

Statements.lean:4554

Meaning: For x > 0, `Real.log x` is the natural logarithm (inverse of `Real.exp`).

Pinned by: definition `log x := if x = 0 then 0 else expOrderIso.symm ⟨|x|, _⟩`; `Real.exp_log`: 0 < x → exp (log x) = x; `Real.log_zero`: log 0 = 0; `Real.log_abs`: log |x| = log x.

Conventions: Junk extensions: log 0 = 0 and log x = log |x| for x < 0.

### G60 — `Real.cot`

Statements.lean:4565

Meaning: `Real.cot x` = cos x / sin x for every real x (defined as the real part of the complex cotangent).

Pinned by: definitions `Real.cot x := (Complex.cot x).re`, `Complex.cot z := cos z / sin z`; `Real.cot_eq_cos_div_sin`: cot x = cos x / sin x (all x : ℝ).

Conventions: At x ∈ πℤ (sin x = 0) the value is cos x / 0 = 0 (`div_zero`); e.g. cot 0 = 0. No Mathlib lemma states this case separately.

### G62 — `Matrix.det` (`A.det`)

Statements.lean:4576

Meaning: For square `M : Matrix n n R` over a commutative ring, `M.det = ∑_σ sign(σ) ∏_i M (σ i) i` over all permutations σ of n; an integer for an integer matrix (`.natAbs` is its absolute value in ℕ).

Pinned by: `Matrix.det_apply`: M.det = ∑ σ : Perm n, Equiv.Perm.sign σ • ∏ i, M (σ i) i; `Matrix.det_mul`: det (M * N) = det M * det N.

Conventions: The empty 0×0 matrix has determinant 1 (`Matrix.det_isEmpty`).

### G63 — `Matrix.mulVec` (`A *ᵥ v`)

Statements.lean:4587

Meaning: For `A : Matrix m n α` and `v : n → α`, `A *ᵥ v` (= `A.mulVec v`) is the vector with i-th entry ∑_j A i j · v j.

Pinned by: `Matrix.mulVec_apply_eq_sum`: (M *ᵥ v) i = ∑ j : n, M i j * v j; `Matrix.mulVec_mulVec`: M *ᵥ N *ᵥ v = (M * N) *ᵥ v.

Conventions: `infixr:73` (scoped in `Matrix`): tighter than + (65), looser than application, so `u + Φ *ᵥ y` is u + Φy, `A i *ᵥ x + c i` is (A i)x + c i, and `M *ᵥ N *ᵥ v` is M(Nv).

### G67 — `Module.Basis ι R M`

Statements.lean:4599

Meaning: A basis b of the R-module M indexed by ι is a linear isomorphism `b.repr : M ≃ₗ[R] (ι →₀ R)`, with `b i := b.repr.symm (Finsupp.single i 1)`; the b i are linearly independent and span M, and `Module.Basis.mk` makes any such family a basis. No inner product or normalization is involved.

Pinned by: `Module.Basis.linearIndependent`: LinearIndependent R b; `Module.Basis.span_eq`: span R (range b) = ⊤; `Module.finrank_eq_card_basis`: (h : Basis ι R M) → finrank R M = Fintype.card ι.

Conventions: With ι = `Fin D` the vectors are indexed 0, …, D−1 and dim M = D. Over a nonzero ring they are pairwise distinct (`Module.Basis.injective`).

### G68 — `Module.finrank`

Statements.lean:4613

Meaning: `Module.finrank R M := Cardinal.toNat (Module.rank R M)`: dim_R M if finite, else 0.

Pinned by: `Module.finrank_of_not_finite`: ¬Module.Finite R M → finrank R M = 0; `Module.finrank_pi`: finrank R (ι → R) = Fintype.card ι.

Conventions: The junk value 0 is not relevant here (the spaces are finite-dimensional).

### G69 — `Submodule.span`, `⊤`, `Submodule`

Statements.lean:4624

Meaning: `Submodule.span R s` is the smallest submodule containing s: the finite linear combinations ∑ c_i v_i with v_i ∈ s. `⊤` is the whole module, so `Submodule.span R s = ⊤` means s spans. A `Submodule ℂ (X → ℂ)` is a complex linear subspace of the functions X → ℂ; ↥V is the type of its elements.

Pinned by: definition `Submodule.span R s := sInf {p | s ⊆ p}`; `Submodule.mem_span_set'`: m ∈ span R s ↔ ∃ (n : ℕ) (f : Fin n → R) (g : Fin n → s), ∑ i, f i • (g i : M) = m; `Submodule.eq_top_iff'`: p = ⊤ ↔ ∀ x, x ∈ p.

Conventions: `Submodule.span_empty`: span R ∅ = ⊥.

### G70 — `V →ₗ[ℂ] V →ₗ⋆[ℂ] ℂ` (sesquilinear forms)

Statements.lean:4637

Meaning: `M →ₗ[R] N` is the type of R-linear maps, `M →ₗ⋆[R] N` of conjugate-linear maps (f (c • y) = conj(c) • f y). Arrows associate to the right: `ip : V →ₗ[ℂ] V →ₗ⋆[ℂ] ℂ` is additive in each argument, ip (c • x) y = c · ip x y and ip x (c • y) = conj(c) · ip x y. Hermitian symmetry and positivity are not implied.

Pinned by: notations `M →ₗ[R] M₂ := LinearMap (RingHom.id R) M M₂`, `M →ₗ⋆[R] M₂ := LinearMap (starRingEnd R) M M₂`; `LinearMap.map_smul₂`: f (r • x) y = r • f x y; `LinearMap.map_smulₛₗ`: f (c • x) = σ c • f x, here σ = `starRingEnd ℂ`, complex conjugation (`starRingEnd_apply`).

Conventions: Same convention as the paper's ⟨f, g⟩: linear in f, conjugate-linear in g.

### G71 — `Summable`, `HasSum`, `∑' i, f i` (`tsum`)

Statements.lean:4653

Meaning: `HasSum f a` (default summation filter, used throughout) means the finite partial sums Σ_{b ∈ s} f b tend to a as the finite set s ⊆ β grows (no ordering of β is used). `Summable f` is ∃ a, HasSum f a, and `∑' i, f i` is that a if f is summable, else 0. For ℝ- or ℂ-valued f, Summable f ↔ Σ |f i| < ∞; so ∑' over ℤ is not the symmetric limit lim_N Σ_{|n| ≤ N}.

Pinned by: `HasSum f a L := Tendsto (fun s : Finset β ↦ ∑ b ∈ s, f b) L.filter (𝓝 a)` with `(unconditional β).filter = atTop`; `HasSum.tsum_eq`: HasSum f a → ∑' b, f b = a; `tsum_eq_zero_of_not_summable`: ¬Summable f → ∑' b, f b = 0; `summable_norm_iff` (finite-dimensional, e.g. ℂ): (Summable fun x => ‖f x‖) ↔ Summable f.

Conventions: Junk value: ∑' of a non-summable family is 0, so a bound on a ∑' is informative only when summability is known or asserted. A family with finite support is summable with ∑' the finite sum (`tsum_eq_sum`). No `∑'[L]` (non-default filter) occurs.

### G72 — `∫ x, f x` and `∫ x in a..b, f x` Statements.lean:4672

Meaning: `∫ t : ℝ, f t` is the Bochner integral with respect to `volume` (Lebesgue measure on ℝ): the Lebesgue integral over ℝ for integrable f. `∫ x in a..b, f x` is ∫_{(a,b]} f − ∫_{(b,a]} f: ∫_{(a,b]} f if a ≤ b, with a sign change when a and b are swapped.

Pinned by: `MeasureTheory.integral_undef`: ¬Integrable f μ → ∫ a, f a ∂μ = 0; definition `intervalIntegral f a b μ := (∫ x in Ioc a b, f x ∂μ) - ∫ x in Ioc b a, f x ∂μ`; `intervalIntegral.integral_undef`: ¬IntervalIntegrable f μ a b → ∫ x in a..b, f x ∂μ = 0; `Real.volume_Icc`: volume (Icc a b) = ofReal (b - a).

Conventions: Junk value 0 for a non-integrable integrand (e.g. ∫ t : ℝ, (1 : ℝ) = 0), so a hypothesis `∫ … ≤ Lam` would hold trivially for it. Integrability criteria: `Continuous.integrable_of_hasCompactSupport`: Continuous f → HasCompactSupport f → Integrable f μ; `Continuous.intervalIntegrable`: Continuous u → IntervalIntegrable u μ a b.

Source: Mathlib/MeasureTheory/Integral/{Bochner/Basic,IntervalIntegral/Basic}.lean, Mathlib/MeasureTheory/Measure/Lebesgue/Basic.lean, Mathlib/MeasureTheory/Function/LocallyIntegrable.lean.

### G73 — `deriv` Statements.lean:4695

Meaning: `deriv f x := fderiv 𝕜 f x 1` is f′(x) = lim_{h→0, h ≠ 0} (f(x+h) − f(x))/h when f is differentiable at x (here 𝕜 = ℝ, f : ℝ → ℂ, so h is real); `deriv (deriv U)` is U″ when U is twice differentiable.

Pinned by: `HasDerivAt.deriv`: HasDerivAt f f' x → deriv f x = f'; `deriv_zero_of_not_differentiableAt`: ¬DifferentiableAt 𝕜 f x → deriv f x = 0; `hasDerivAt_iff_tendsto_slope`: HasDerivAt f f' x ↔ Tendsto (slope f x) (𝓝[≠] x) (𝓝 f').

Conventions: Junk value 0 where f is not differentiable.

### G75 — `fwdDiff` Statements.lean:4708

Meaning: For f : M → G (M an additive commutative monoid, G an additive commutative group) and a step h : M, `fwdDiff h f` (also Δ_[h] f) is y ↦ f (y + h) − f y.

Pinned by: definition `fwdDiff h f := fun n ↦ f (n + h) - f n` (no separate apply lemma).

### G76 — `GenContFract.of` and `.h`, `.partDens`, `.nums`, `.dens` (with `get?`, `getD`) Statements.lean:4717

Meaning: `GenContFract.of α` is the regular continued fraction α = [a₀; a₁, a₂, …] of the floor algorithm: a head `h = ⌊α⌋ = a₀` (a real) and a possibly finite sequence of pairs (partial numerator 1, partial denominator a_{i+1}), i = 0, 1, …. `nums s`, `dens s` (s ≥ 0) are the convergents' numerators and denominators p_s, q_s, p_s/q_s = [a₀; a₁, …, a_s]: p₀ = a₀, q₀ = 1, p₁ = a₁a₀ + 1, q₁ = a₁, p_{s+2} = a_{s+2}p_{s+1} + p_s, q_{s+2} = a_{s+2}q_{s+1} + q_s.

Pinned by: `GenContFract.of_h_eq_floor`: (of v).h = ⌊v⌋; `GenContFract.get?_of_eq_some_of_succ_get?_intFractPair_stream`: IntFractPair.stream v (n + 1) = some ifp → (of v).s.get? n = some ⟨1, ifp.b⟩; `GenContFract.exists_int_eq_of_partDen`: (of v).partDens.get? n = some b → ∃ z : ℤ, b = z; `GenContFract.of_one_le_get?_partDen` (same hypothesis): 1 ≤ b; `GenContFract.zeroth_num_eq_h`, `zeroth_den_eq_one`, `first_num_eq`, `first_den_eq`, `nums_recurrence`, `dens_recurrence` (the formulas above, with partial numerators 1 by `GenContFract.of_partNum_eq_one`); `GenContFract.terminatedAt_iff_partDen_none`: g.TerminatedAt n ↔ g.partDens.get? n = none; `GenContFract.terminates_iff_rat`: (of v).Terminates ↔ ∃ q : ℚ, v = q; `GenContFract.of_convergence`: Tendsto (of v).convs atTop (𝓝 v).

Conventions: Index shift: `partDens.get? i` is `some a_{i+1}` (i ≥ 0), and a₀ = ⌊α⌋ is `h`, so a bound on all `partDens` entries bounds a₁, a₂, … but not a₀. `get?` returns `none` after a rational α's expansion ends, never for irrational α. `Option.getD o d` is x if o = some x and d if o = none. After termination `nums`/`dens` stay constant (`GenContFract.nums_stable_of_terminated`). The values are integral reals with dens s ≥ 1 (`zeroth_den_eq_one`, `GenContFract.of_den_mono`). Coprimality of p_s, q_s follows from `GenContFract.determinant`: g.nums n * g.dens (n + 1) - g.dens n * g.nums (n + 1) = ∏ i ∈ Finset.range (n + 1), -(g.partNums.get? i).getD 0, which is (−1)^{n+1} here.

Terms used only inside definition bodies, one line each:

- G17 — `Fin.castSucc`: Fin n → Fin (n+1), i ↦ i (same value) (`Fin.val_castSucc`).
- G22 — `toLex`: α ×ₗ β is α × β ordered lexicographically; toLex (a, b) is (a, b) in that order (`Prod.Lex.toLex_lt_toLex`).
- G27 — `Finset.image`: s.image f = {f x | x ∈ s}; equal values merge, so #(s.image f) ≤ #s (`Finset.mem_image`).

- G30 — `Finset.inf'`: s.inf' H f is the minimum of f over s, H proving s nonempty (`Finset.inf'`).
- G34 — `Finset.piAntidiag s n`: all f : ι → μ supported in s with ∑_{i ∈ s} f i = n (`Finset.mem_piAntidiag`).
- G35 — ↥s = {x // x ∈ s}: elements of s paired with membership proofs (`SetLike.coe_mem`).
- G36 — `Set.Icc a b` = [a, b], `Set.Ico a b` = [a, b): Sets, not Finsets (`Set.mem_Icc`).
- G38 — `⋃ i, S i` = {x | ∃ i, x ∈ S i} (`Set.mem_iUnion`).
- G43 — `Function.support f` = {x | f x ≠ 0}, not its closure (`Function.support_subset_iff'`).
- G44 — `Nat.find h`: the least n : ℕ with p n, for h : ∃ n, p n (`Nat.find_spec`).
- G45 — `Int.toNat z`: z if z ≥ 0, and 0 if z < 0.
- G46 — `Int.natAbs z`: |z| as a natural number (`Int.natCast_natAbs`).
- G50 — ⌈x⌉: the least integer ≥ x; `round x`: the nearest integer, ties rounded up (`Int.ceil_le`).
- G53 — `Nat.multinomial s f` = (∑_{i∈s} f i)! / ∏_{i∈s} (f i)!, an exact quotient ≥ 1 (`Nat.multinomial_spec`).
- G54 — `Nat.factorial n` = n! = 1·2⋯n, with 0! = 1.
- G61 — `starRingEnd ℂ`: complex conjugation as a ring homomorphism ℂ → ℂ (`starRingEnd_apply`).
- G64 — `Aᵀ` (`Matrix.transpose`): $A^T$ i j = A j i (`Matrix.transpose_apply`).
- G65 — `Matrix.diagonal d`: diagonal matrix with entries d i; `A.IsDiag`: off-diagonal entries 0 (`Matrix.diagonal_apply`).
- G66 — `Matrix.fromBlocks`: block matrix [[A, B], [C, D]] on sum indices; `reindex` relabels rows and columns (`Matrix.fromBlocks_apply₁₁`).
- G74 — `ContDiff ℝ ((⊤ : ℕ∞) : WithTop ℕ∞) U`: U is C^∞; analyticity (ω) is not required.
- G77 — `SimpleGraph.fromRel r`: edge a–b iff a ≠ b and (r a b or r b a) (`SimpleGraph.fromRel_adj`).
- G78 — `SimpleGraph.connectedComponentMk`: G.connectedComponentMk v is the connected component of G containing v (`SimpleGraph.ConnectedComponent.eq`).

# Alphabetical definition index

Statements.lean:4793

Search for the heading `### D…` to jump to a body (scope: see the file header). `Paper.definition3Weak` is D025w. Library names are not indexed (glossary G15–G78 above; G01–G14 before Theorem 1).

- `AllDegreesProof.faceLift` (D042): Paper context / faces
- `AllDegreesProof.faceScale` (D043): Paper context / faces
- `BalancedDecompData` (D060): Paper context / decomposition
- `IsBalancedPair` (D044): Paper context / balanced
- `IsBalancedWrt` (D059): Paper context / components
- `IsDecomposition` (D073): Paper context / transitions
- `IsSmoothCutoff` (D045): Paper context / cutoff
- `IsTransition` (D072): Paper context / transitions
- `IsUNmap` (D029): Paper context / simplex
- `Paper.definition3` (D025): Definition 3 / Paper context / affine
- `Paper.definition3Weak` (D025w): Paper context / affine
- `Piece` (D061): Paper context / pieces
- `Piece.IsMixed` (D067): Paper context / pieces
- `Piece.index` (D071): Paper context / pieces
- `Piece.longCoords` (D066): Paper context / pieces
- `Piece.longPart` (D084): Paper context / split
- `Piece.minSide` (D064): Paper context / pieces
- `Piece.mixedIneqs` (D068): Paper context / pieces
- `Piece.nLong` (D069): Paper context / pieces
- `Piece.nMix` (D070): Paper context / pieces
- `Piece.prod` (D078): Paper context / products
- `Piece.quad` (D062): Paper context / pieces
- `Piece.shortCoords` (D065): Paper context / pieces
- `Piece.shortPart` (D083): Paper context / split
- `Piece.side` (D063): Paper context / pieces
- `Piece.splitLambda` (D086): Paper context / split
- `Piece.splitMinusW` (D088): Paper context / split
- `Piece.splitPlusW` (D087): Paper context / split
- `PolySpaceH` (D003): Paper context / polynomials
- `PropUNmap` (D032): Paper context / basis
- `ThmBox` (D031): Paper context / basis
- `UniformBoundUnit.BddPartQuot` (D021): Paper context / fractions
- `UniformBoundUnit.PropUNmap` (D034): Paper context / basis
- `UniformBoundUnit.ThmBox` (D033): Paper context / basis
- `UniformBoundUnit.boxLenZ` (D019): Paper context / geometry
- `UniformBoundUnit.boxSectZ` (D020): Paper context / geometry
- `UniformBoundUnit.boxVolZ` (D018): Paper context / geometry
- `UniformBoundUnit.cSphere` (D005): Paper context / polynomials
- `UniformBoundUnit.latticePtsZ` (D017): Paper context / geometry
- `basisPhi` (D030): Paper context / basis
- `blockDiagMat` (D077): Paper context / products

- `boxB` (D028): Paper context / simplex
- `boxConstraints` (D082): Paper context / split
- `boxLen` (D013): Paper context / geometry
- `boxMinAffine` (D085): Paper context / split
- `boxSect` (D014): Paper context / geometry
- `boxSide` (D011): Paper context / geometry
- `boxVol` (D012): Paper context / geometry
- `boxZ` (D010): Paper context / geometry
- `constraintGraph` (D058): Paper context / graph
- `constraintRho` (D057): Paper context / graph
- `coordVec` (D081): Paper context / split
- `degIdx` (D002): Paper context / polynomials
- `expTwoPiI` (D001): Paper context / conventions
- `foldCoord` (D091): Paper context / fold
- `foldInv` (D096): Paper context / fold
- `foldMap` (D092): Paper context / fold
- `foldMapGlobal` (D093): Paper context / fold
- `foldPrefixSum` (D089): Paper context / fold
- `foldSourcePiece` (D090): Paper context / fold
- `foldTargetBase` (D094): Paper context / fold
- `foldTargetPiece` (D095): Paper context / fold
- `gaussPhasePhi` (D054): Paper context / gaussphase
- `hatEta` (D038): Paper context / fourier
- `lcmUpTo` (D026): Paper context / simplex
- `matInftyNorm` (D023): Paper context / affine
- `multiwallT` (D047): Paper context / cutoff
- `multiwallTerm` (D046): Paper context / cutoff
- `oplusConstraints` (D076): Paper context / products
- `oscIntegralW` (D053): Paper context / integrals
- `padLeftVec` (D074): Paper context / products
- `padRightVec` (D075): Paper context / products
- `pieceAffineImage` (D022): Paper context / affine
- `poissonH` (D036): Paper context / poisson
- `poissonQ` (D035): Paper context / poisson
- `poissonQLoc` (D039): Paper context / fourier
- `poissonRho` (D037): Paper context / poisson
- `polyBoxZ` (D015): Paper context / geometry
- `polyEval` (D004): Paper context / polynomials
- `polyInner` (D006): Paper context / pairing
- `polyP` (D008): Paper context / geometry
- `polyPZ` (D007): Paper context / geometry
- `quadExpSum` (D016): Paper context / geometry
- `quadGaussSum` (D052): Paper context / integrals
- `quadPhase` (D051): Paper context / integrals
- `restrictVec` (D080): Paper context / split
- `seqFourier` (D040): Paper context / fourier

- `sgnCoeff` (D049): Paper context / sign
- `sgnCotWeight` (D048): Paper context / sign
- `sgnIndex` (D050): Paper context / sign
- `shiftedBasisPhi` (D041): Paper context / shifted
- `sigmaPieceIndexEquiv` (D079): Paper context / products
- `simplexS` (D027): Paper context / simplex
- `transitionSize` (D024): Paper context / affine
- `vecSupport` (D009): Paper context / geometry
- `weylProgIco` (D055): Paper context / weyl
- `weylProgSum` (D056): Paper context / weyl

Paper-numbered aliases in `Compatibility.lean` (not shown; each is `abbrev name := @_root_.target`, so it adds nothing to its target):

- `Paper.definition13` = `IsBalancedPair` (D044)
- `Paper.definition20` = `IsBalancedWrt` (D059)
- `Paper.definition22` = `Piece` (D061)
- `Paper.Definition22.minimumSide` = `Piece.minSide` (D064)
- `Paper.Definition22.polyhedron` = `polyPZ` (D007)
- `Paper.Definition22.quadraticForm` = `Piece.quad` (D062)
- `Paper.Definition22.side` = `Piece.side` (D063)
- `Paper.definition23` = `IsTransition` (D072)
- `Paper.Definition23.affineImage` = `pieceAffineImage` (D022)
- `Paper.Definition23.decomposition` = `IsDecomposition` (D073)
- `Paper.Definition23.size` = `transitionSize` (D024)
- `Paper.Definition3.simplexBoxAdmissibleMap` = `IsUNmap` (D029)
- `Paper.definition8` = `shiftedBasisPhi` (D041)
- `Paper.Lemma11.lift` = `AllDegreesProof.faceLift` (D042)
- `Paper.Lemma11.scale` = `AllDegreesProof.faceScale` (D043)
- `Paper.Lemma16.coefficient` = `sgnCoeff` (D049)
- `Paper.Lemma16.indexSet` = `sgnIndex` (D050)
- `Paper.Lemma24.productPiece` = `Piece.prod` (D078)
- `Paper.Proposition9.indexEquiv` = `AllDegreesProof.goodEquiv` (`Core.lean:3379`; not shown, used only by this alias)

LEAN · END OF FILE line 4923

```
4923 end PaperReview
```